QR AI LAB

前端 × 後端實際串接測試報告

Next.js 16 前端 → FastAPI 後端 → 排程器（Redis 事件、subprocess runner）→ trainer，在單機沙盒以 SQLite 跑完整流程所得的 124 張截圖。

日期	2026年10月16日
帳號	admin / admin1234（管理員）、user / user1234（一般使用者）
資料	全部為合成資料：WAT 良率表（480 lot）、SEM 風格影像（golden 40 張、調機機台 16 張）、OCD 光譜（264 片 wafer × 200 波長）
環境	backend: SQLite、SCHEDULER_EMBEDDED=true（排程器與 API 同一程序，無 Redis）、JOB_RUNNER=auto → subprocess（沙盒沒有 systemd user manager；伺服器上會是 systemd cgroup）；frontend: next build + next start；瀏覽器: Chromium（Playwright）
結果	所有流程完成；訓練 / 推論任務皆 succeeded；強制中斷流程正確

目錄

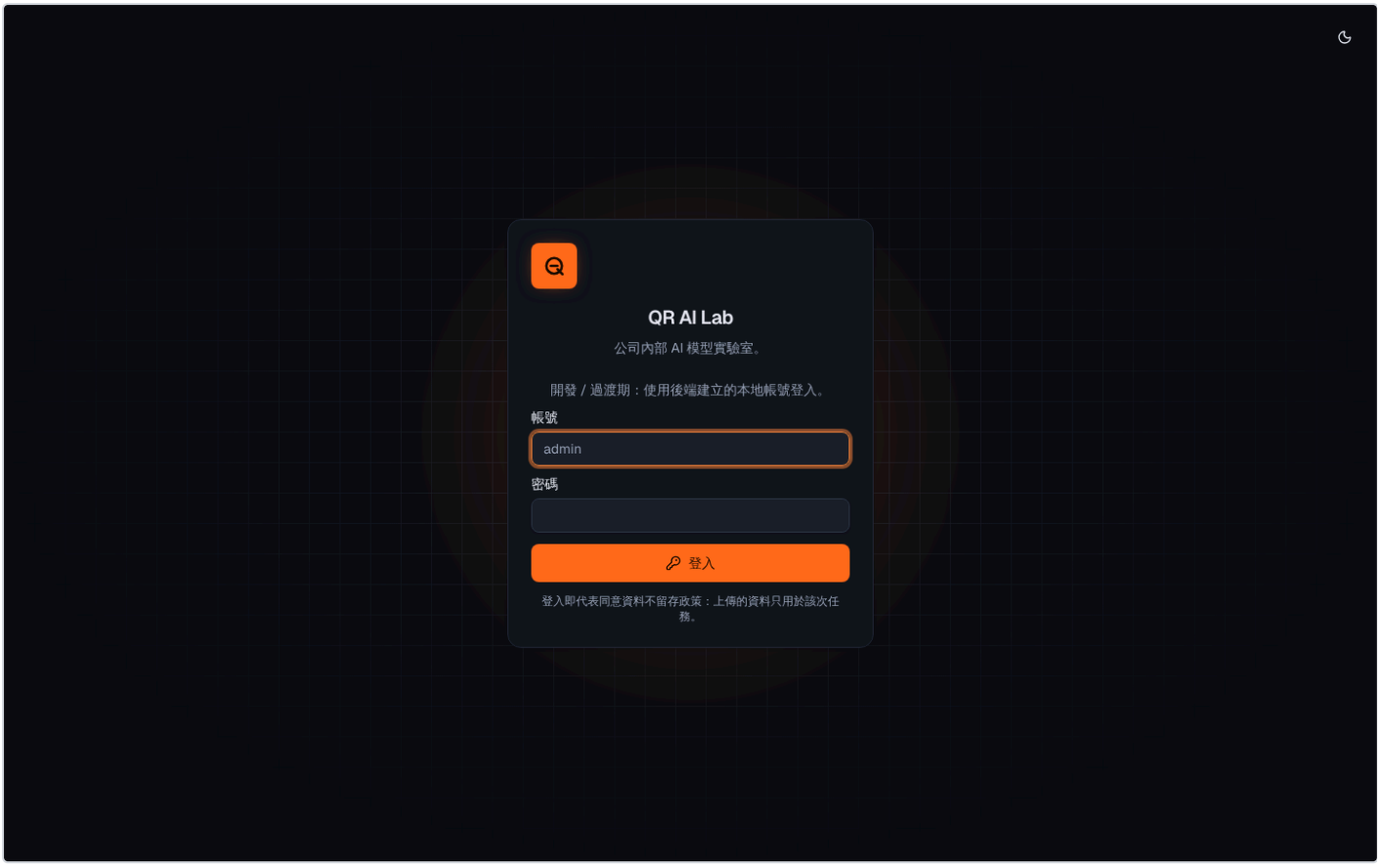
1. 一般使用者：登入、專案申請 8 張
2. 管理員：審核、配額、總資源池 11 張
2b. 管理員：模型庫（伺服器不連外） 6 張
3. 專案：成員、資料來源、設定 8 張
4. 訓練任務：上傳 → 前處理預覽 → AutoML → 即時 log → 產物 17 張
5. 推論任務：直接輸入 → 結果預覽與下載 4 張
6. 影像品質基準（golden tool vs 調機機台） 7 張
7. 光譜鑑別器（Masked AutoEncoder） 4 張
8. 討論區（問題單） 6 張
9. 管理員：佇列、強制中斷、重新執行、稽核 10 張
10. 三種系統角色：superadmin / admin / user 11 張
11. 資源使用率歷史 3 張
12. 資料前處理服務（任務模式、不保存） 6 張
12b. 資料分析服務 5 張
13. Light mode 與手機版 9 張

14. 登出 2 張
15. Keycloak 登入（AUTH_MODE=keycloak） 7 張

1. 一般使用者：登入、專案申請

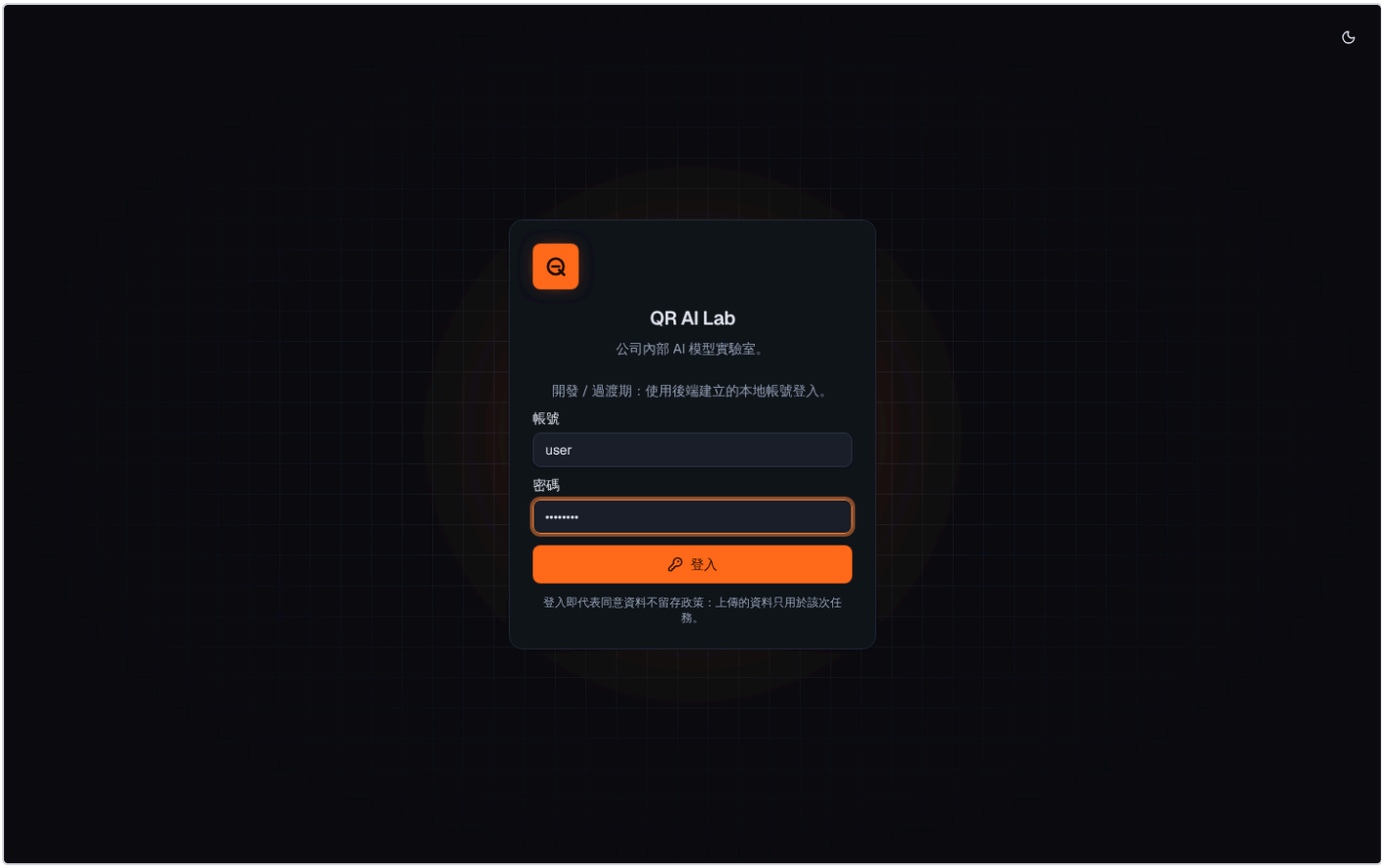
AUTH_MODE=local 的登入頁（superadmin / admin / user 三個本地帳號）→ user 填寫結構化申請（題目、資料類型 × 任務、資源需求）→ 送出 → 在討論串留言。

01 登入頁（AUTH_MODE=local）：開發環境用後端建立的本地帳號 superadmin / admin / user；正式環境改為 Keycloak 按鈕 dark



02 以一般使用者 user 登入

dark



03 一般使用者首頁：尚無專案、任務與申請（總資源池來自 backend /capacity）

dark



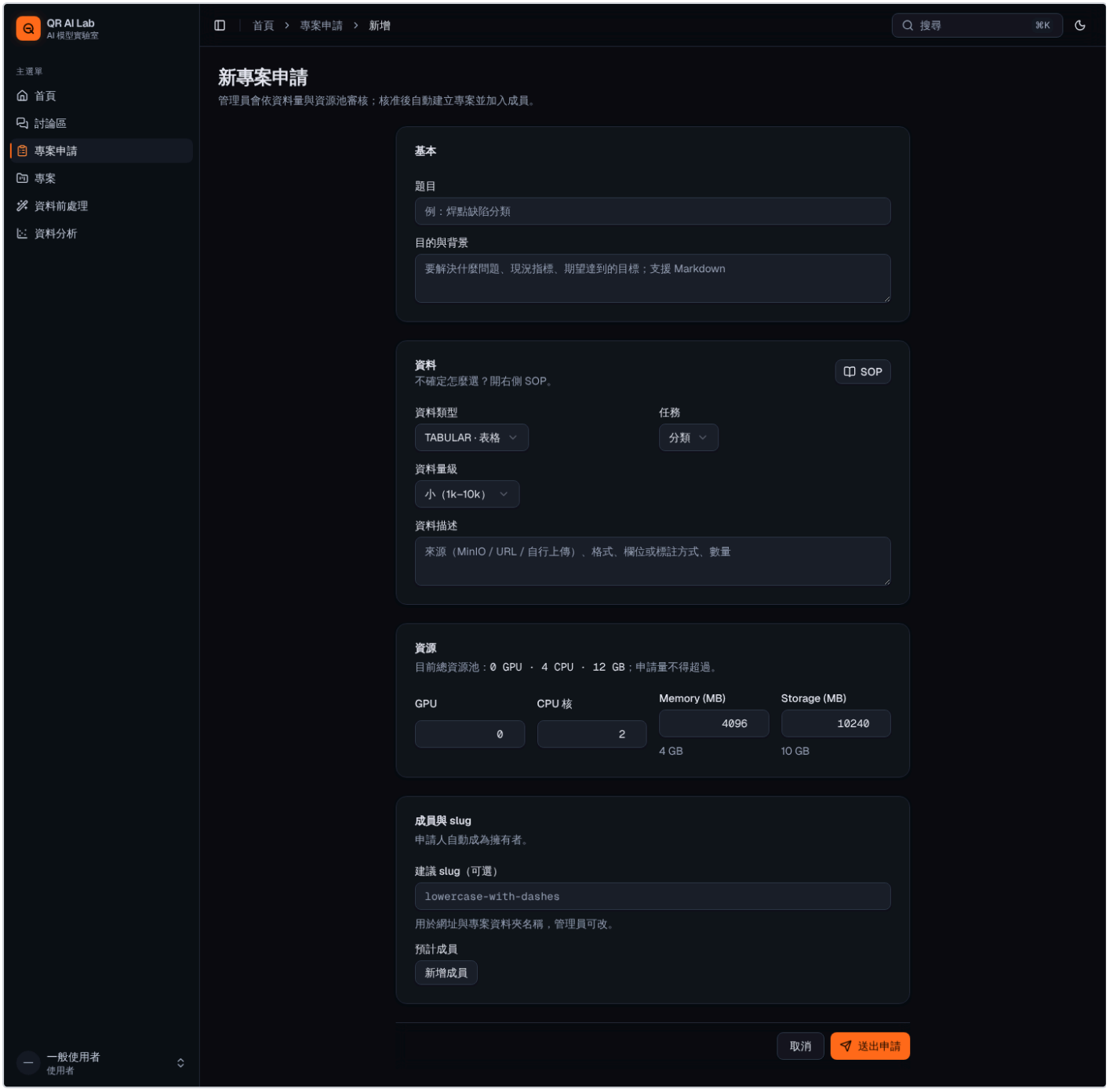
04 專案申請列表（空狀態）

dark

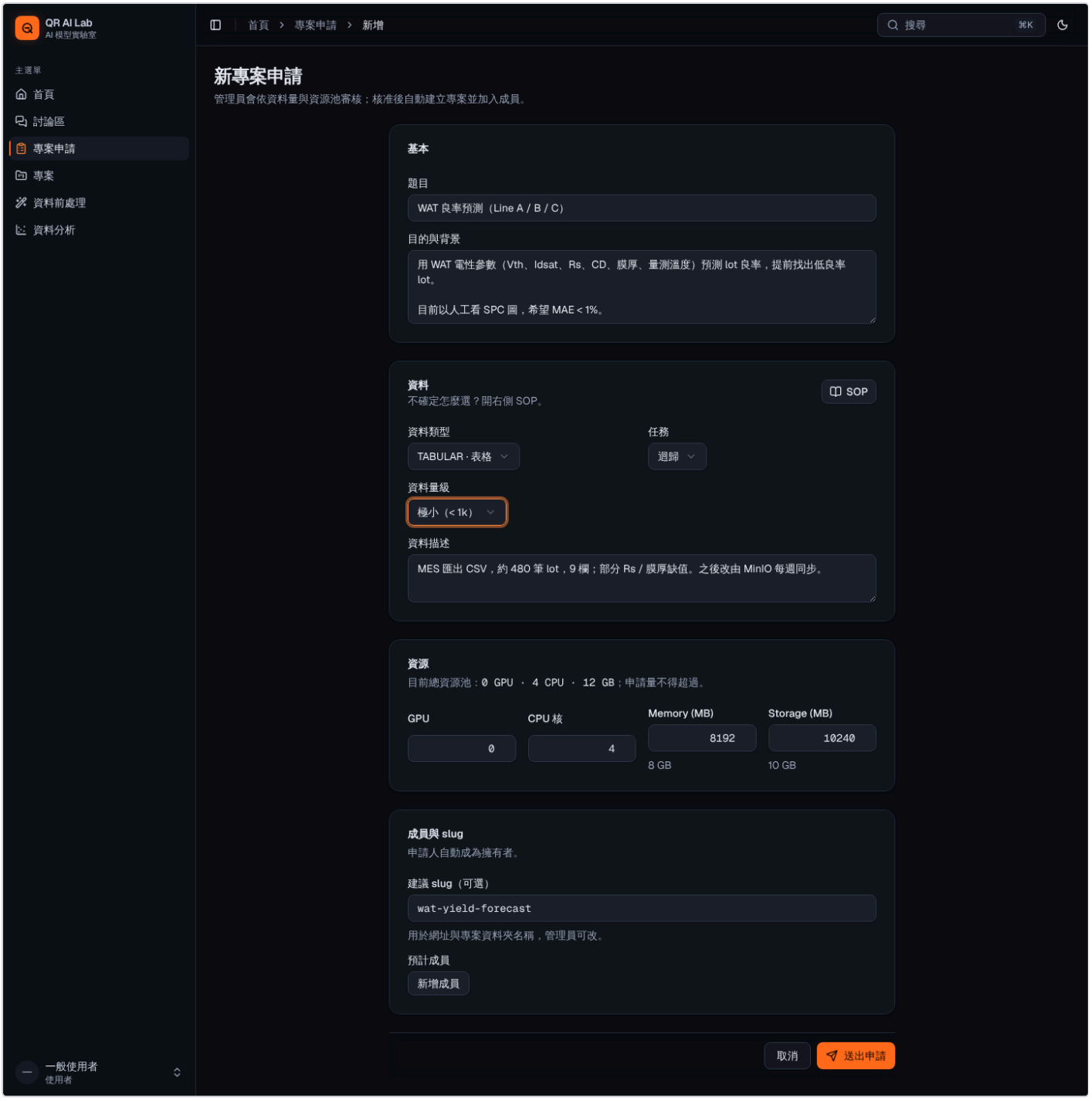


05 新申請表單（結構化：題目、資料類型、任務、資料規模、資源需求、成員）

dark



06 填好的申請：表格 × 迴歸、0 GPU / 4 CPU / 8 GB、建議 slug dark



07 送出後的申請詳情（狀態 submitted，申請人可撤回或留言）

dark



08 申請討論串：申請人留言

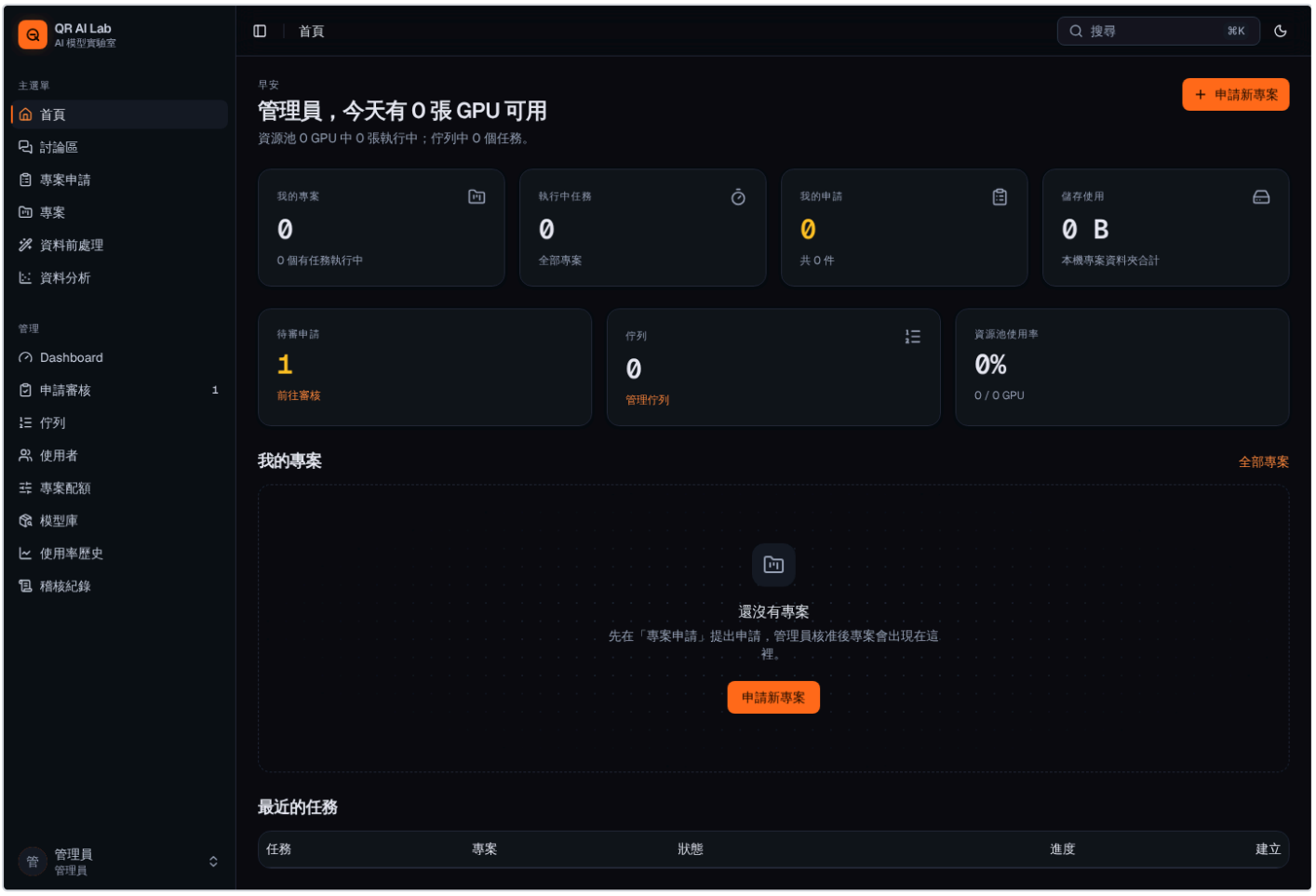
dark



2. 管理員：審核、配額、總資源池

admin 登入 → 開始審核 → 核准並指定 slug / 配額（受總資源池限制）→ 專案自動建立；另兩件申請由 API 核准。總資源池只有 superadmin 能改：GPU 卡片、依顯存共用 GPU 開關、確認對話框與稽核。

09 管理員首頁：側欄顯示待審申請數、全站任務與資源池 dark



10 管理員：申請審核列表

dark



11 管理員看申請詳情：右側審核面板

dark



12 狀態變為 in_review，出現核准 / 退回按鈕

dark



13 核准對話框：指定 slug 與配額（GPU / CPU / Memory / Storage），不得超過總資源池

dark



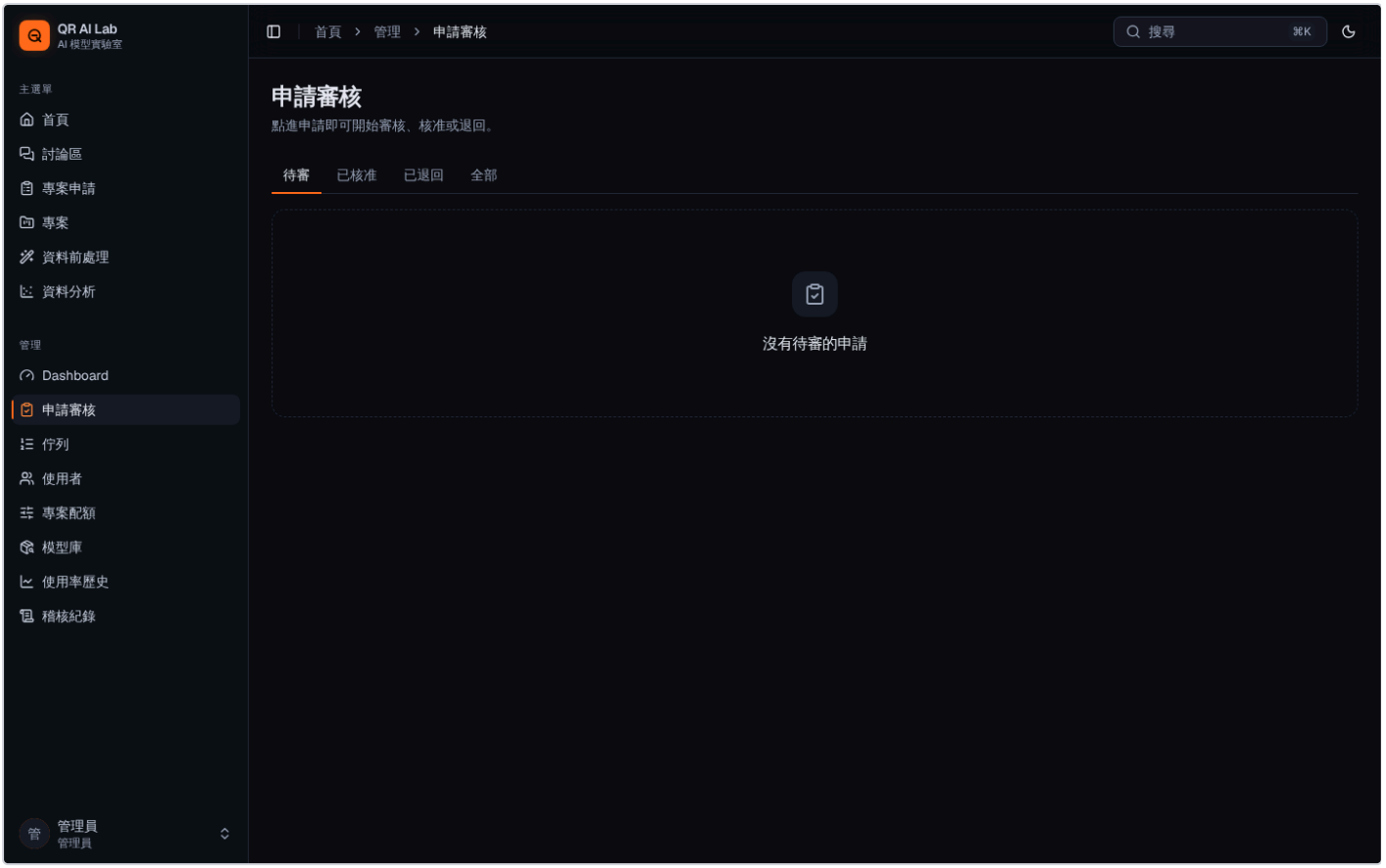
14 核准後：狀態 approved、專案已自動建立並連結

dark



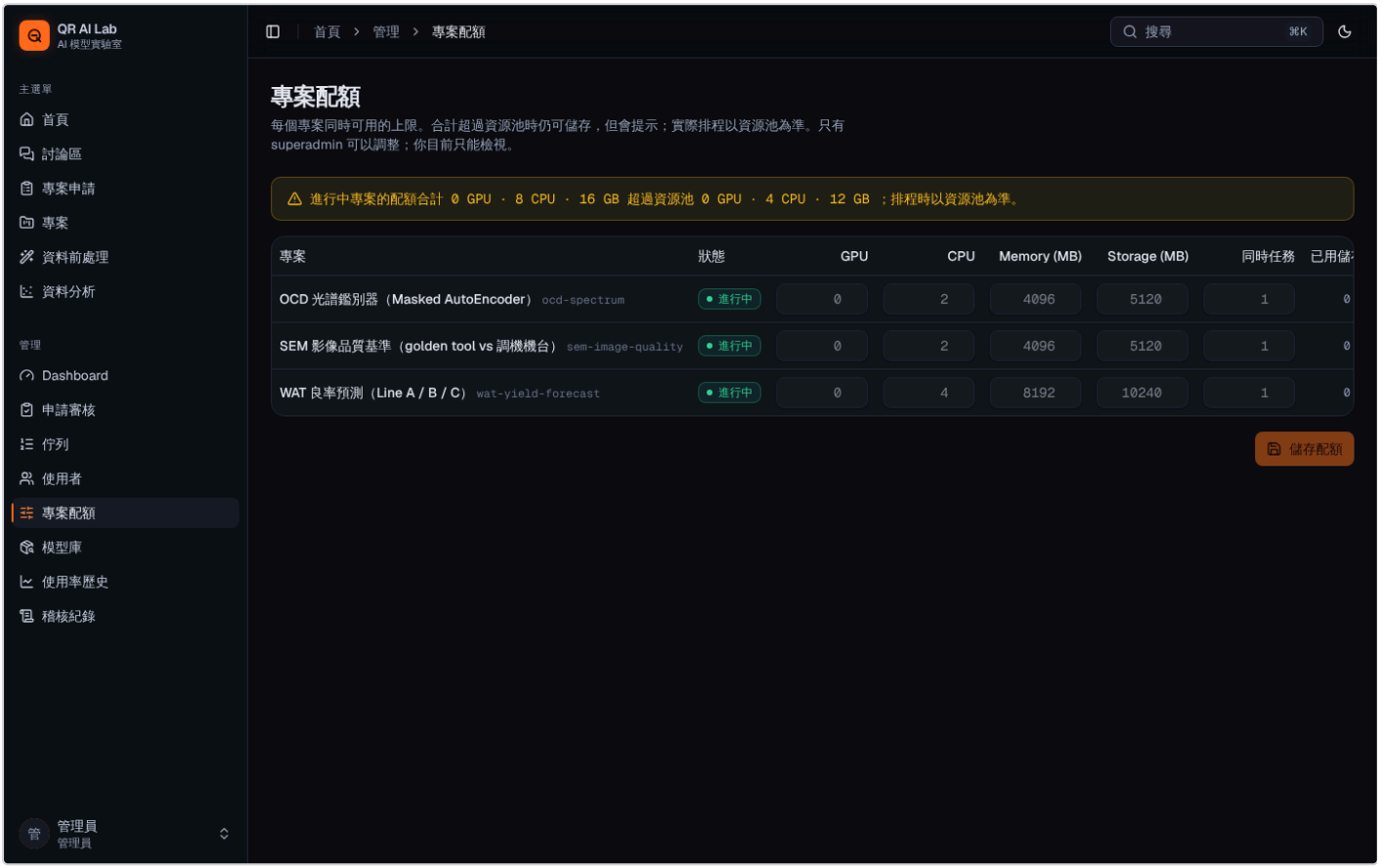
15 審核列表：三件申請都已核准

dark

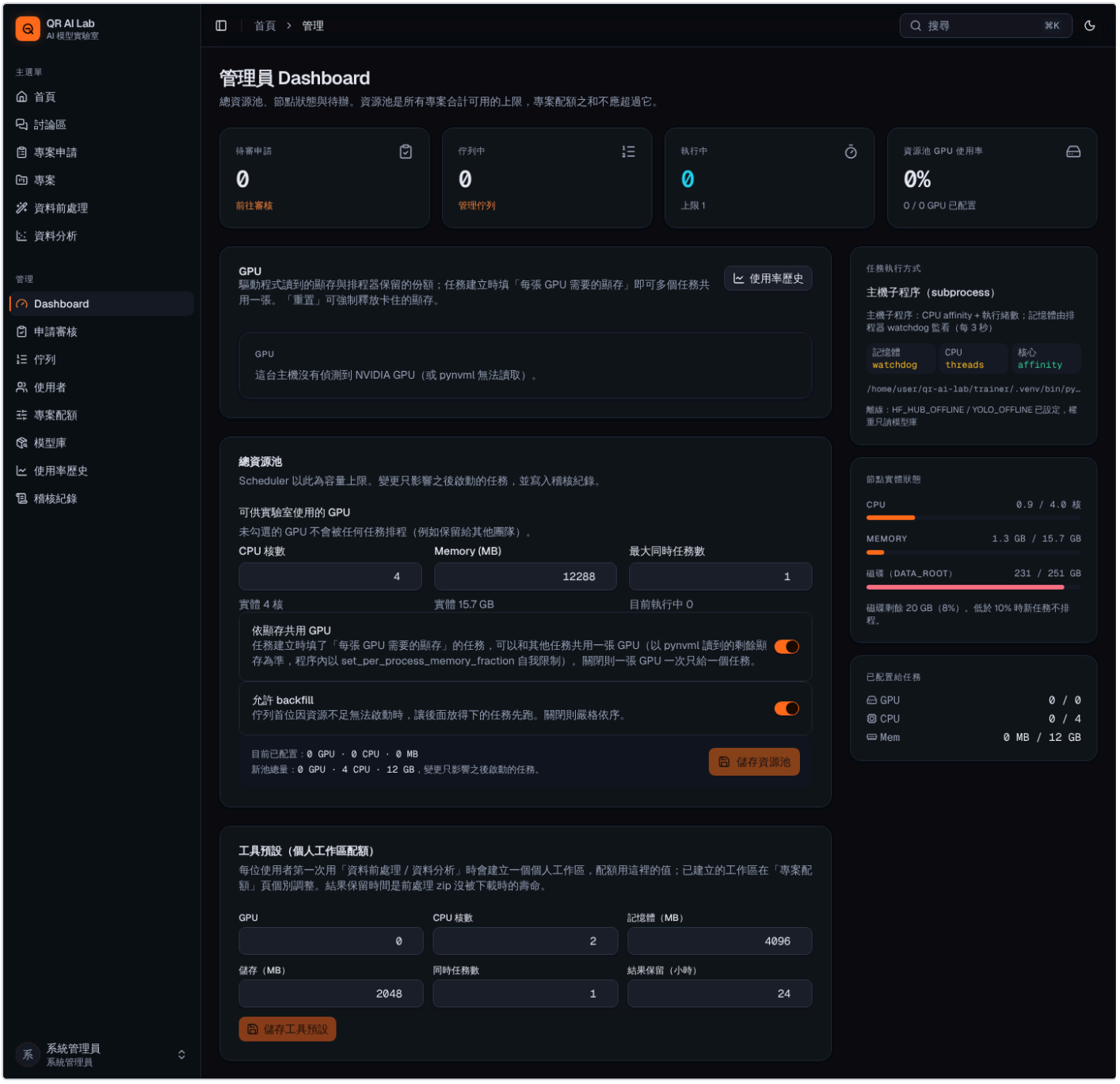


16 管理員：各專案配額一覽（合計超過資源池時提示；排程仍以資源池為準）

dark



17 superadmin 的 Dashboard：GPU 卡片、總資源池設定（所有專案合計可用的 GPU / CPU / Memory / 同時任務數、依顯存共用 GPU）、節點狀態、任務執行方式（runner） dark

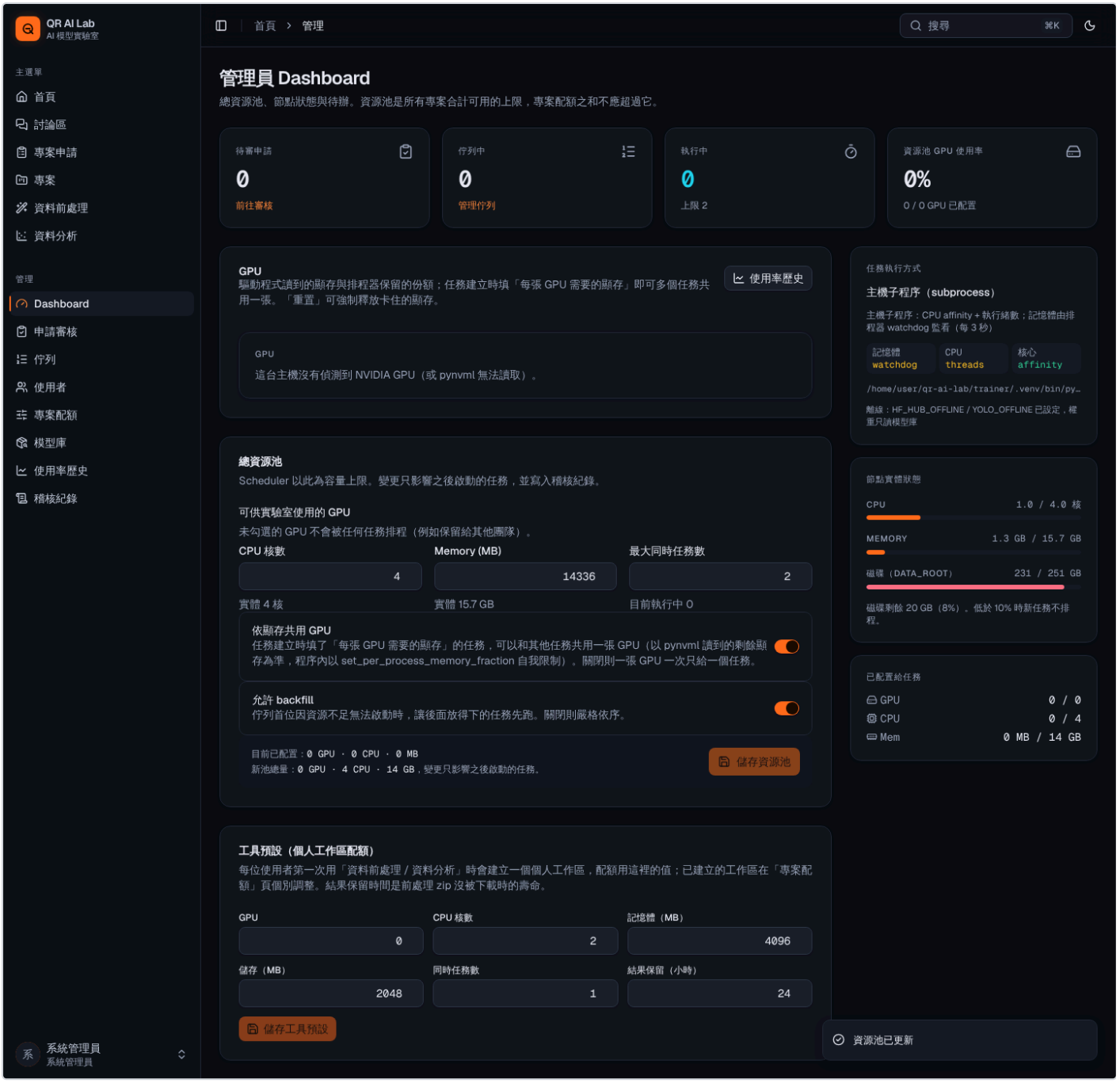


18 更新總資源池前的確認（寫入 audit_logs）

dark



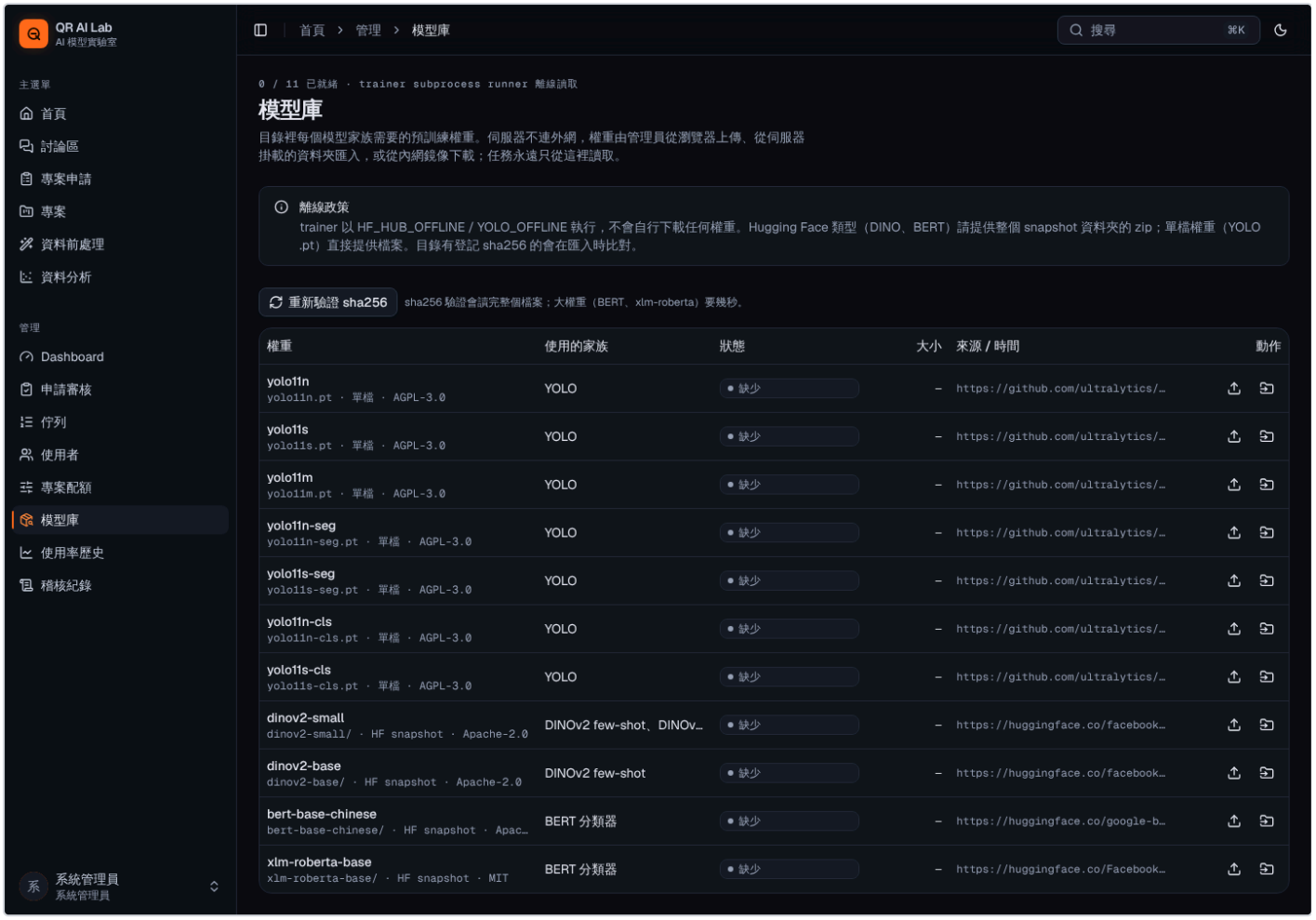
19 總資源池已更新：14 GB、最多 2 個同時任務（寫入 audit_logs） dark



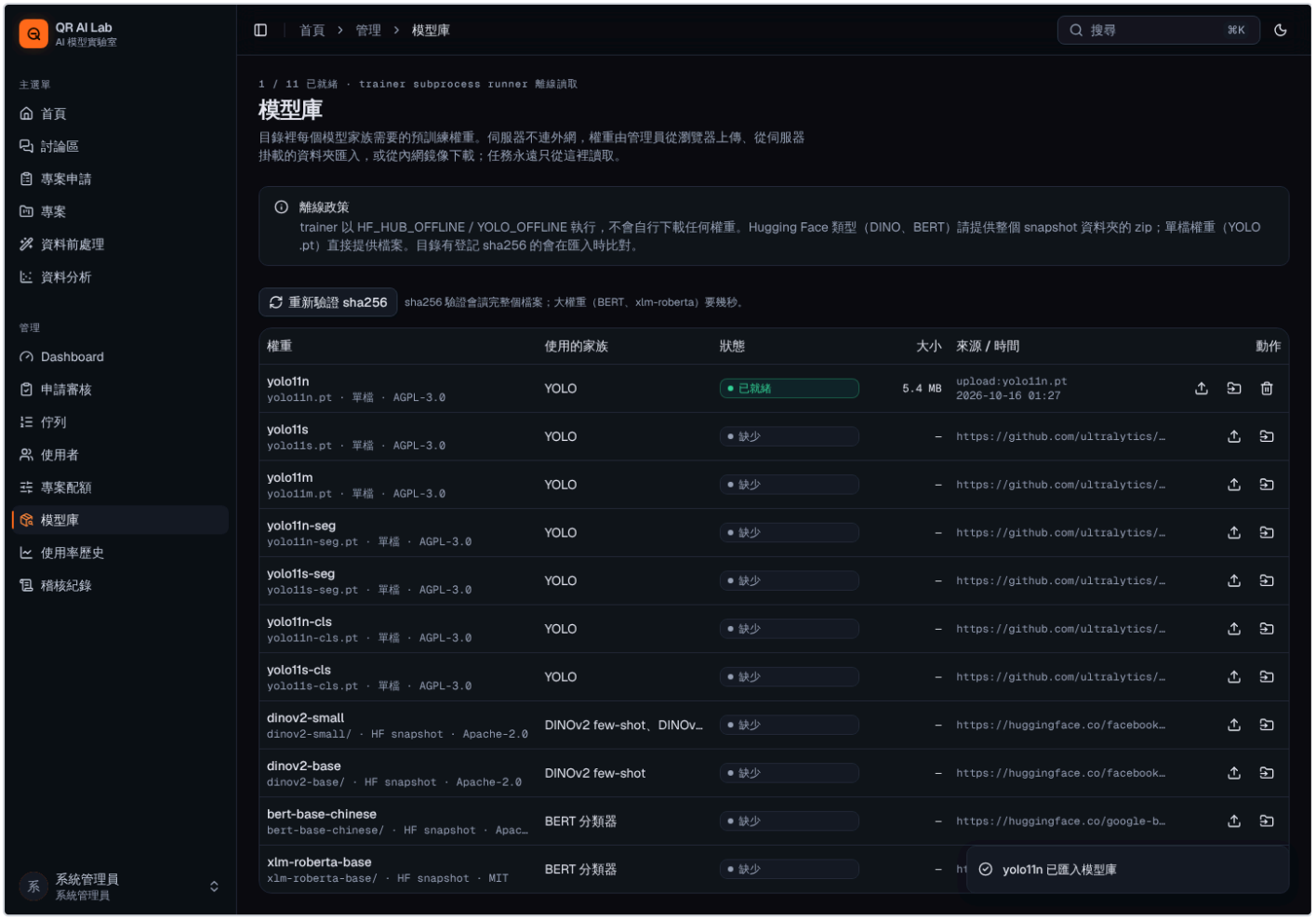
2b. 管理員：模型庫（伺服器不連外）

三種把預訓練權重帶進模型庫的方式：瀏覽器上傳、伺服器路徑（NAS / USB）、內網 URL（公司鏡像）；sha256 驗證與稽核。trainer 永遠離線，只讀模型庫。

20 模型庫（superadmin 才能上傳 / 匯入 / 移除）：目錄需要的預訓練權重與狀態；伺服器不連外，權重由管理員帶進來 dark

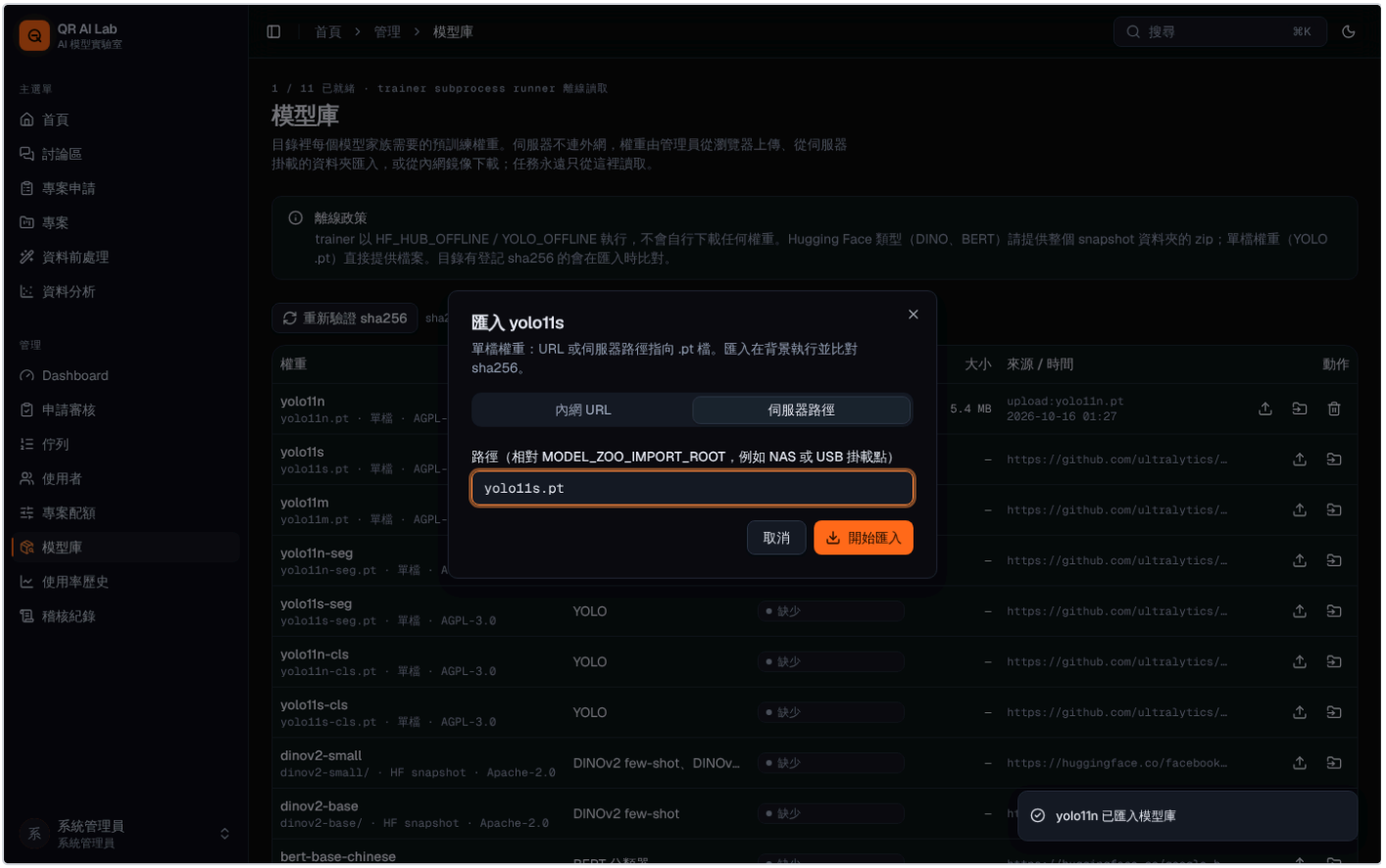


21 從瀏覽器上傳 yolo11n.pt：寫進 MODEL_ZOO_ROOT、sha256 記進 manifest.json，狀態「已就緒」 dark



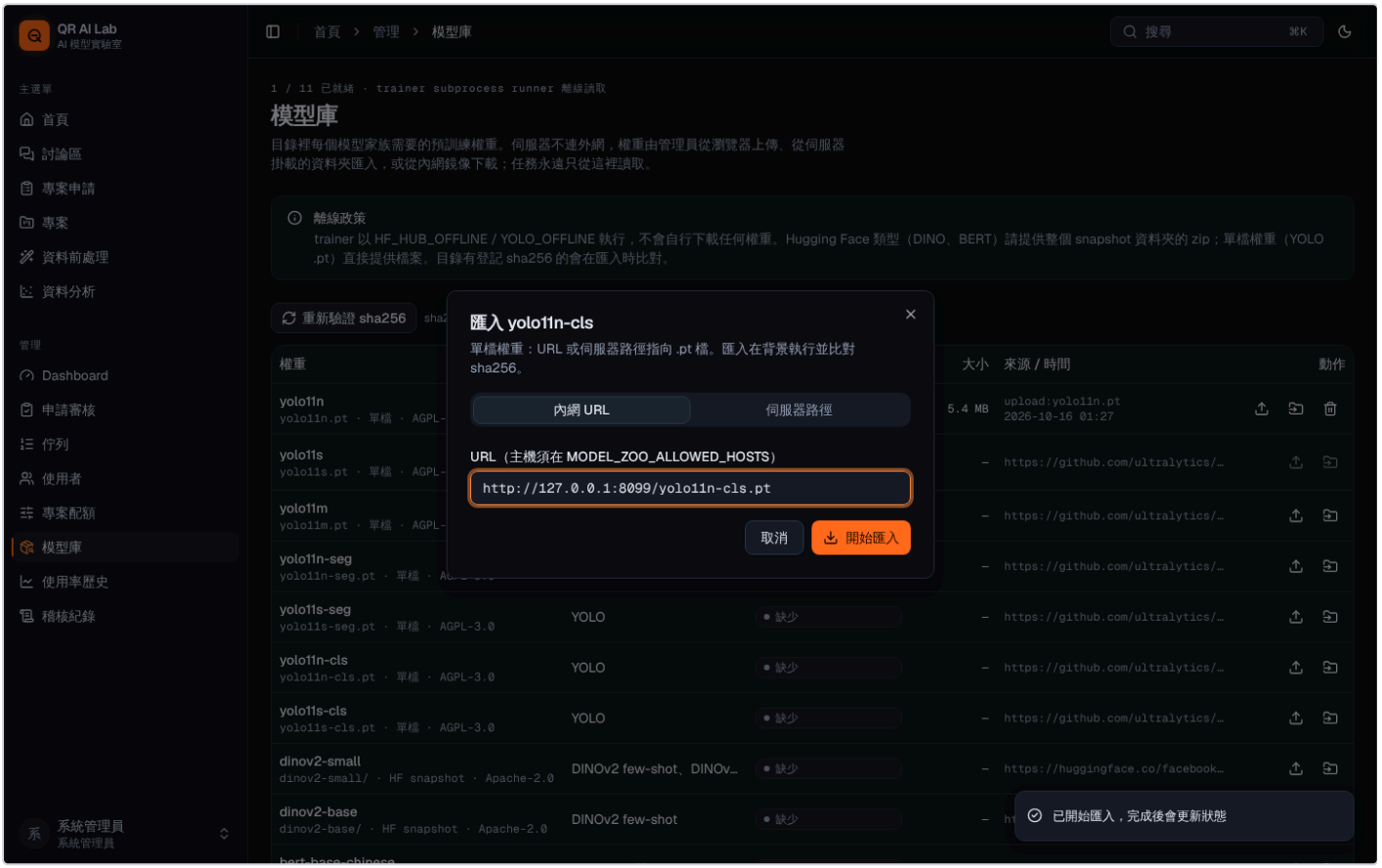
22 從伺服器路徑匯入（NAS / USB 掛載點，限 MODEL_ZOO_IMPORT_ROOT 底下）

dark

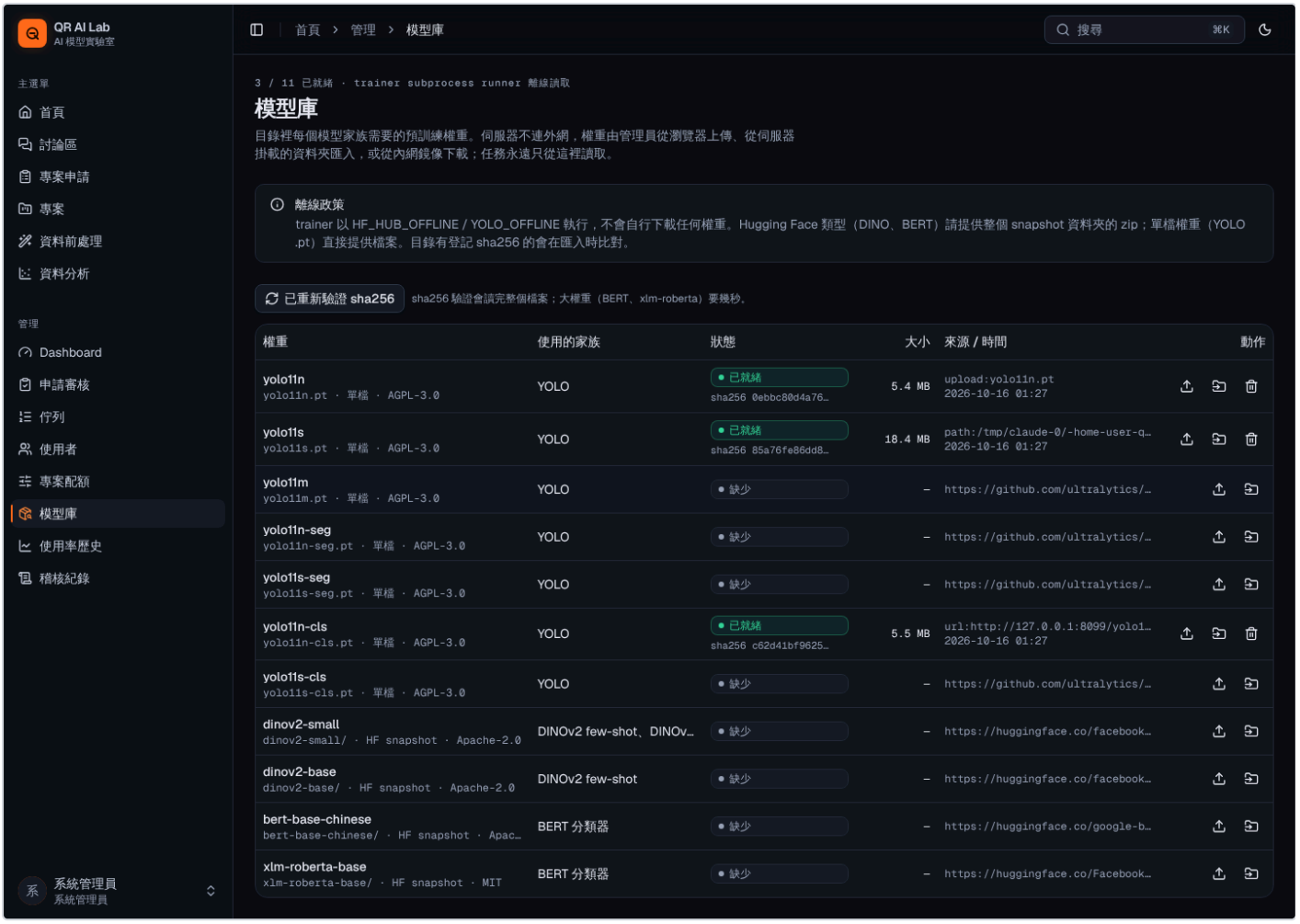


23 從內網 URL 下載（公司鏡像 / 檔案伺服器；主機白名單）

dark

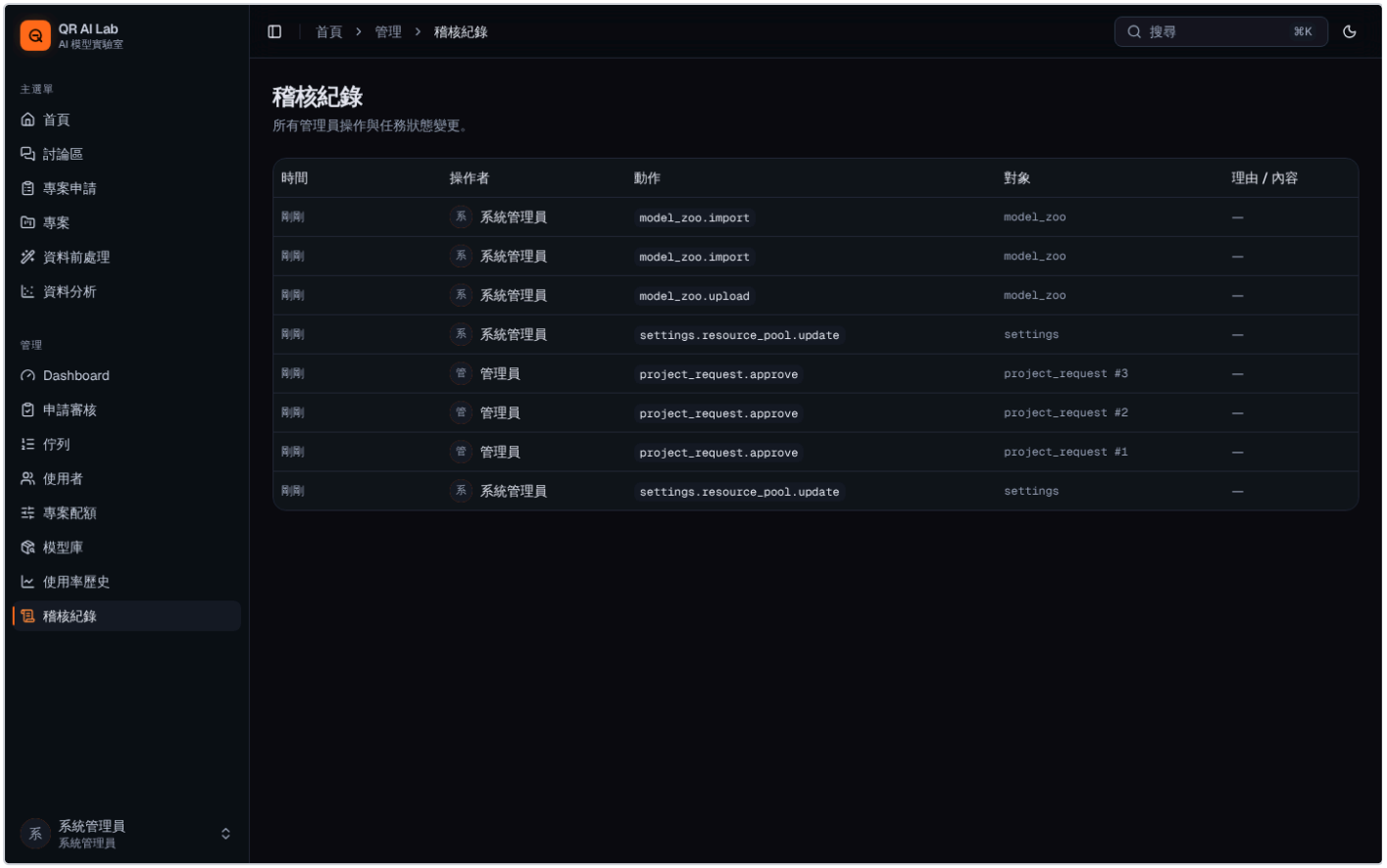


24 三種方式匯入後重新驗證 sha256：三個 YOLO 權重已就緒，來源與時間都有記錄；DINO / BERT 仍缺 dark



25 稽核紀錄：模型庫的上傳 / 匯入都有操作者與來源

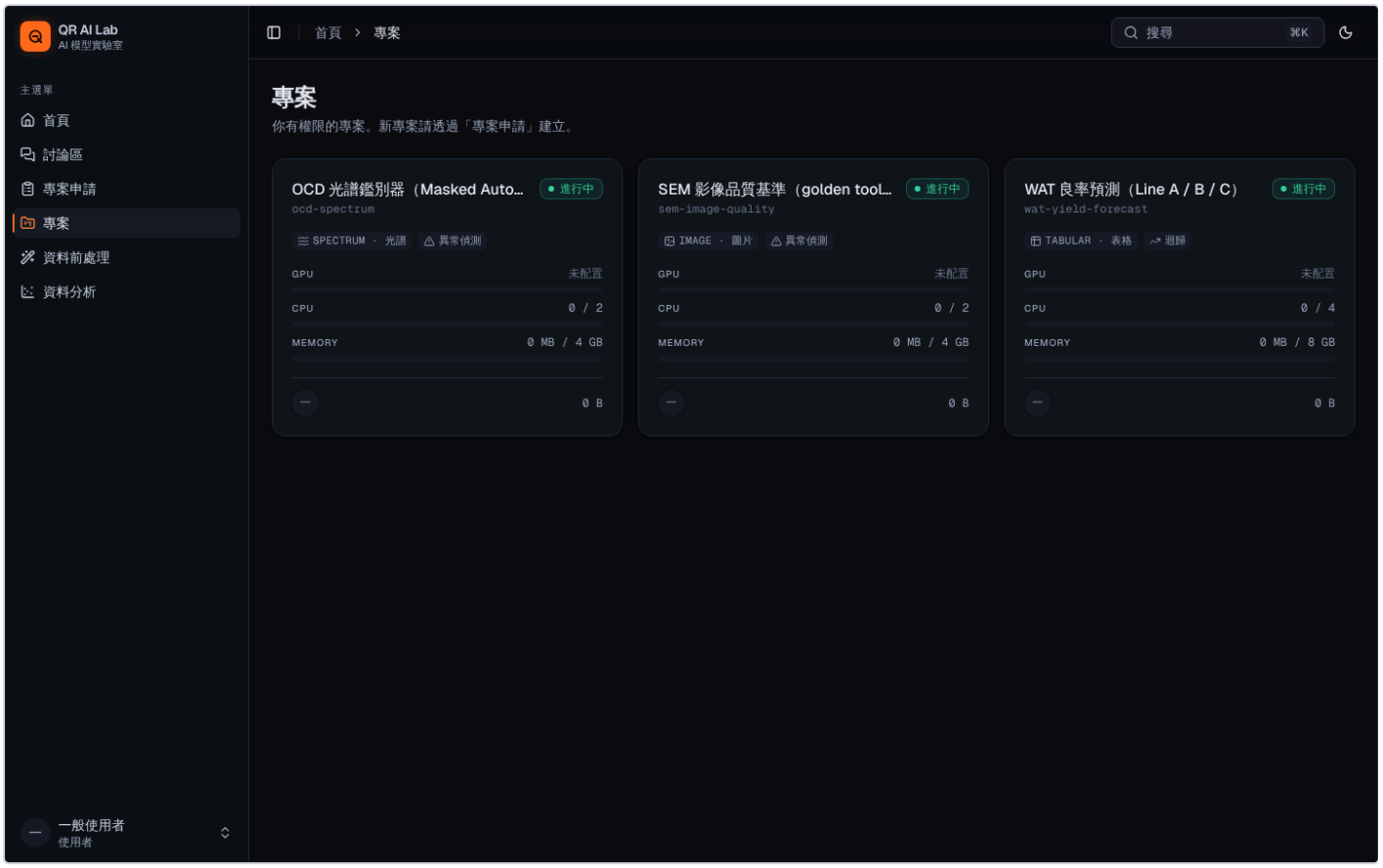
dark



3. 專案：成員、資料來源、設定

專案列表與總覽 → 邀請成員 → 新增 URL 資料來源（憑證加密、連線檢查）→ 專案設定。

26 專案列表：核准後自動建立的三個專案（配額、成員、最近任務） dark



27 專案總覽：配額用量、資料來源、最近任務、SOP 提示

dark



28 邀請成員（帳號或 email + 專案角色）

dark



29 成員列表：擁有者 user、維護者 admin

dark



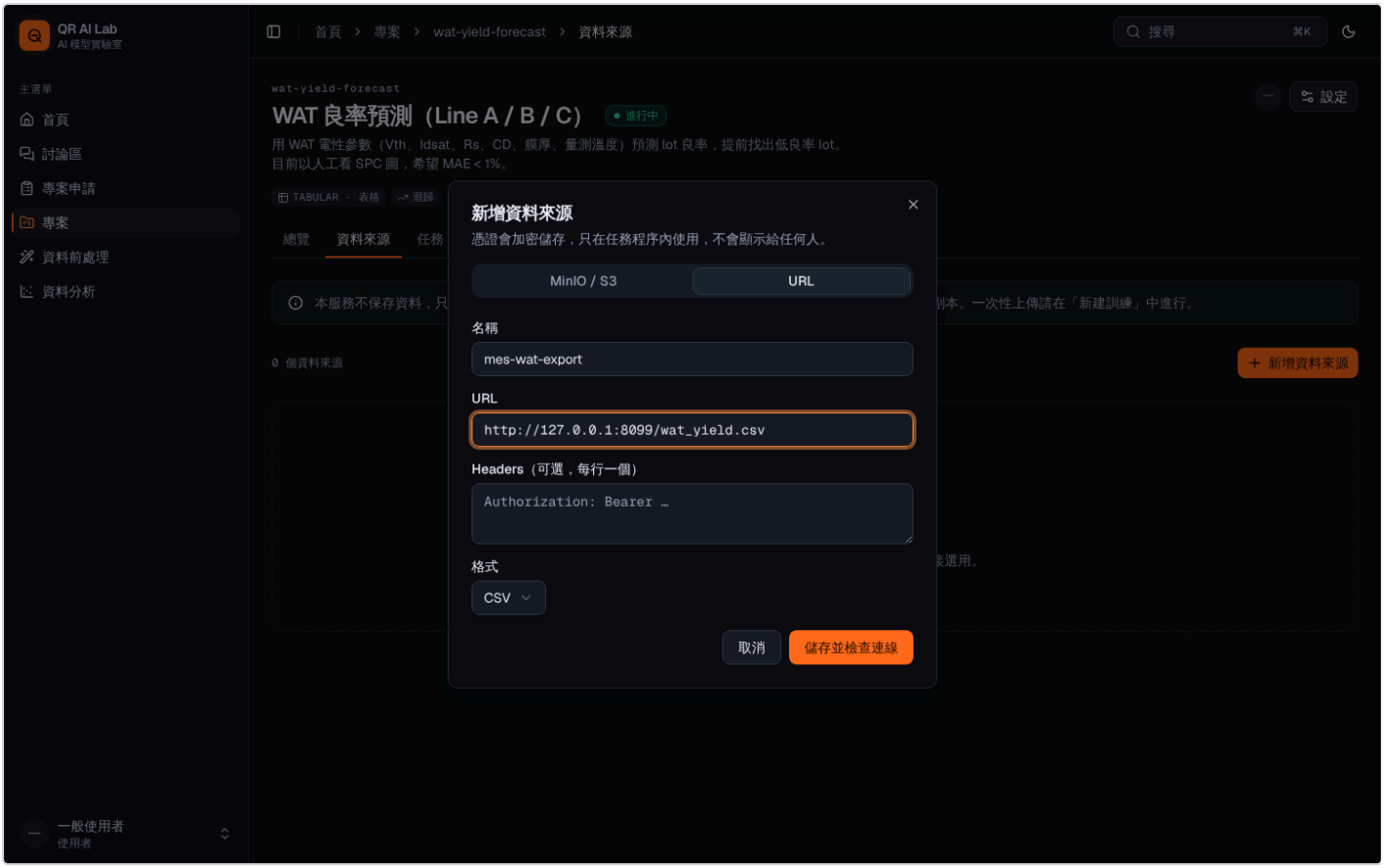
30 資料來源（空）：本服務不保存資料，只記連線參考與加密憑證

dark



31 新增 URL 資料來源（MinIO 分頁可填 endpoint / bucket / access key）

dark



32 資料來源列表：連線檢查結果、格式、最後檢查時間

dark



33 專案設定：名稱、描述、預設逾時；配額由管理員調整 dark



4. 訓練任務：上傳 → 前處理預覽 → AutoML → 即時 log → 產物

一次性上傳 CSV（任務結束即刪）→ 指定欄位、加入前處理步驟並抽樣預覽 → AutoML（XGBoost / 隨機森林 / 線性 / SVM）→ 建立後經排程器以 subprocess runner 執行 → SSE 即時進度與 log → leaderboard、產物、預設模型。

34 任務列表（空）

dark



35 建立訓練任務 ① 資料：一次性上傳 / 資料來源；上傳檔只存在於任務執行期間

dark

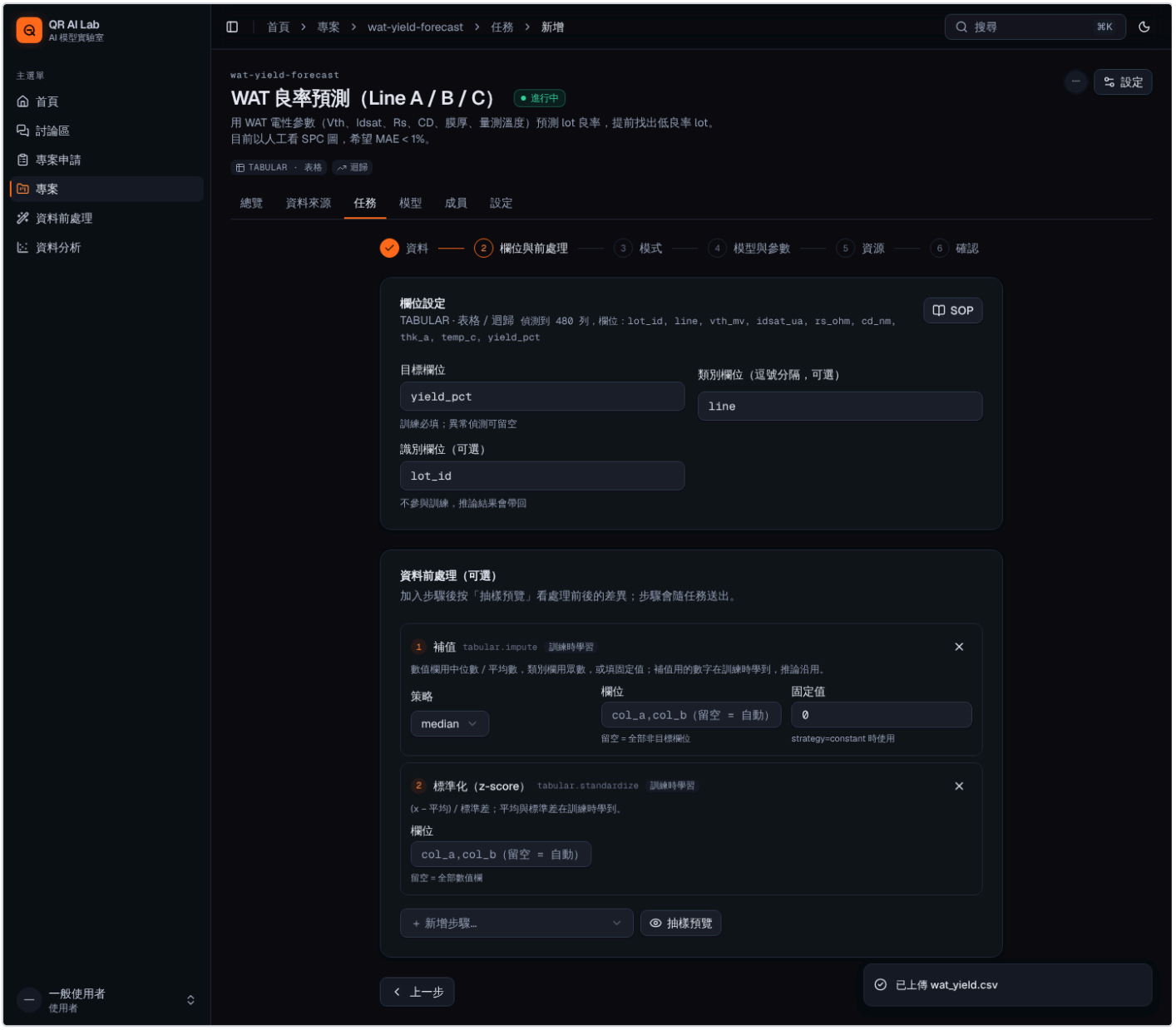


36 CSV 上傳完成：backend 偵測到 480 列與欄位

dark

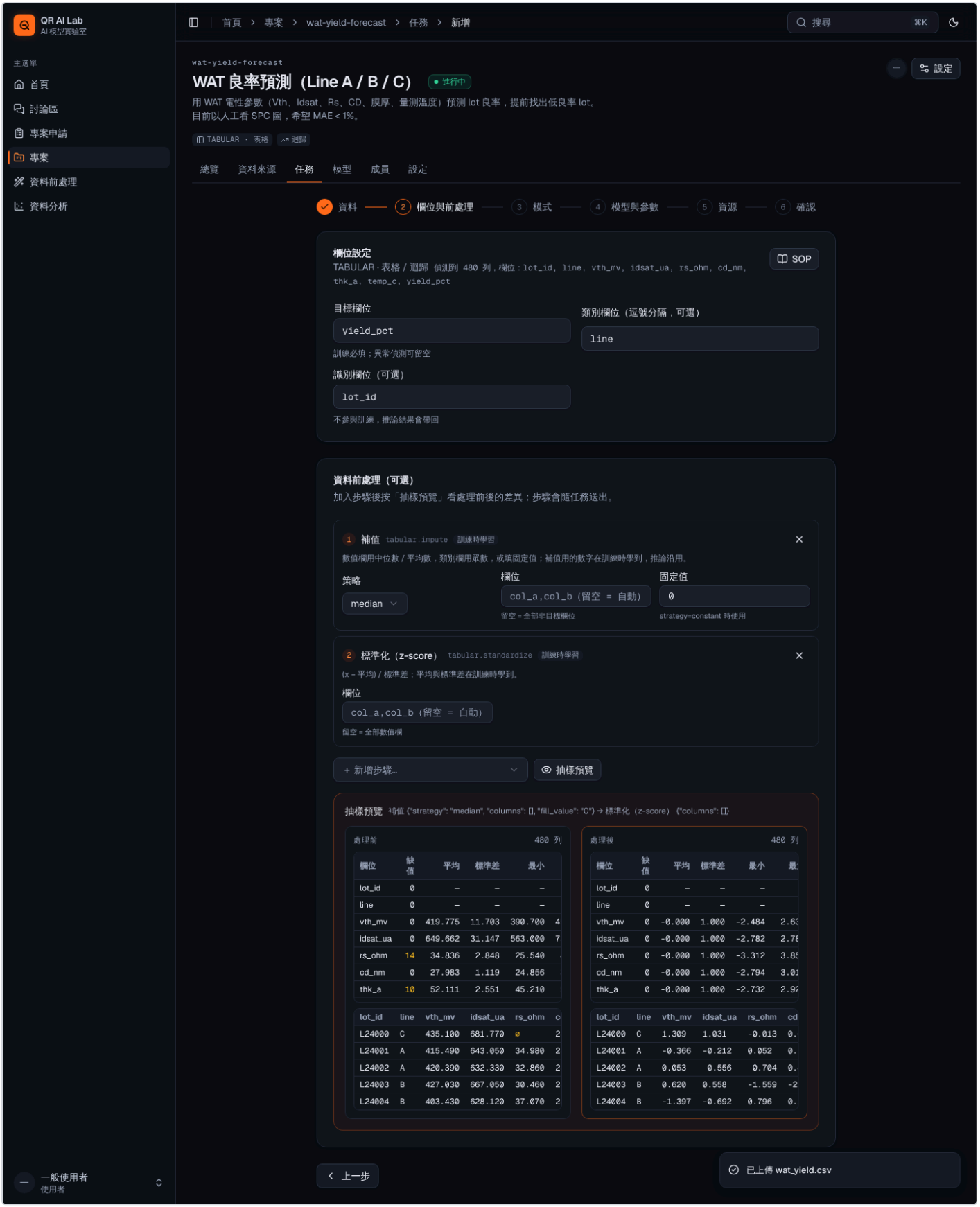


37 ② 欄位與前處理：目標欄位 yield_pct、類別欄位 line、識別欄位 lot_id；加入「補值」與「標準化」兩個步驟 dark



38 抽樣預覽：從上傳檔取樣，顯示前處理「套用前 / 套用後」的統計與資料列

dark



39 ③ 模式：自動（AutoML）或手動

dark



40 ④ AutoML 候選模型與預算：XGBoost / 隨機森林 / 線性 / SVM，最多 6 個 trial

dark



41 ⑤ 資源：GPU 0、CPU 2、2 GB、逾時 30 分鐘（不得超過專案配額） dark



42 ⑥ 確認：送出後 API 回 202 建立 jobs row，由排程器依佇列執行 dark



43 任務建立：狀態 queued（佇列位置、申請的資源）

dark



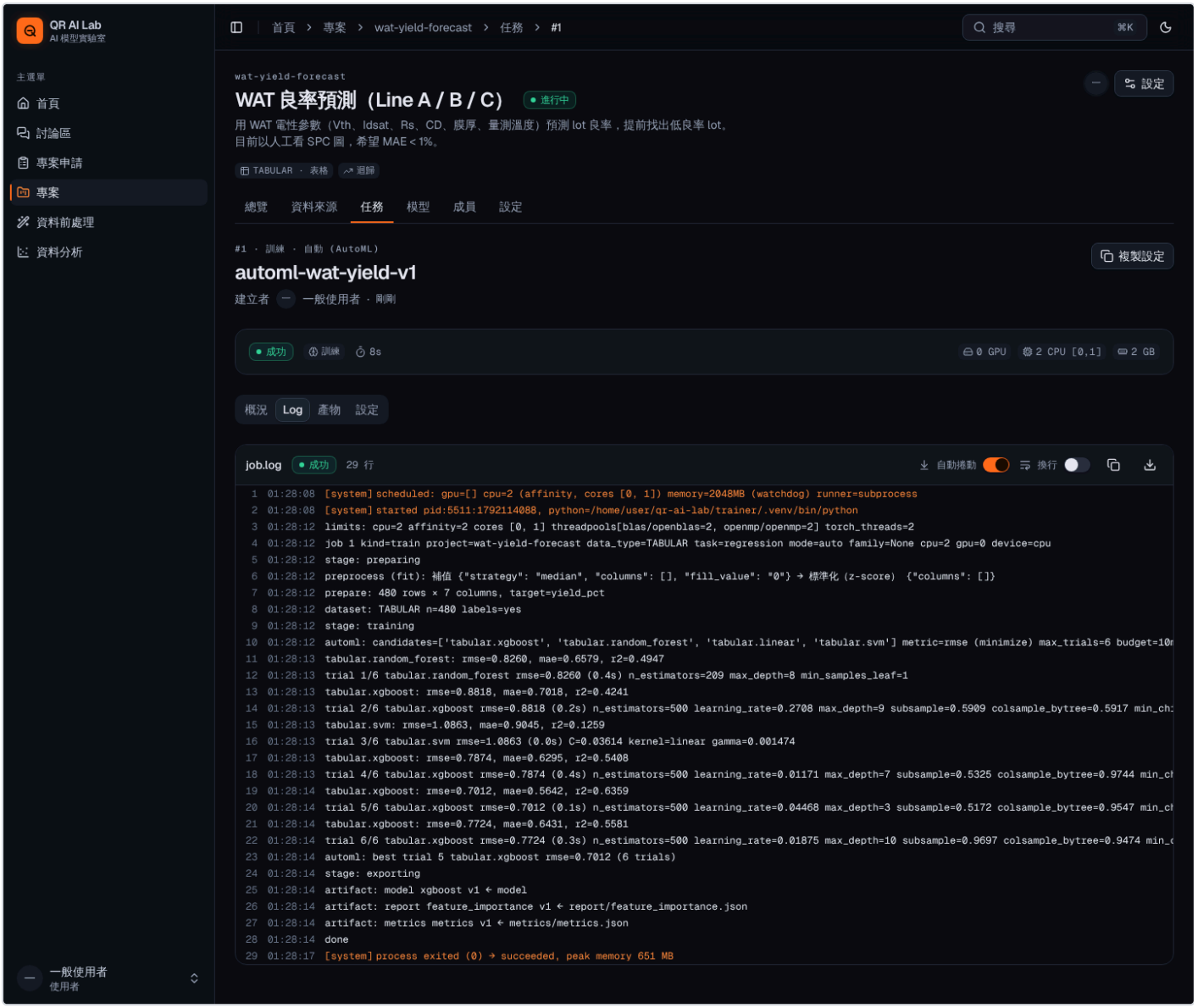
44 排程器以主機子程序啟動 trainer（trainer venv，CPU 核心 affinity、記憶體 watchdog）：running，進度卡由 SSE 即時更新

dark



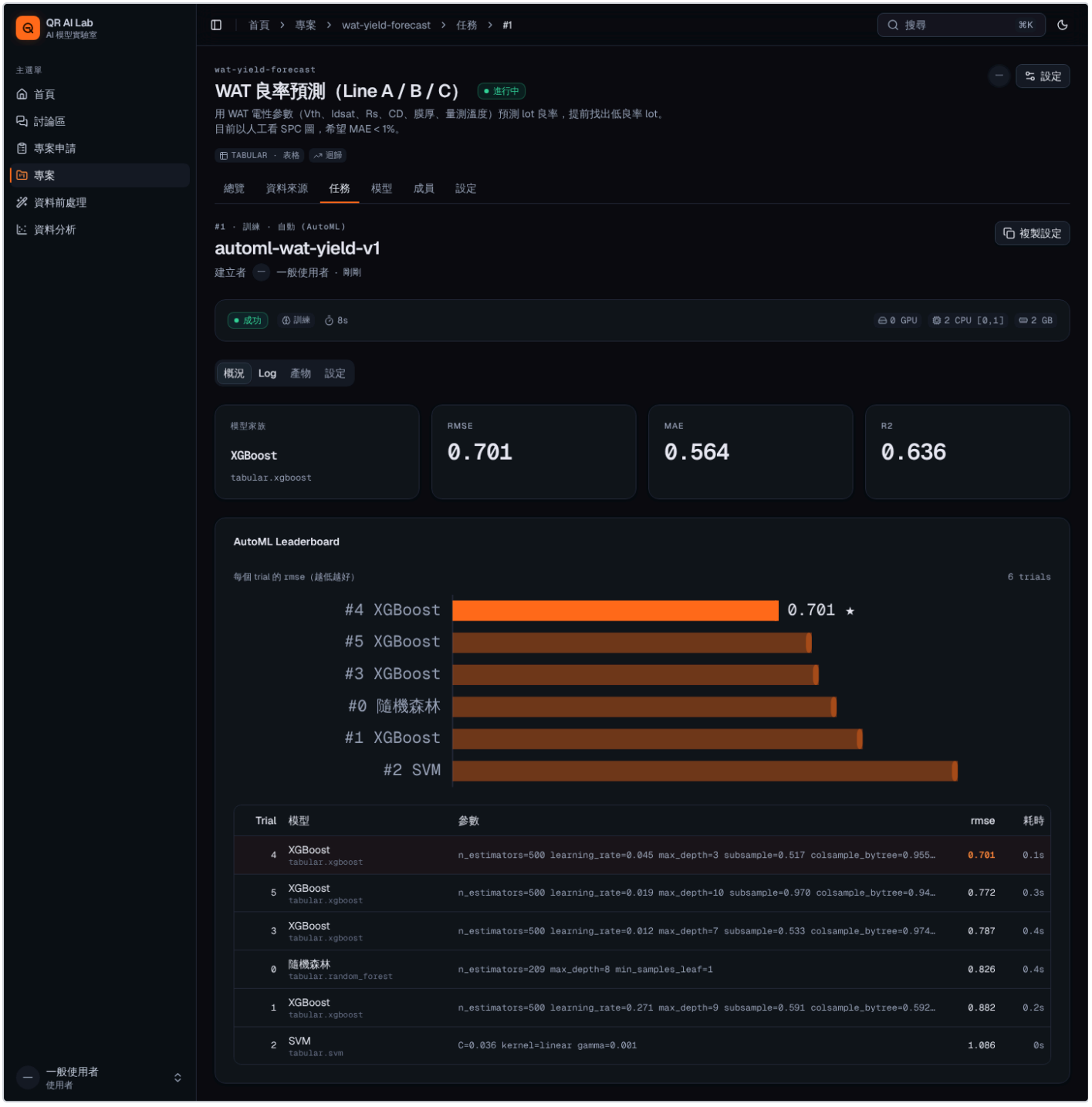
45 Log 分頁：trainer 直接寫 log 檔，排程器 tail 後經 SSE 即時串流（[system] 行寫明 runner 與限制方式）

dark



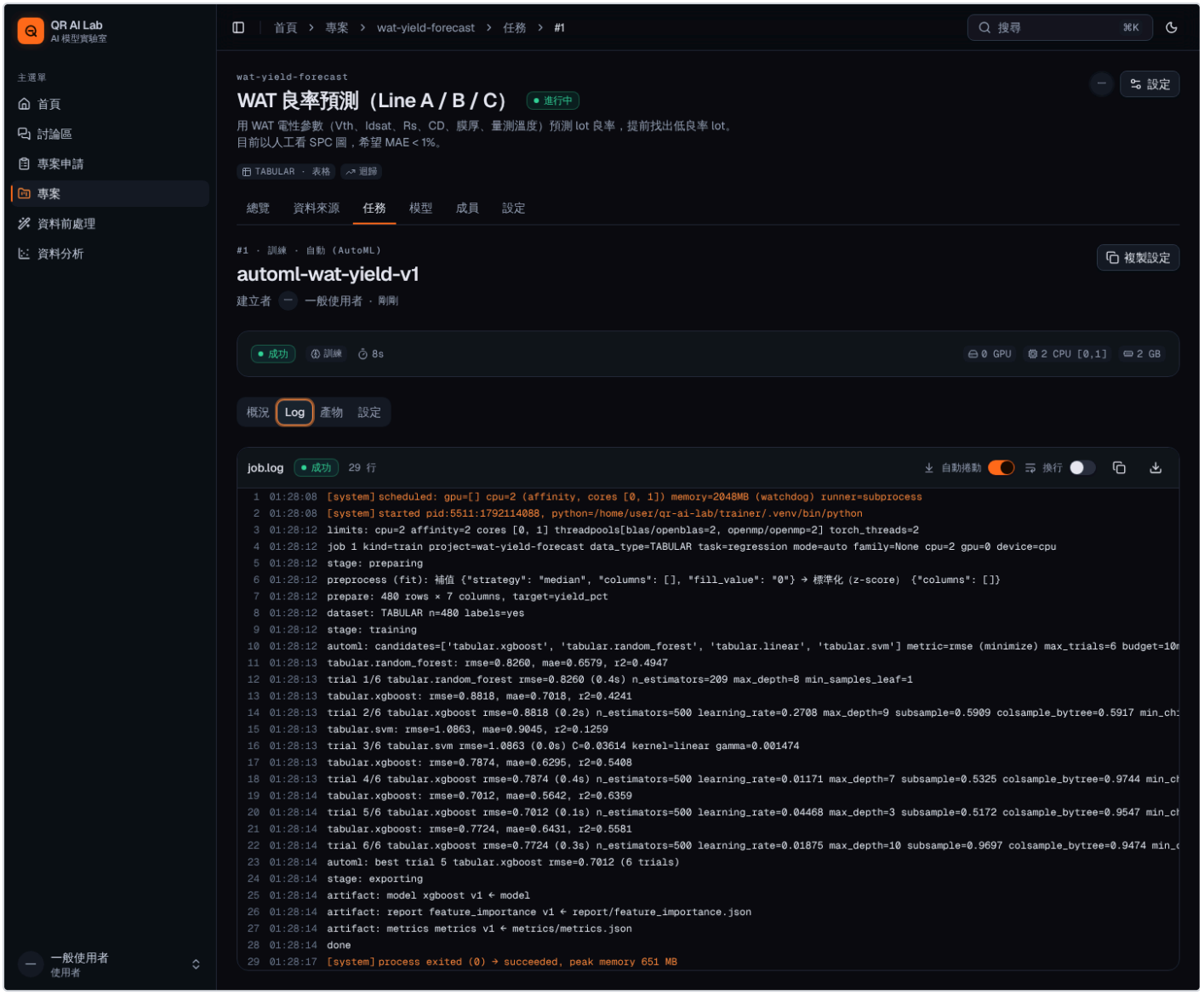
46 訓練完成：最佳家族回填、指標卡、AutoML leaderboard（圖 + 表）

dark



47 完整 log（可複製 / 下載）

dark

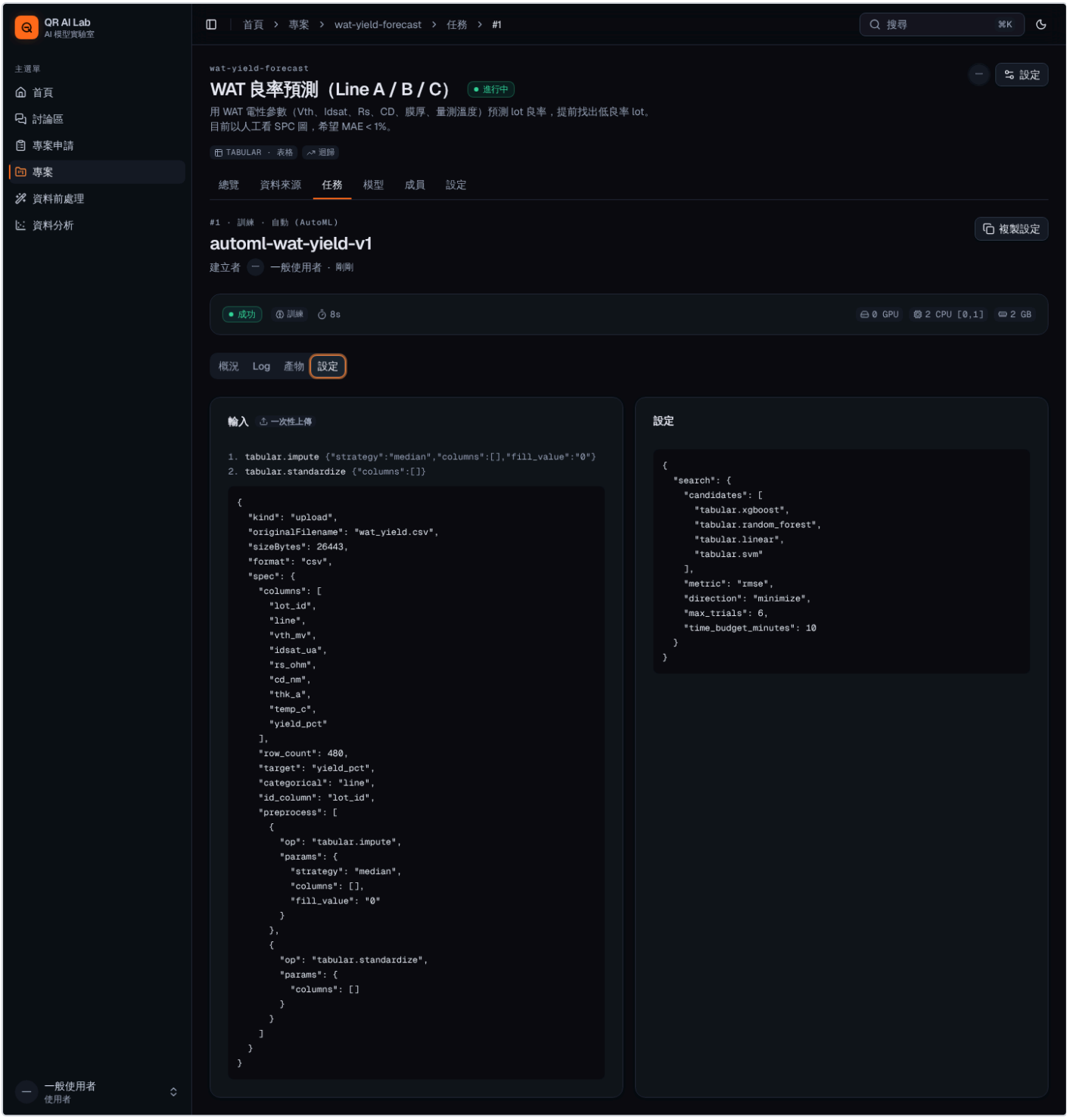


48 產物：模型、前處理狀態、指標；可下載或設為專案預設 dark



49 設定分頁：輸入規格（含前處理步驟）與搜尋設定

dark



50 模型頁：AutoML 最佳模型已登記為預設

dark



5. 推論任務：直接輸入 → 結果預覽與下載

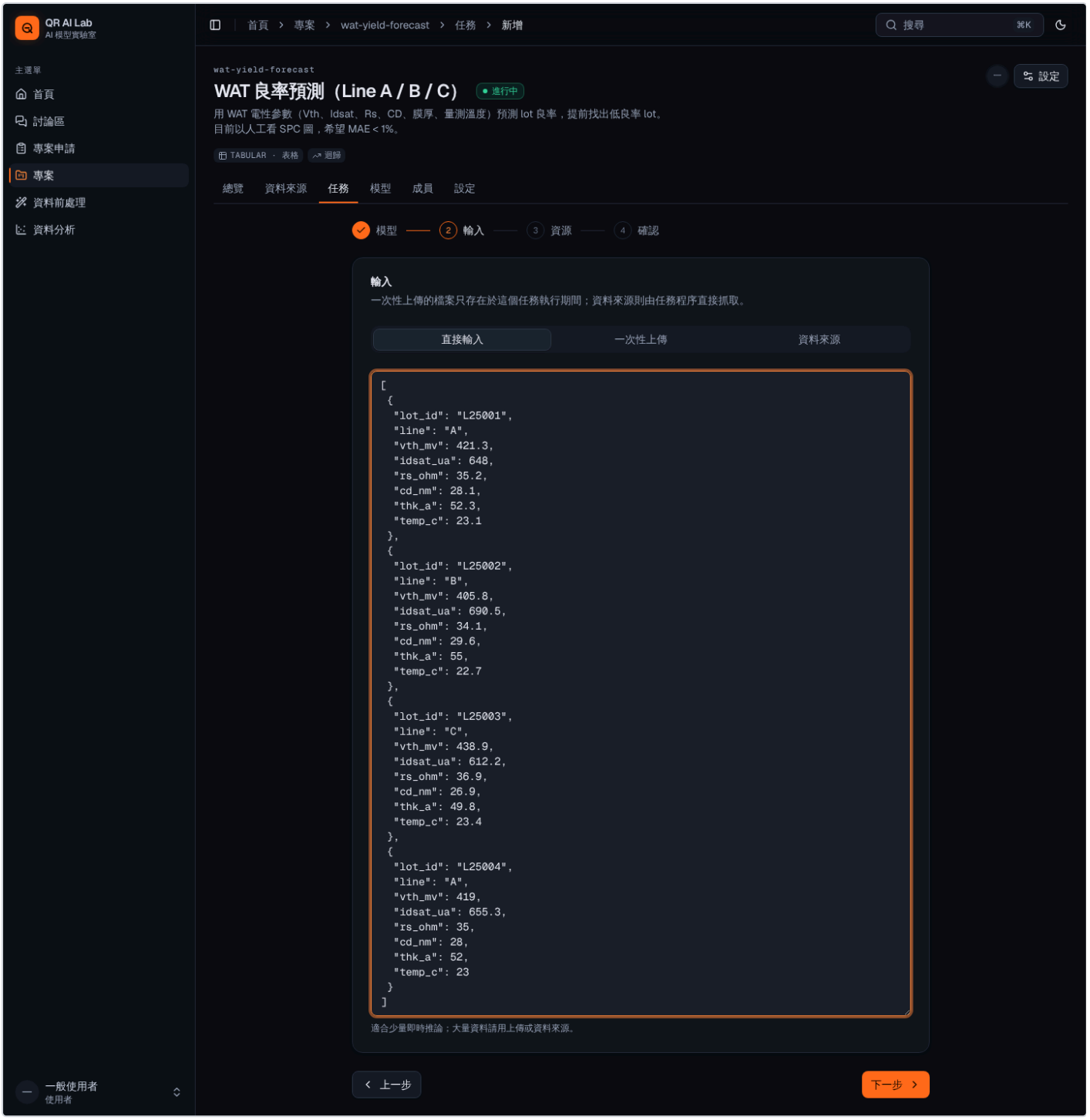
用預設模型對新 lot 做 predict → 結果摘要、前 20 筆預覽（帶回 lot_id）、CSV 下載。

51 建立推論任務 ① 模型：預設選專案的預設模型，列出支援的推論模式 dark

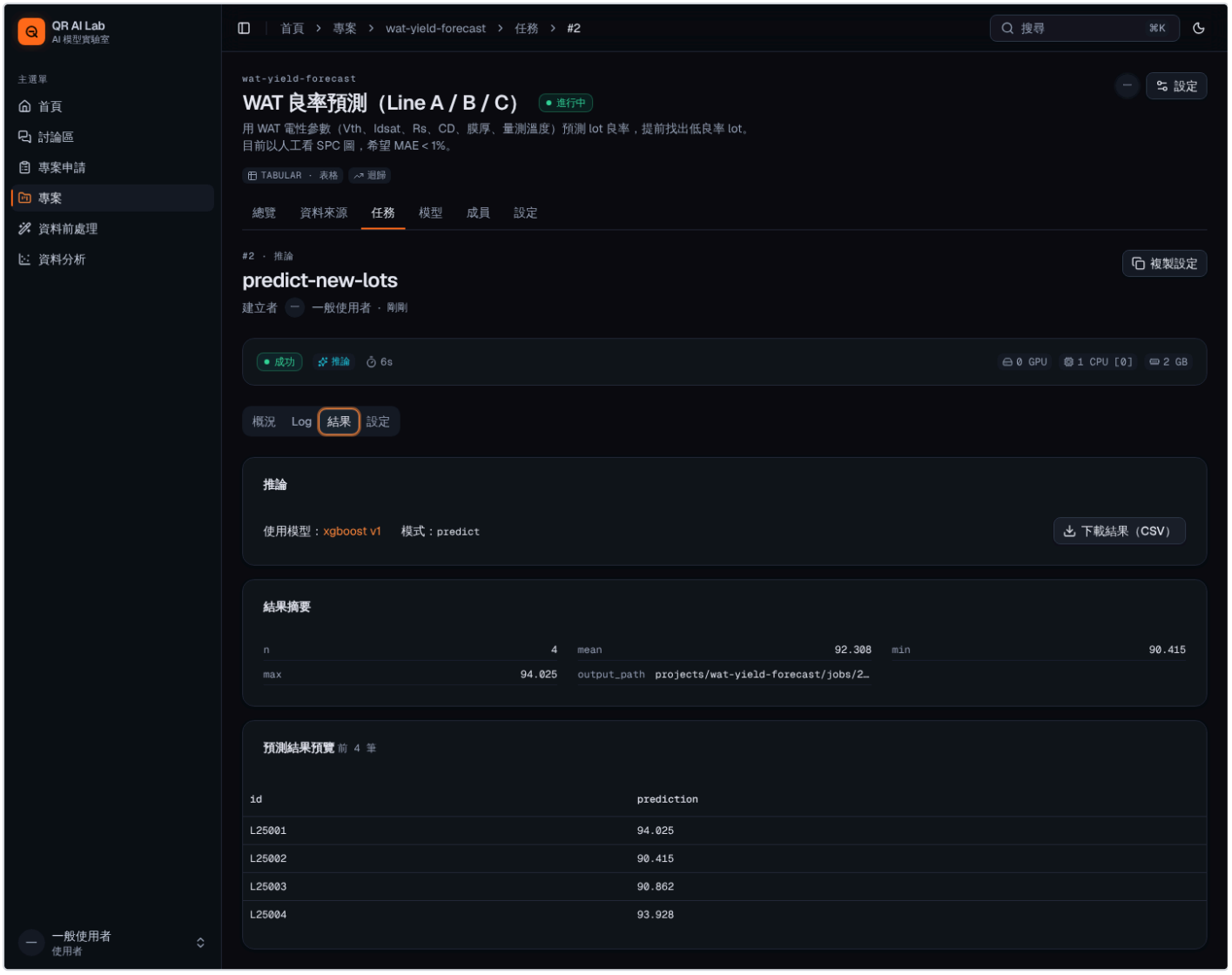


52 ② 輸入：直接貼 JSON 列（也可上傳檔案或用資料來源），推論模式 predict

dark



53 推論結果：摘要、前 20 筆預覽（帶回 lot_id）、下載 CSV dark



54 任務列表：訓練與推論都在 jobs 表，狀態、資源、耗時

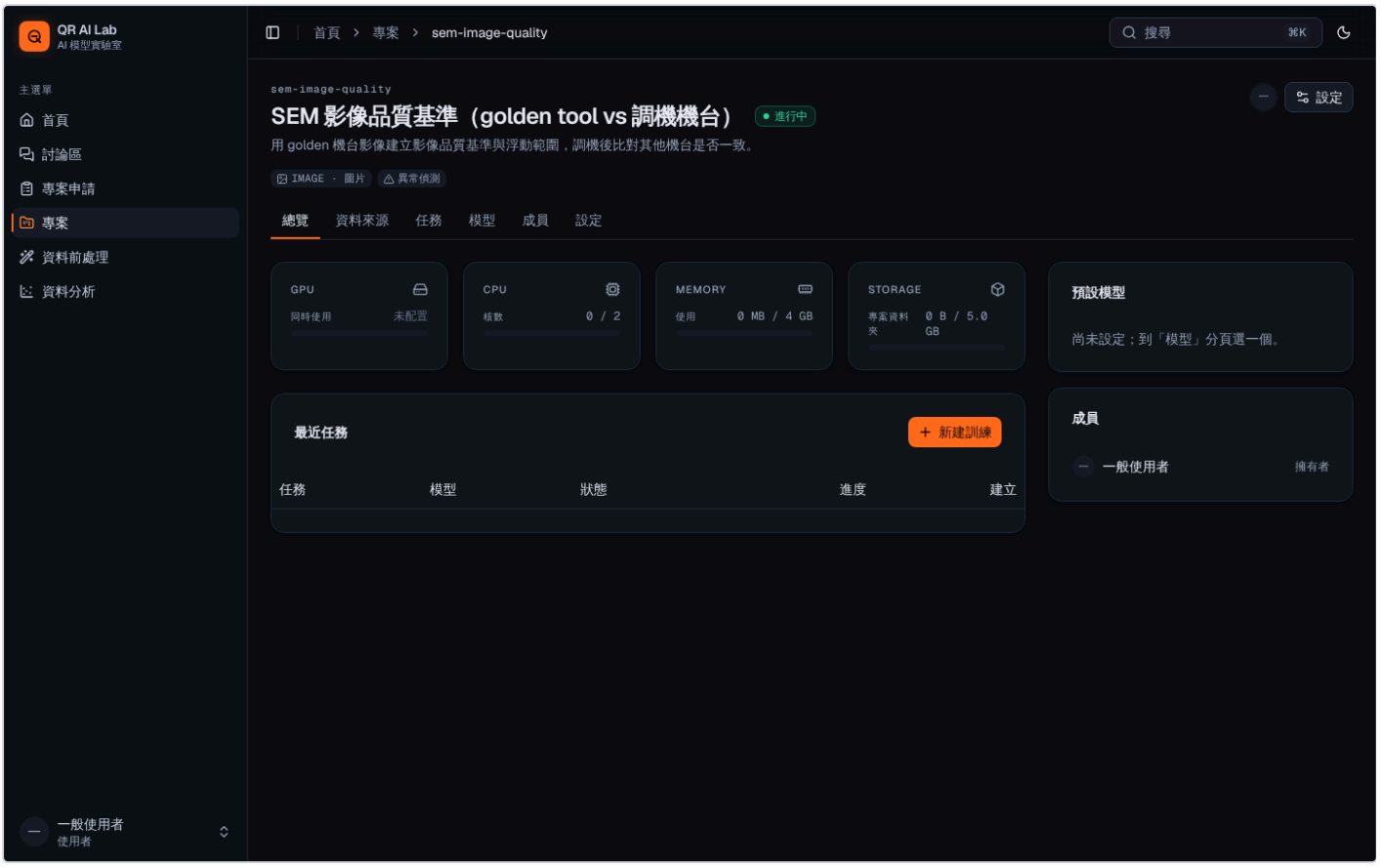
dark



6. 影像品質基準（golden tool vs 調機機台）

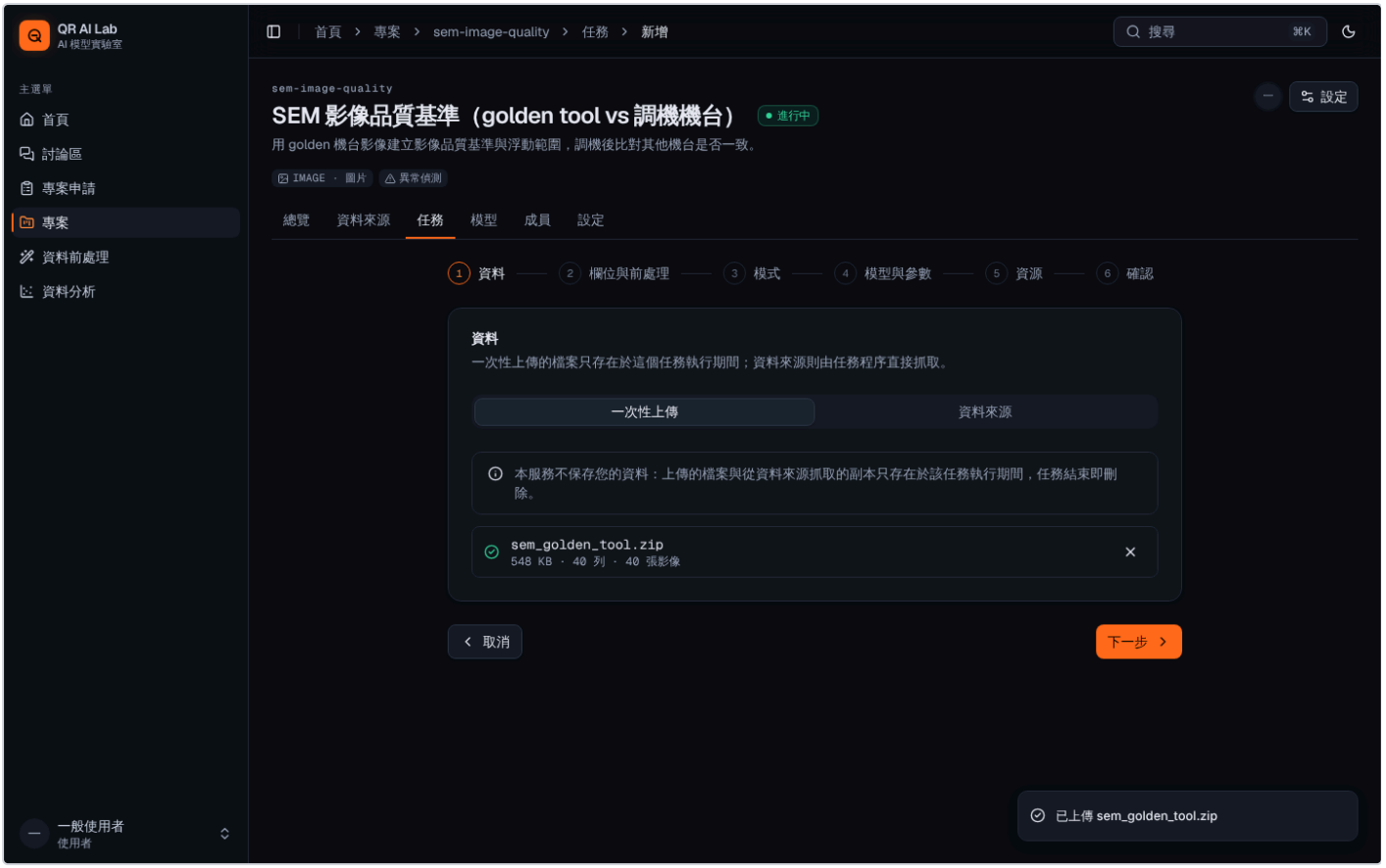
上傳 golden 機台 40 張影像建立基準（17 個無參考指標的中位數 / 浮動範圍 / Mahalanobis 門檻）→ 上傳調機機台影像以 compare 模式產生機台比對報告。

55 影像品質專案（IMAGE × 異常偵測） dark



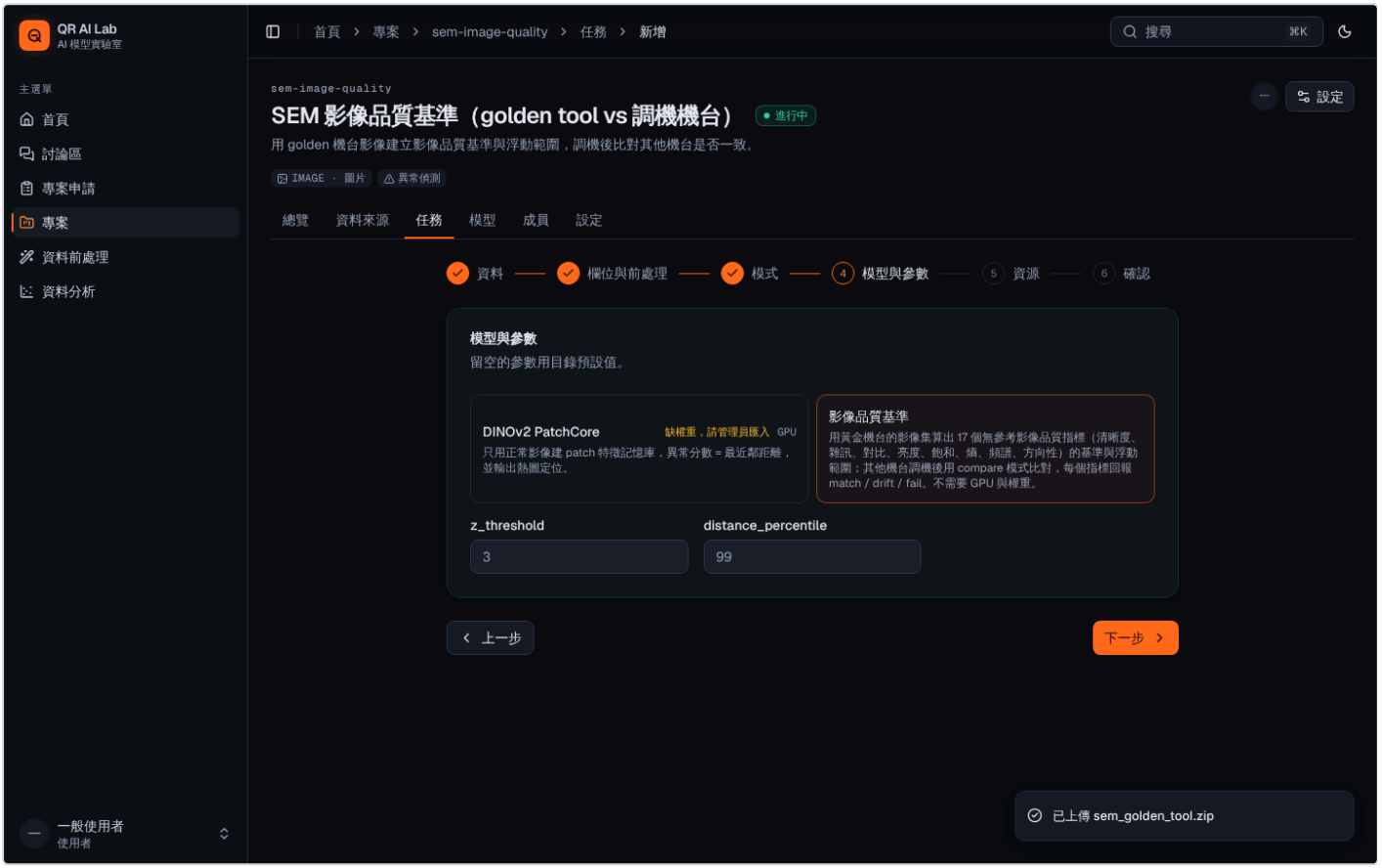
56 上傳 golden 機台 40 張 SEM 影像（zip）

dark



57 手動選「影像品質基準」：17 個無參考 IQA 指標 → 中位數 / 浮動範圍 / Mahalanobis 門檻

dark



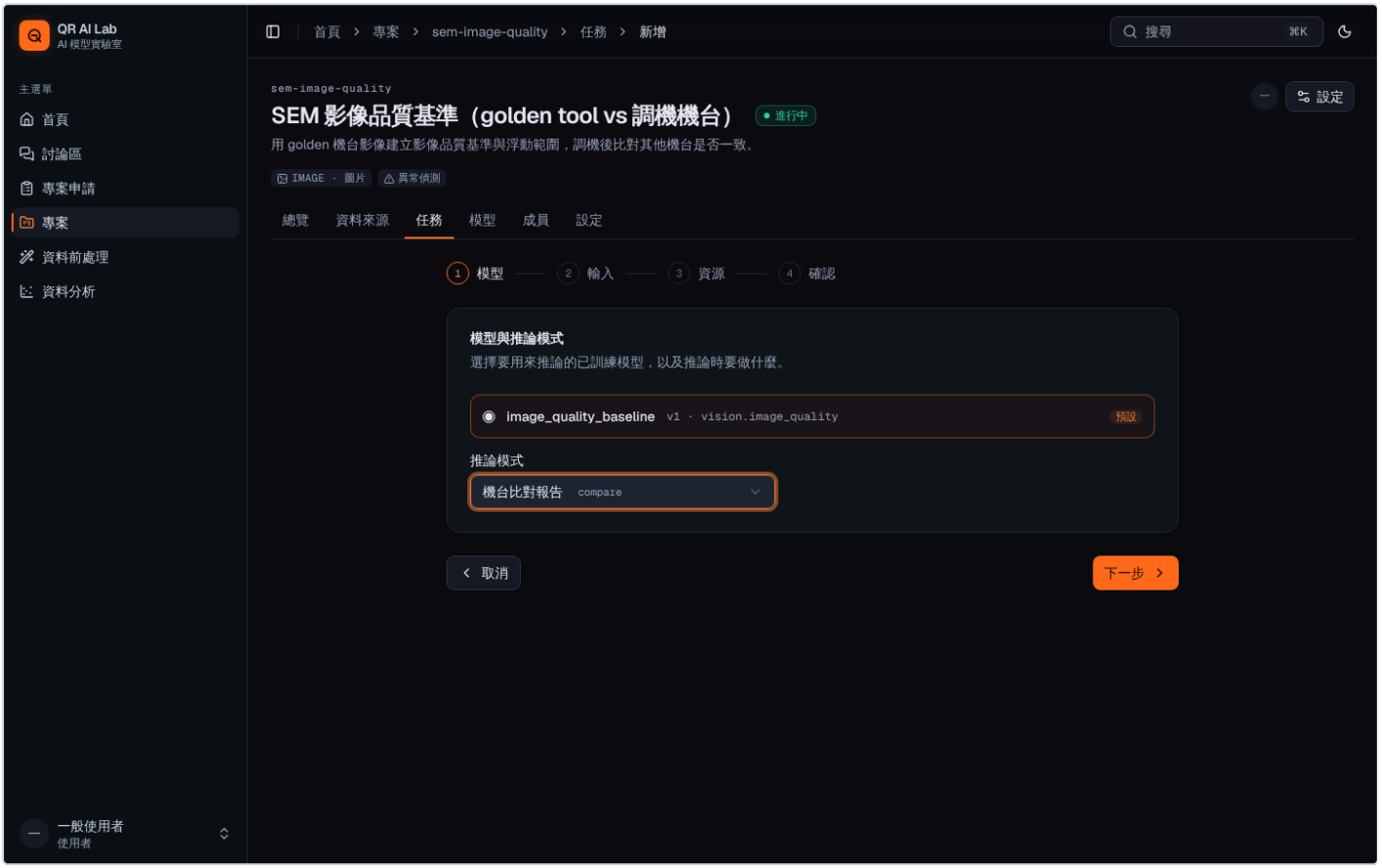
58 基準建立完成：golden 影像數、驗證指標、產物

dark



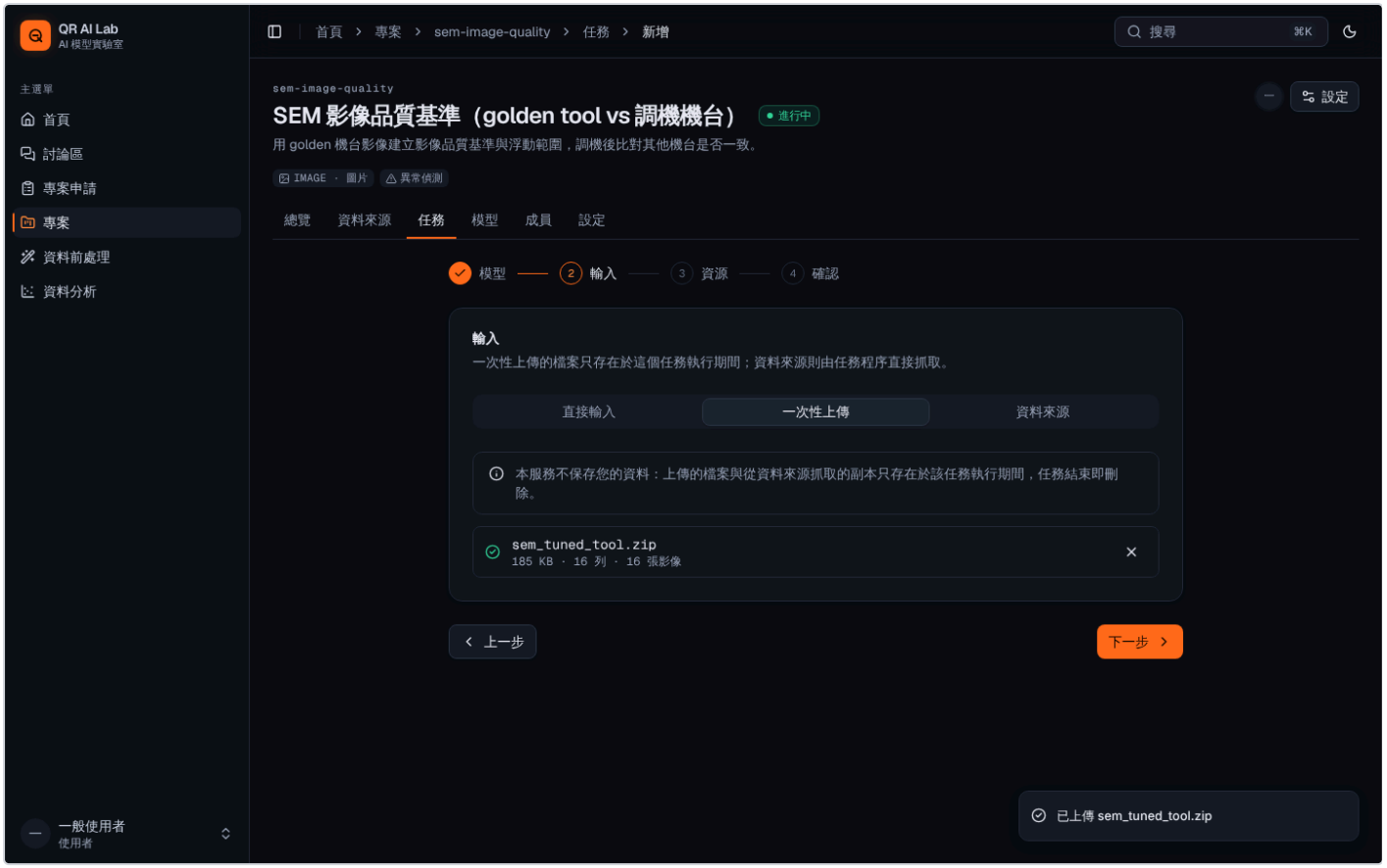
59 推論 ① 模型：影像品質基準模型支援 compare / score / embed，選「機台比對報告」

dark



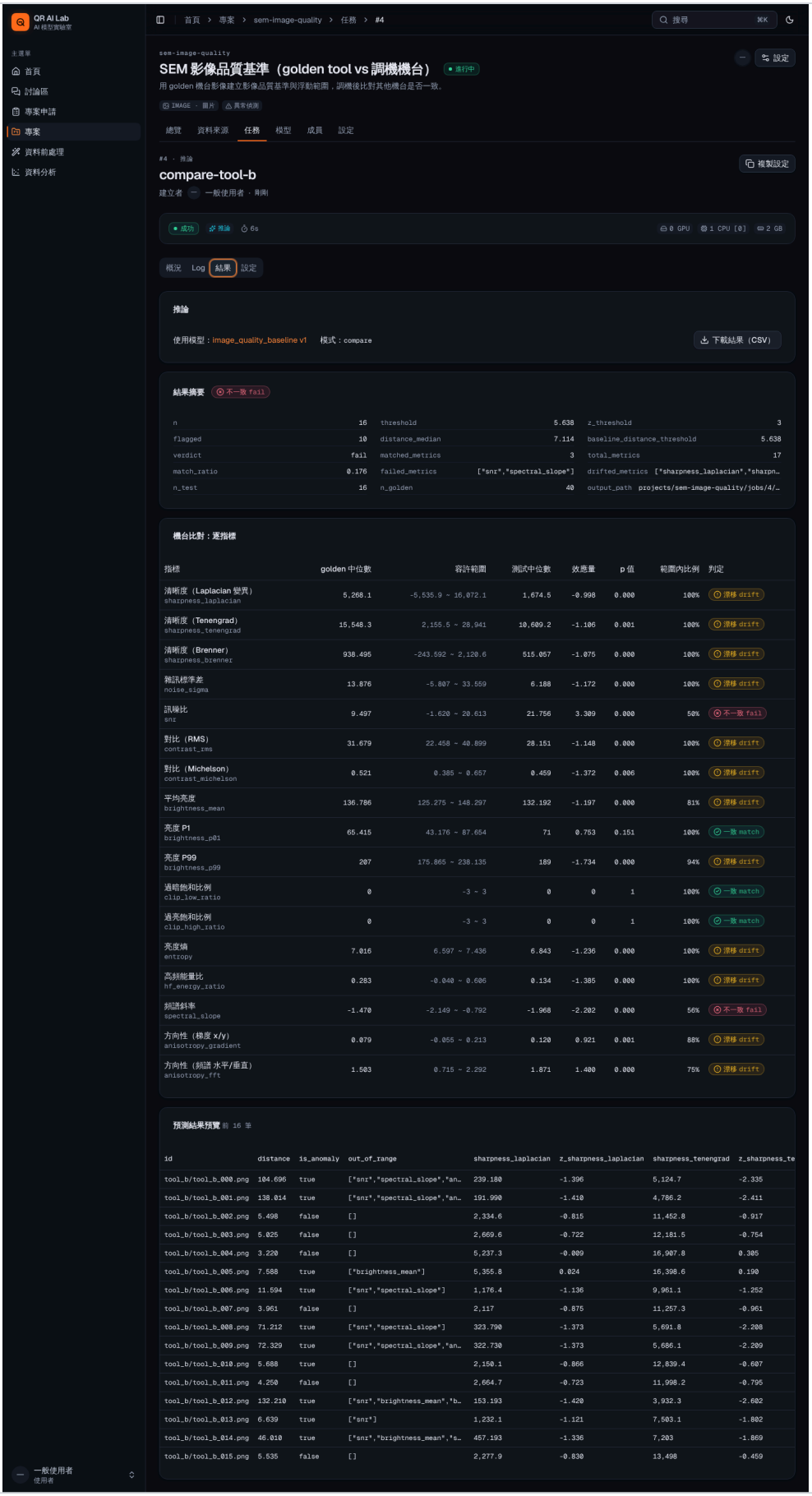
60 推論 ② 輸入：上傳調機機台 16 張影像（zip）

dark



61 機台比對報告：整體判定 + 逐指標（golden 中位數 / 容許範圍 / 測試中位數 / 效應量 / KS p 值 / 判定）

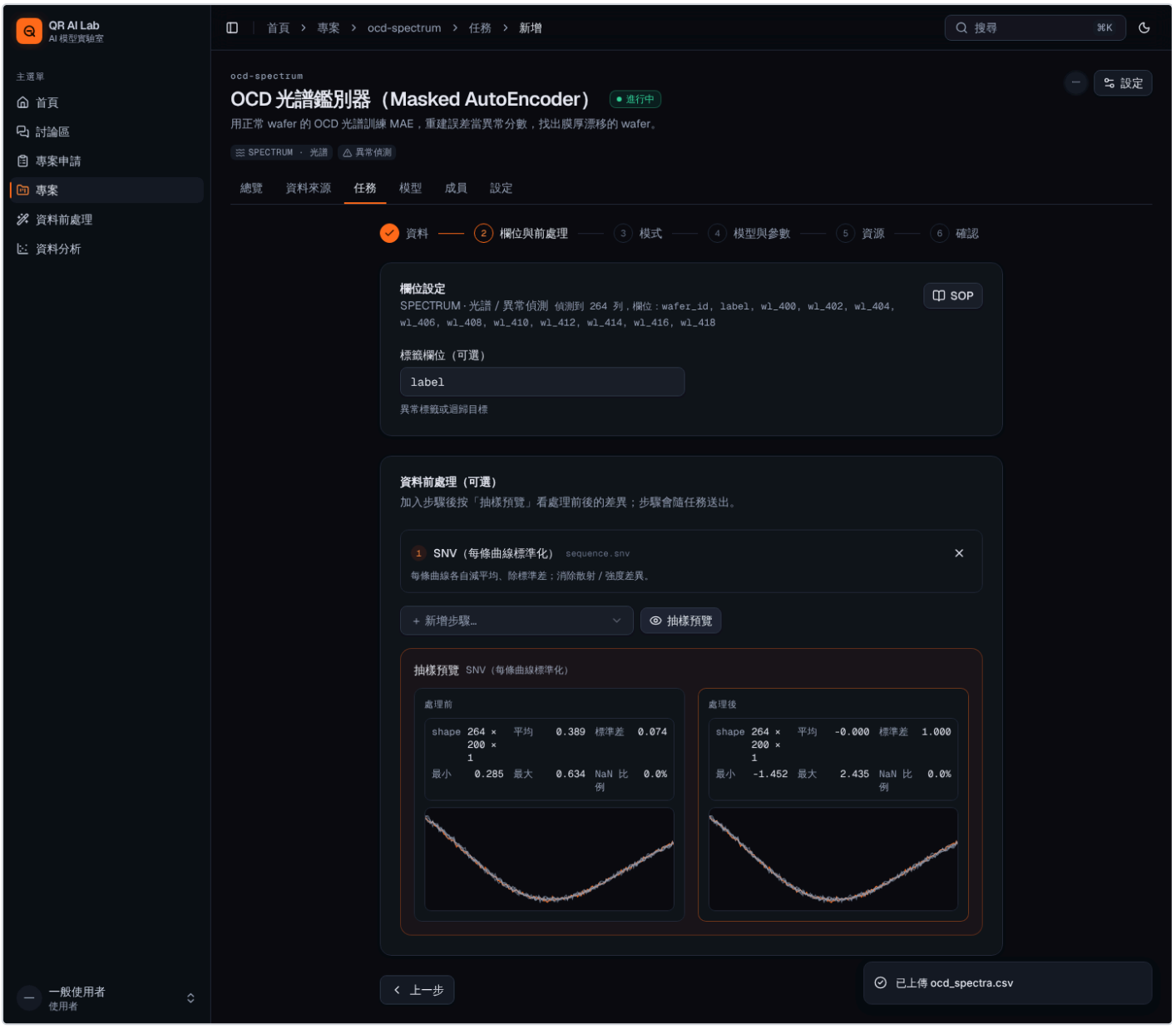
dark



7. 光譜鑑別器（Masked AutoEncoder）

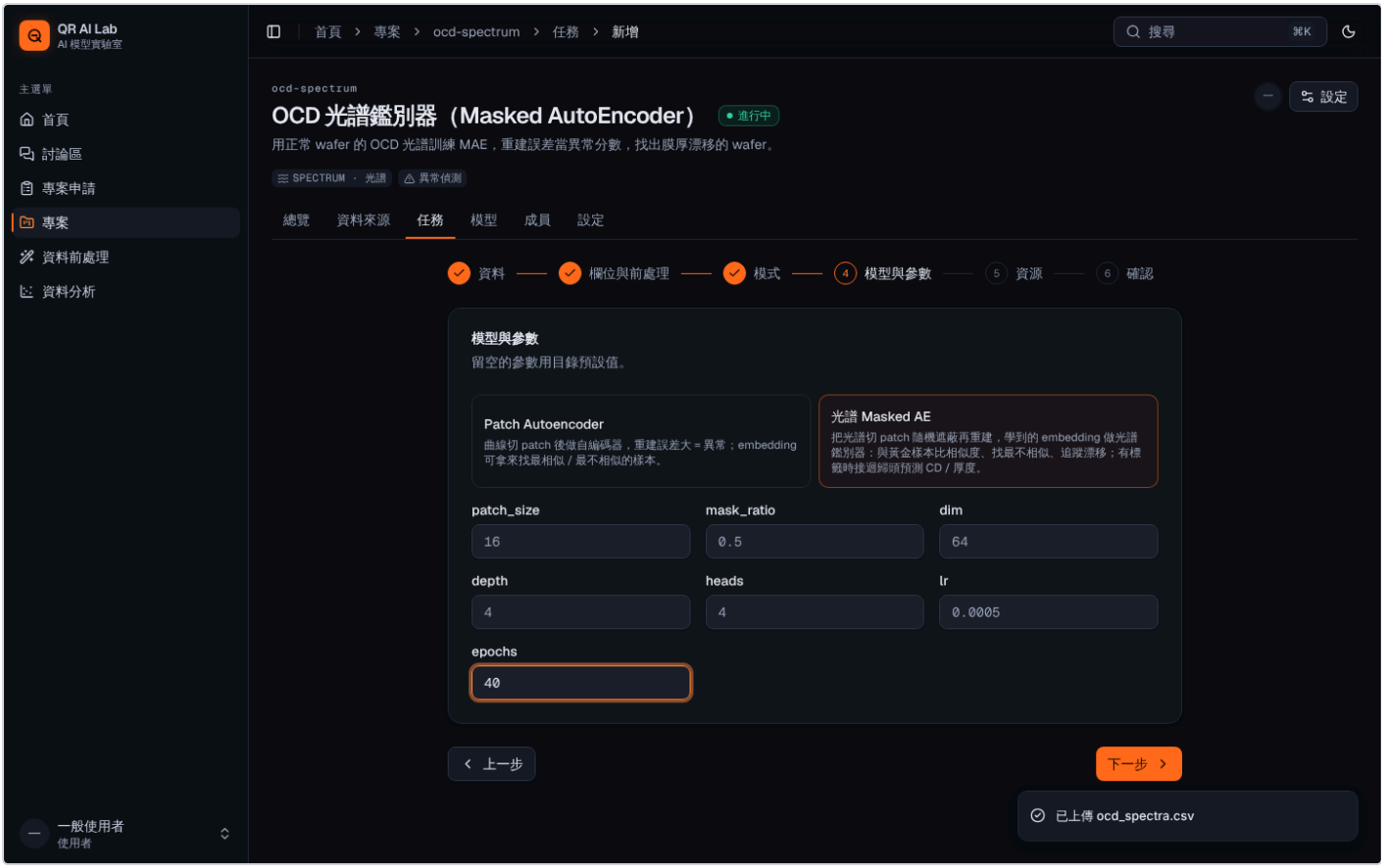
寬表 CSV（每列一片 wafer、200 個波長）→ SNV 前處理預覽 → 手動選 MAE 訓練（CPU）→ 即時 epoch 進度與指標。

62 光譜前處理預覽：SNV 標準化前後的曲線 dark



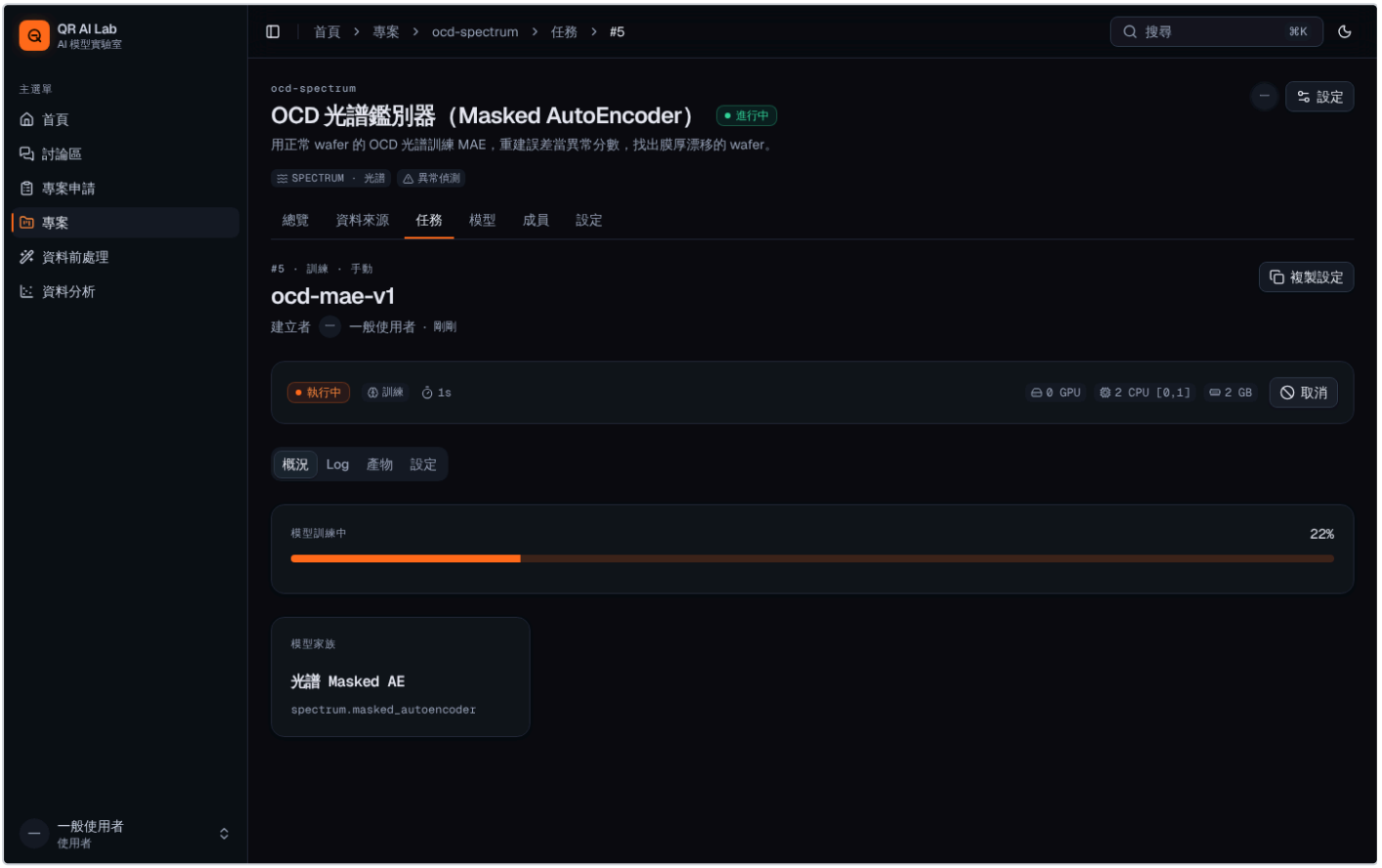
63 手動選「光譜 Masked AE」，覆寫 epochs=40（CPU 上約 30 秒）

dark

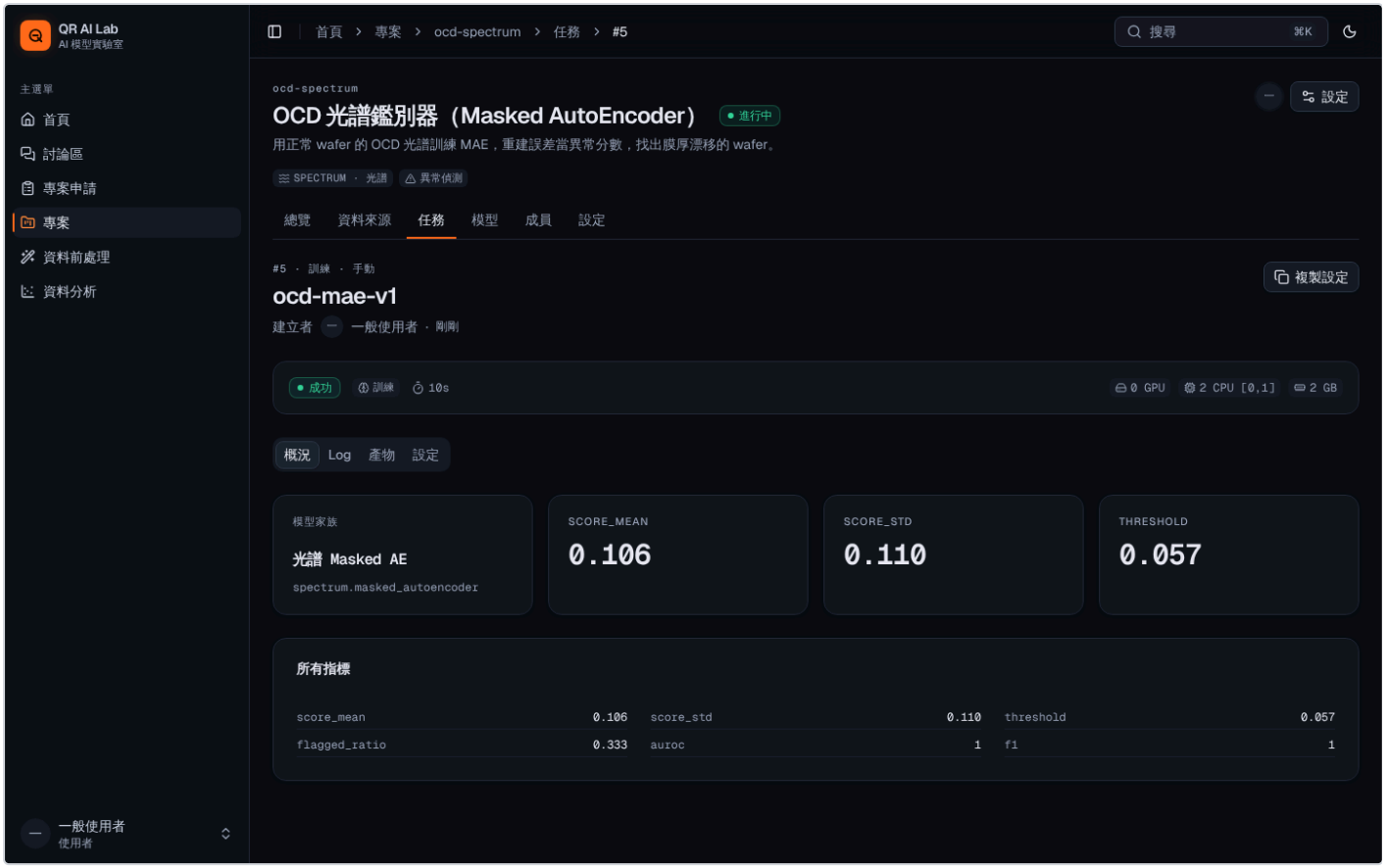


64 MAE 訓練中：epoch 進度與即時指標

dark



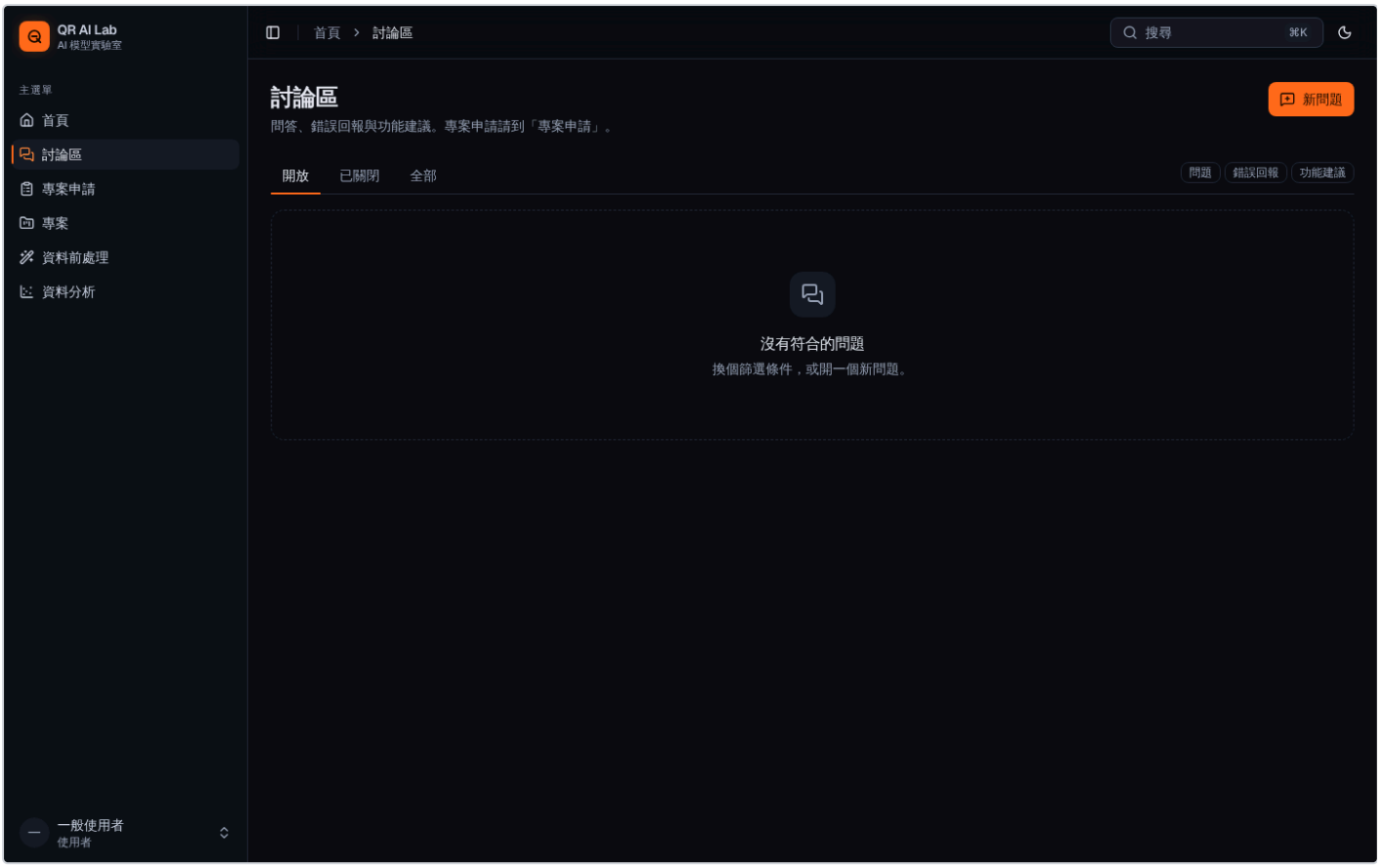
65 光譜鑑別器訓練完成：AUROC 等指標、重建誤差門檻 dark



8. 討論區（問題單）

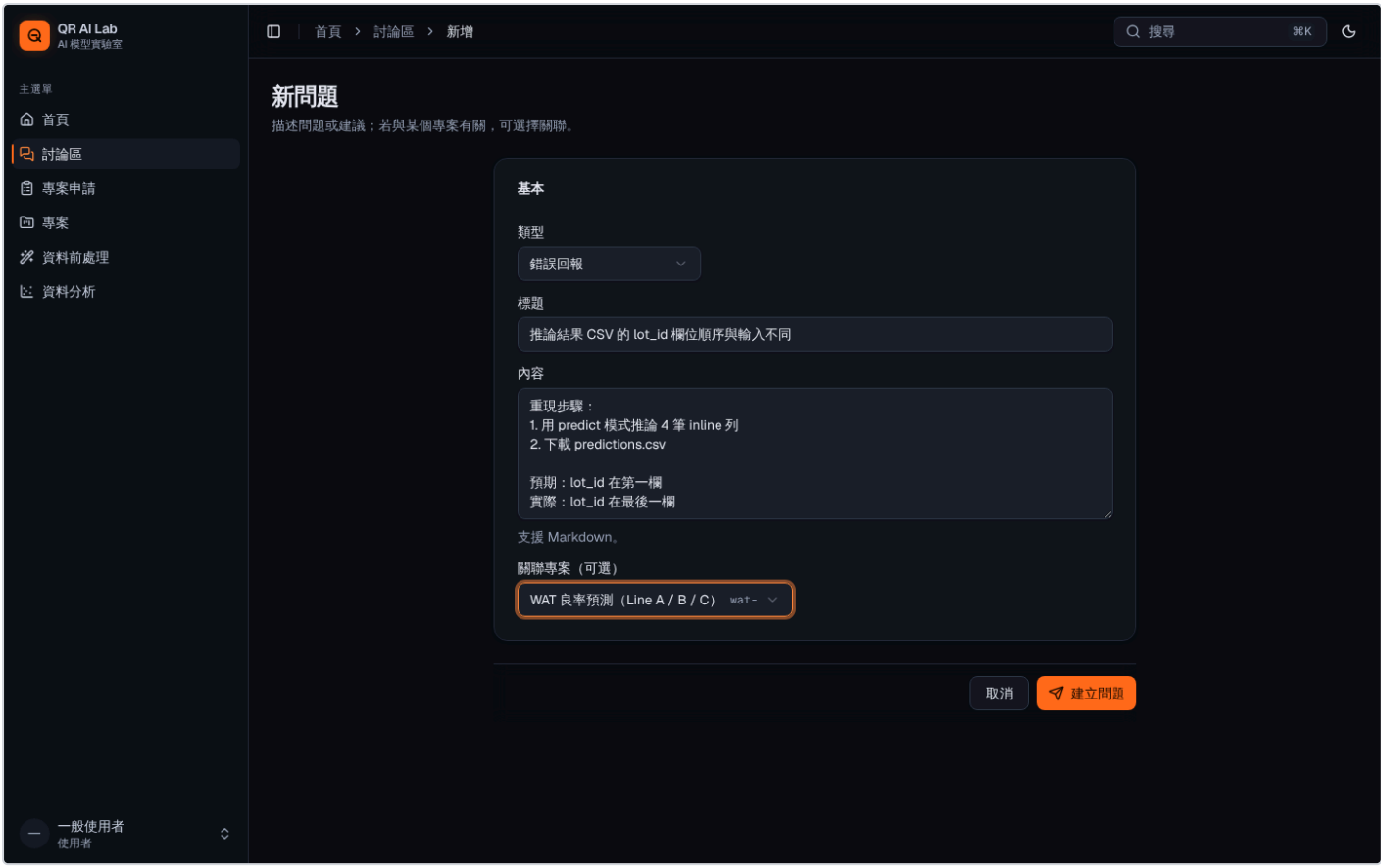
user 建立錯誤回報並關聯專案 → admin 回覆、寫處理結果並關閉 → 列表。

66 討論區（獨立於專案申請）：問題 / 錯誤回報 / 功能建議 dark



67 新增問題單：類型、標題、內容（Markdown）、關聯專案

dark



68 問題單詳情（open）

dark



69 管理員關閉問題單並寫處理結果

dark



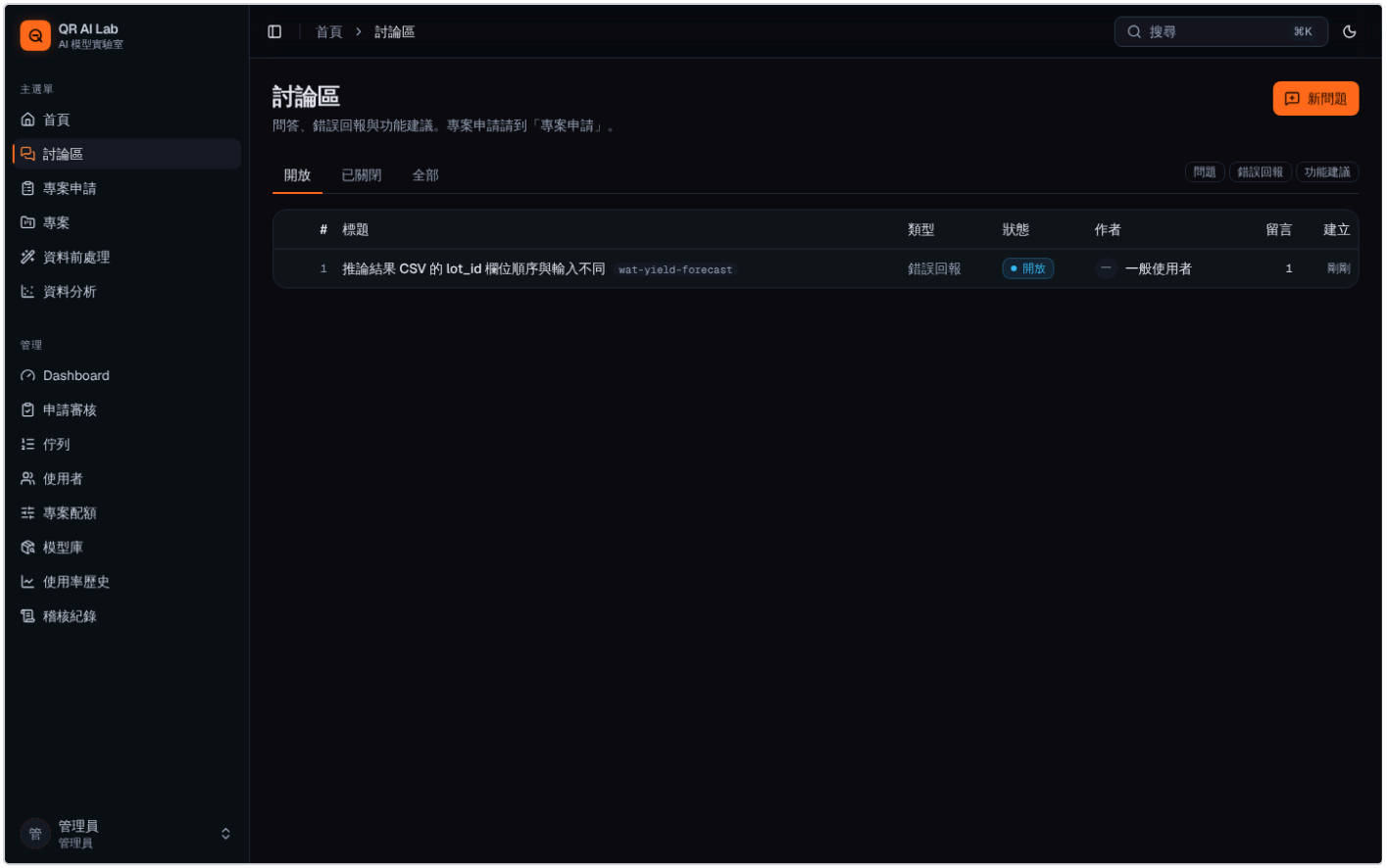
70 已關閉的問題單：處理結果卡 + 管理員回覆

dark



71 討論區列表：狀態、類型、關聯專案、回覆數

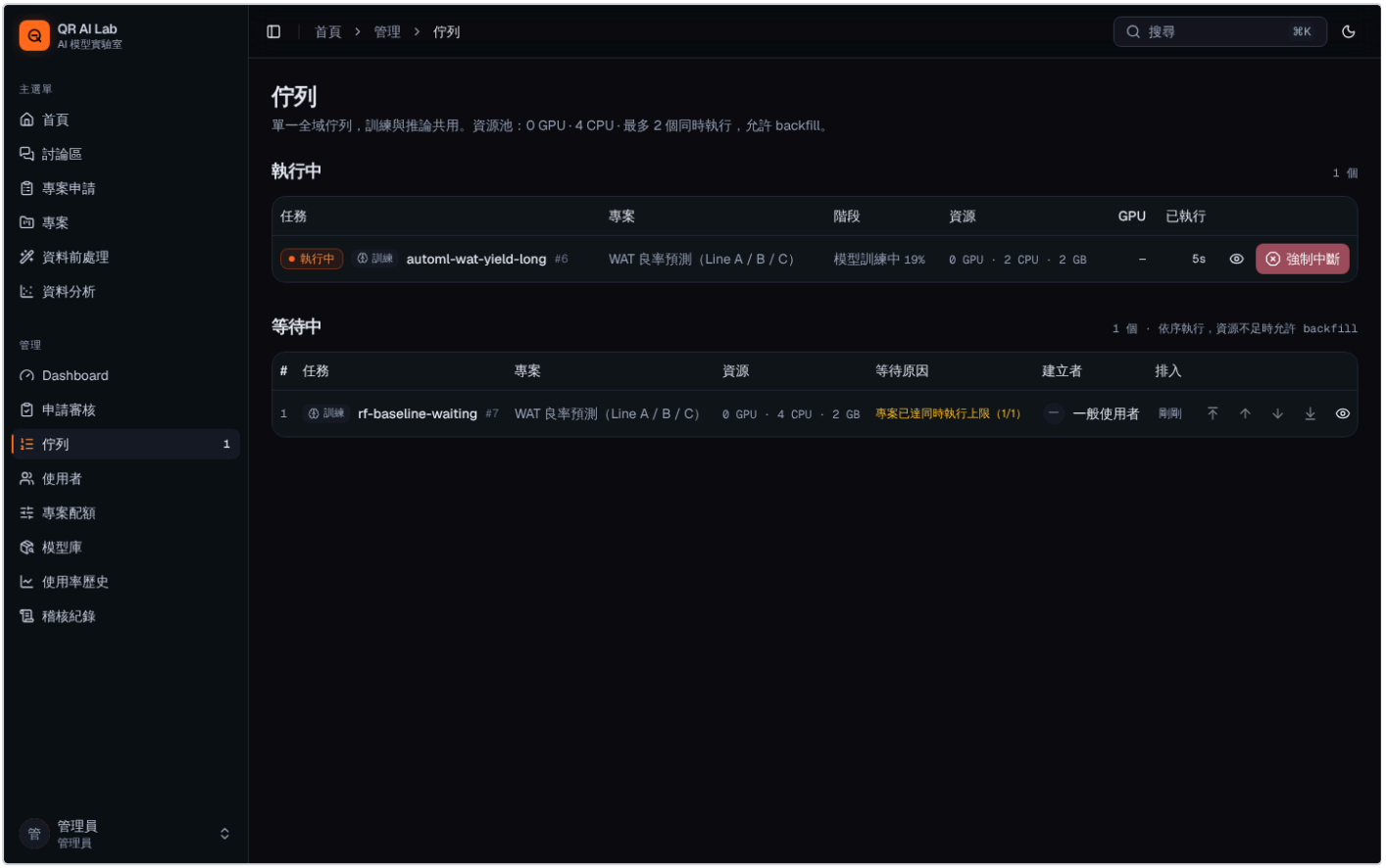
dark



9. 管理員：佇列、強制中斷、重新執行、稽核

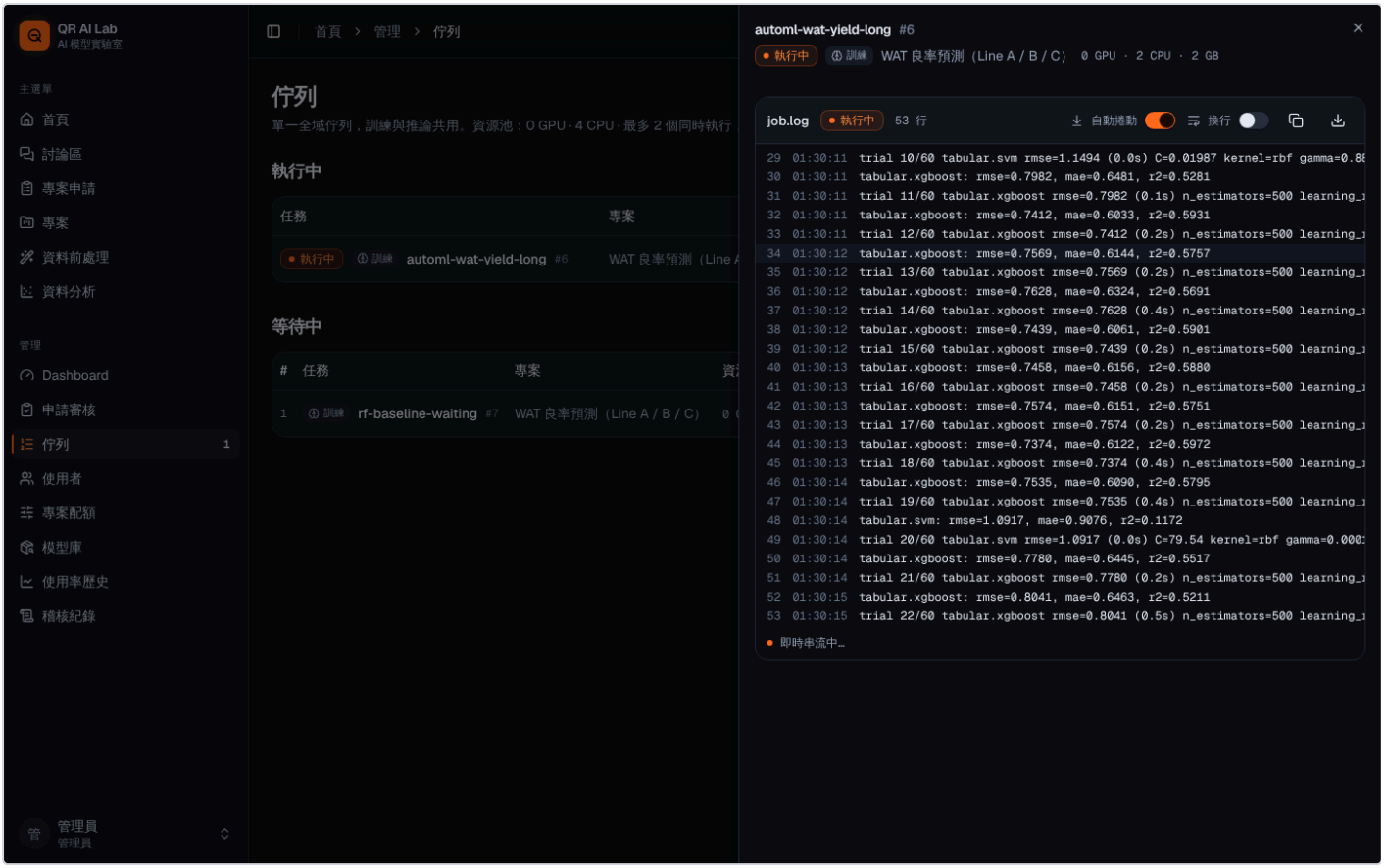
user 透過 API 建立一個 60 trial 的 AutoML 與一個等待中的任務 → admin 在佇列看到執行中 / 等待原因 → 任務抽屜看即時 log → 強制中斷（需理由）→ 等待中的任務接著執行 → 稽核紀錄 → Dashboard（資源池唯讀）→ 任務頁的管理員操作：重新執行成一個新任務；user 端看到被中斷的任務與理由。

72 管理員佇列：執行中的 AutoML（階段、進度、分配的資源）與等待中的任務及等待原因（專案已達同時執行上限） dark



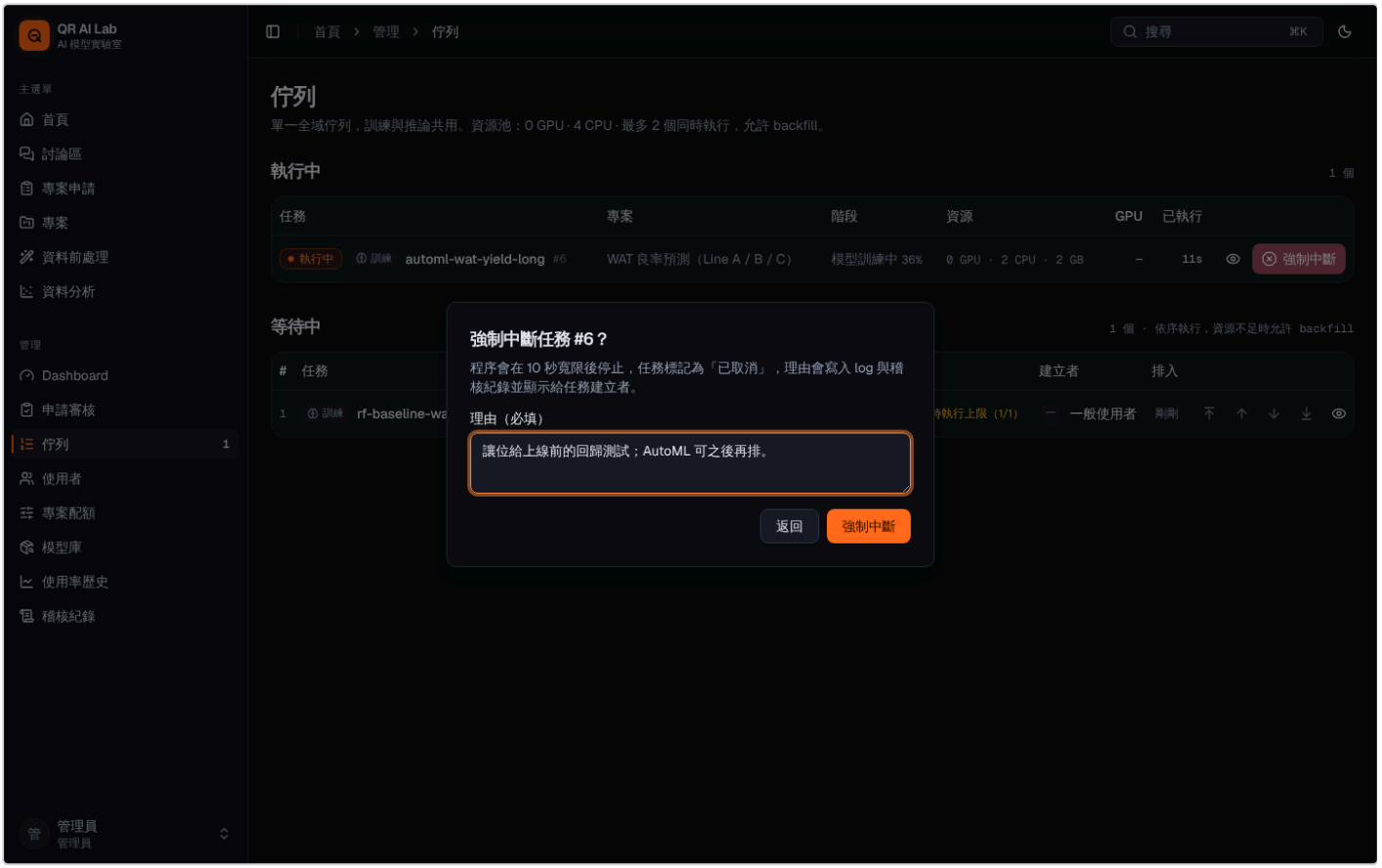
73 任務抽屜：與專案頁共用的 JobStatusBar + LogViewer（即時 log）

dark



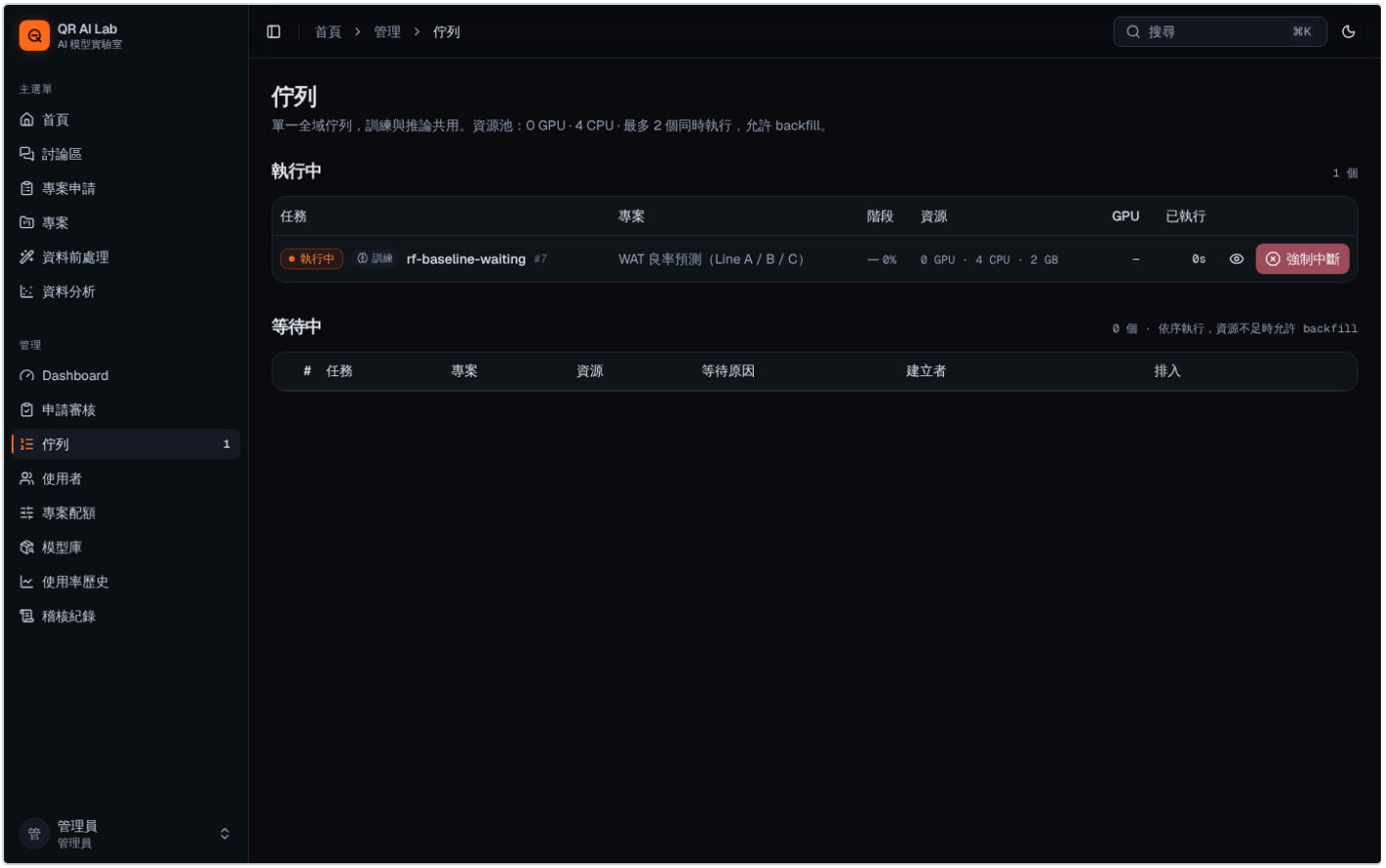
74 強制中斷需要理由（寫入 audit_logs，並顯示給任務建立者）

dark



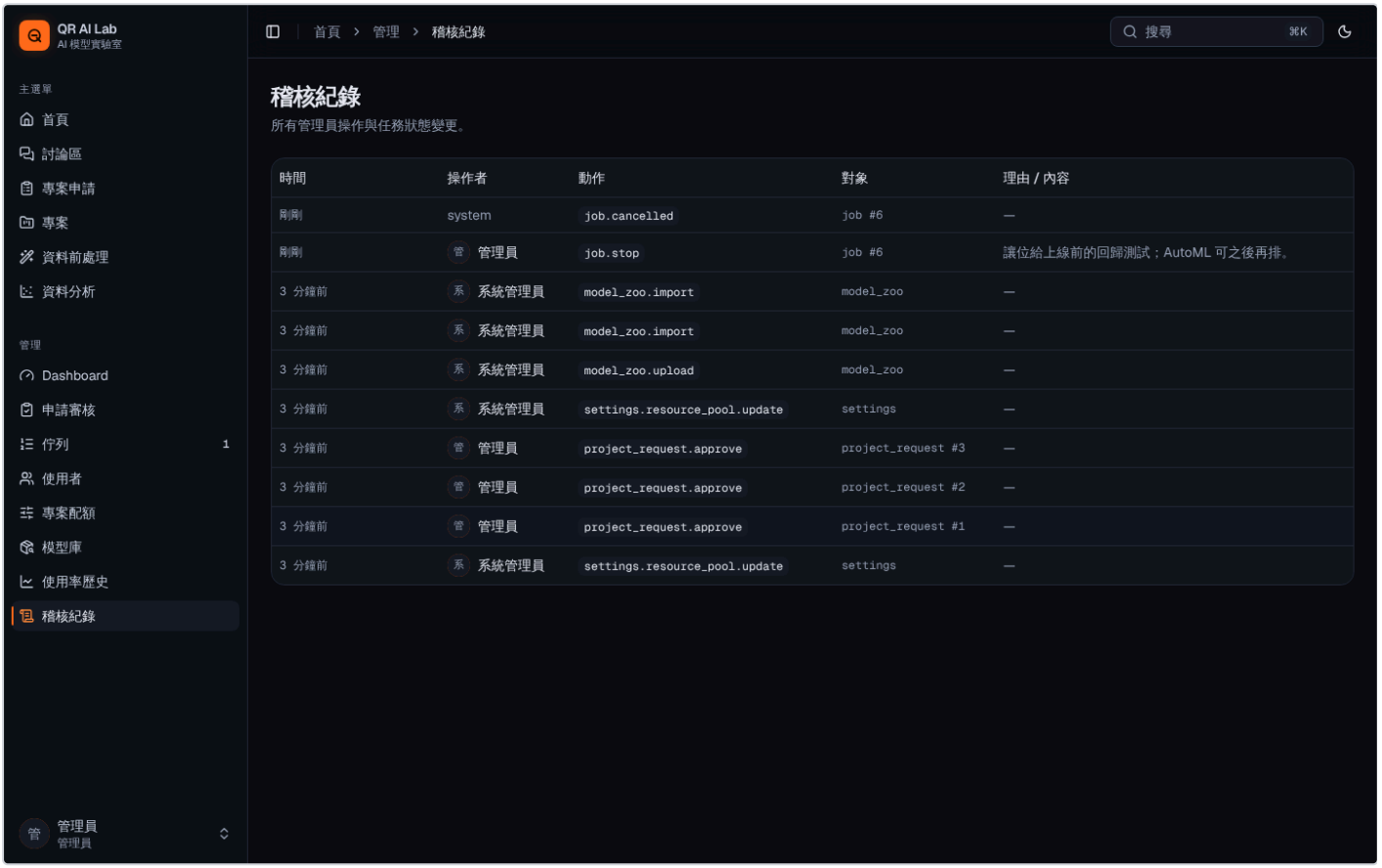
75 中斷後：等待中的任務立刻取得資源開始執行

dark

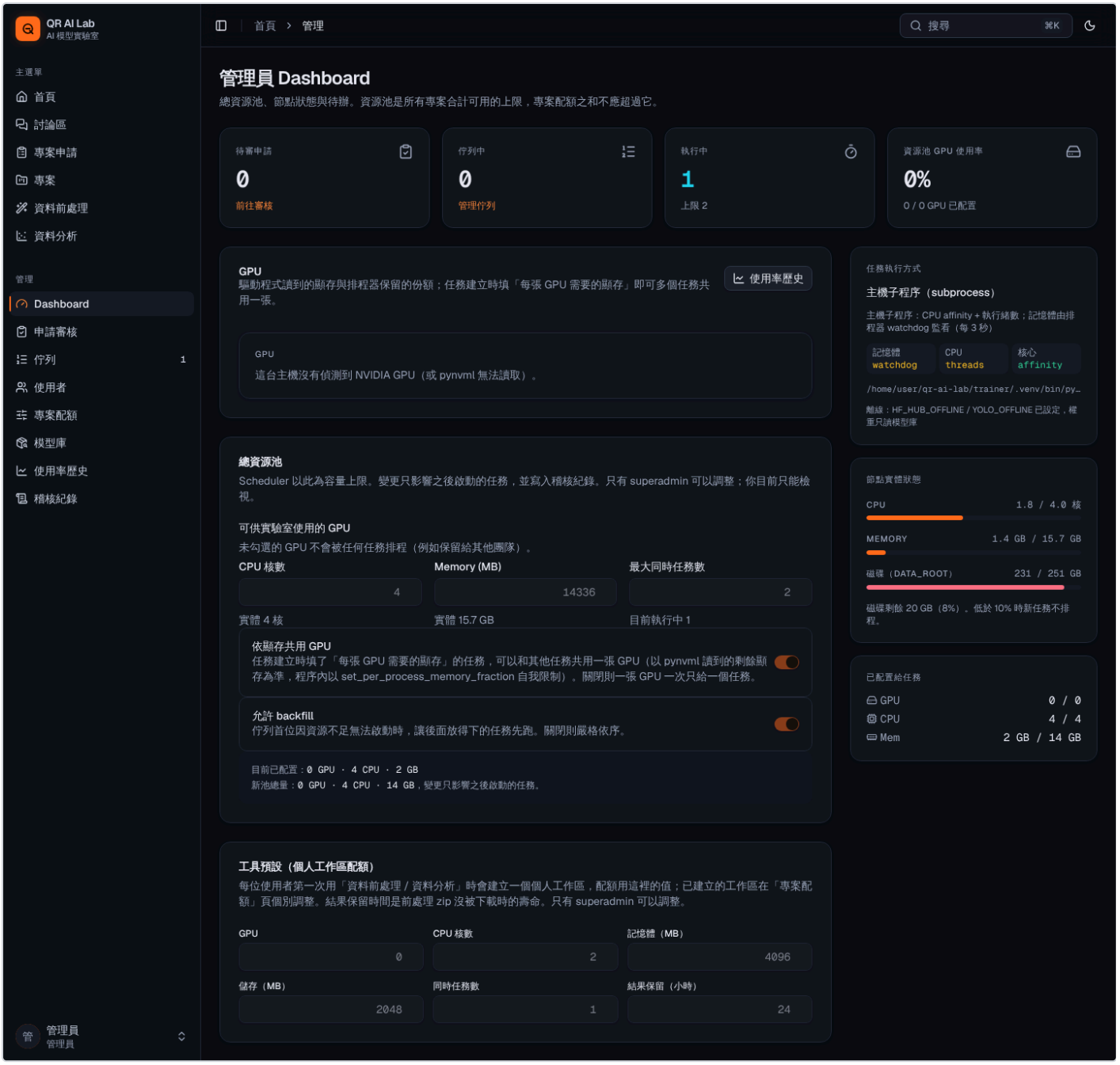


76 稽核紀錄：核准申請、更新資源池、強制中斷...都有操作者、對象與理由

dark



77 admin 的儀表板：GPU 卡片、資源池（唯讀，只有 superadmin 能改）、節點狀態與「任務執行方式」 dark



78 管理員在任務頁多了「重新執行」「刪除」（執行中則是「強制中斷」）；一次性上傳的任務不能重新執行 dark

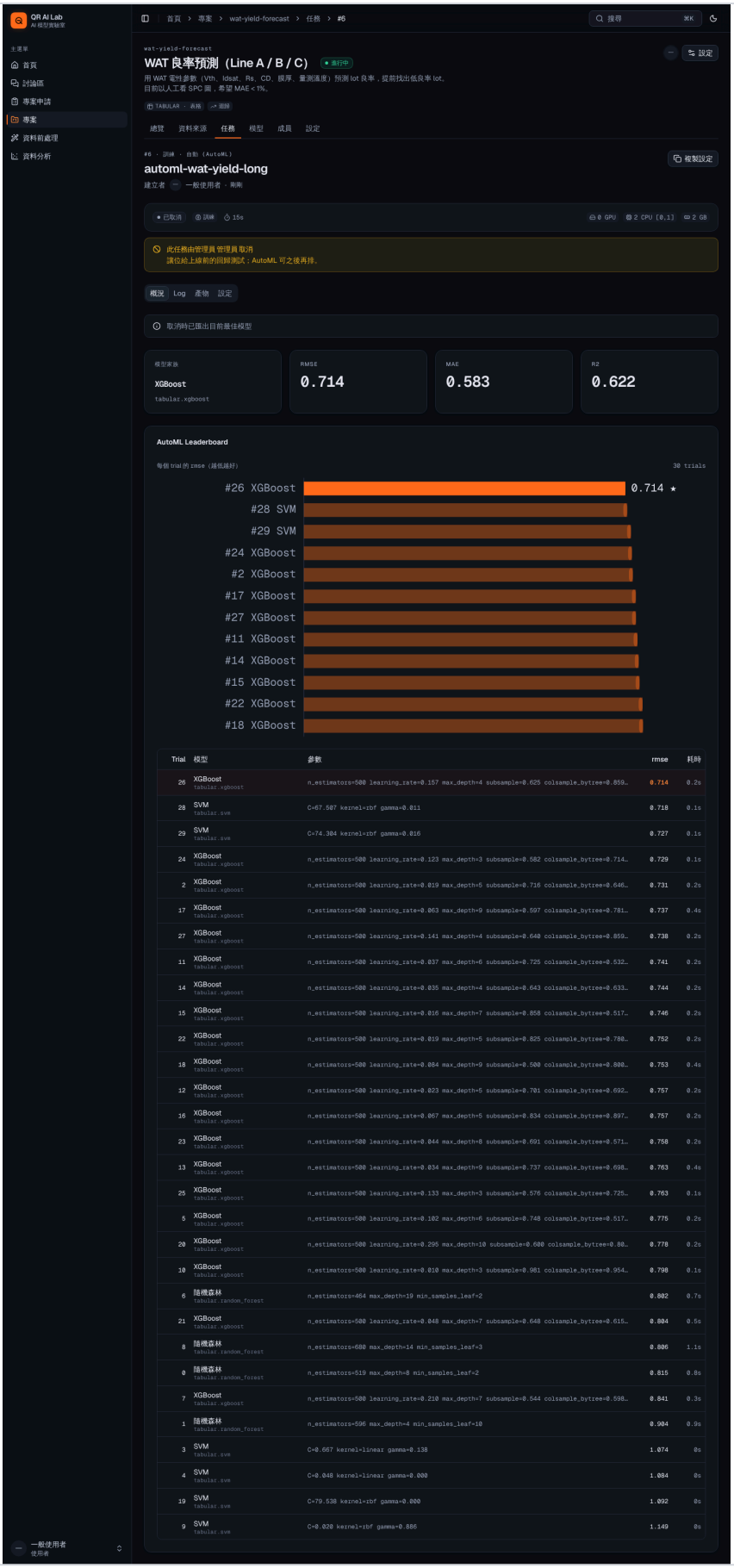


79 重新執行 = 新任務（config.restarted_from 指向原任務），原任務與 log 保留 dark



80 使用者看到的被中斷任務：誰中斷、理由；log 保留

dark



81 任務列表：成功 / 已取消 / 推論任務一覽

dark



10. 三種系統角色：superadmin / admin / user

superadmin 在使用者頁改角色（升 / 降、稽核）、刪除專案（輸入 slug 確認）；admin 看配額與模型庫是唯讀、可刪討論串；user 沒有管理選單，/admin 是 404。後端以 require_perm 判斷，前端只隱藏。

82 superadmin 的使用者頁：三種系統角色（superadmin / admin / user）的權限說明，可直接改角色與啟用 dark

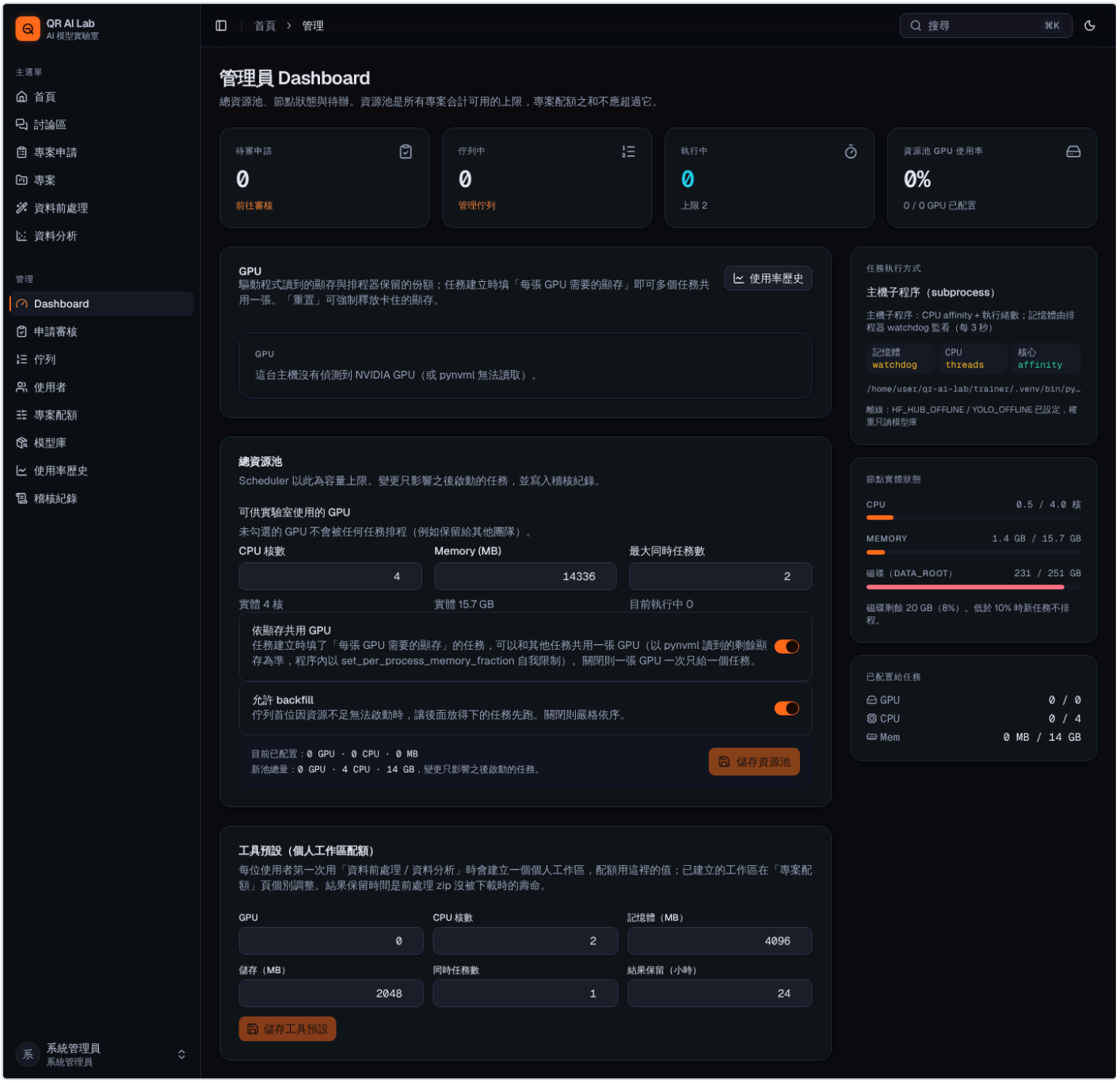


83 把 user 升為 admin：下一個請求就生效（後端每次都從 users 表讀角色），並寫入稽核紀錄 dark



84 superadmin 的儀表板：GPU 卡片有「重置」、資源池可編輯（含「依顯存共用 GPU」開關）

dark



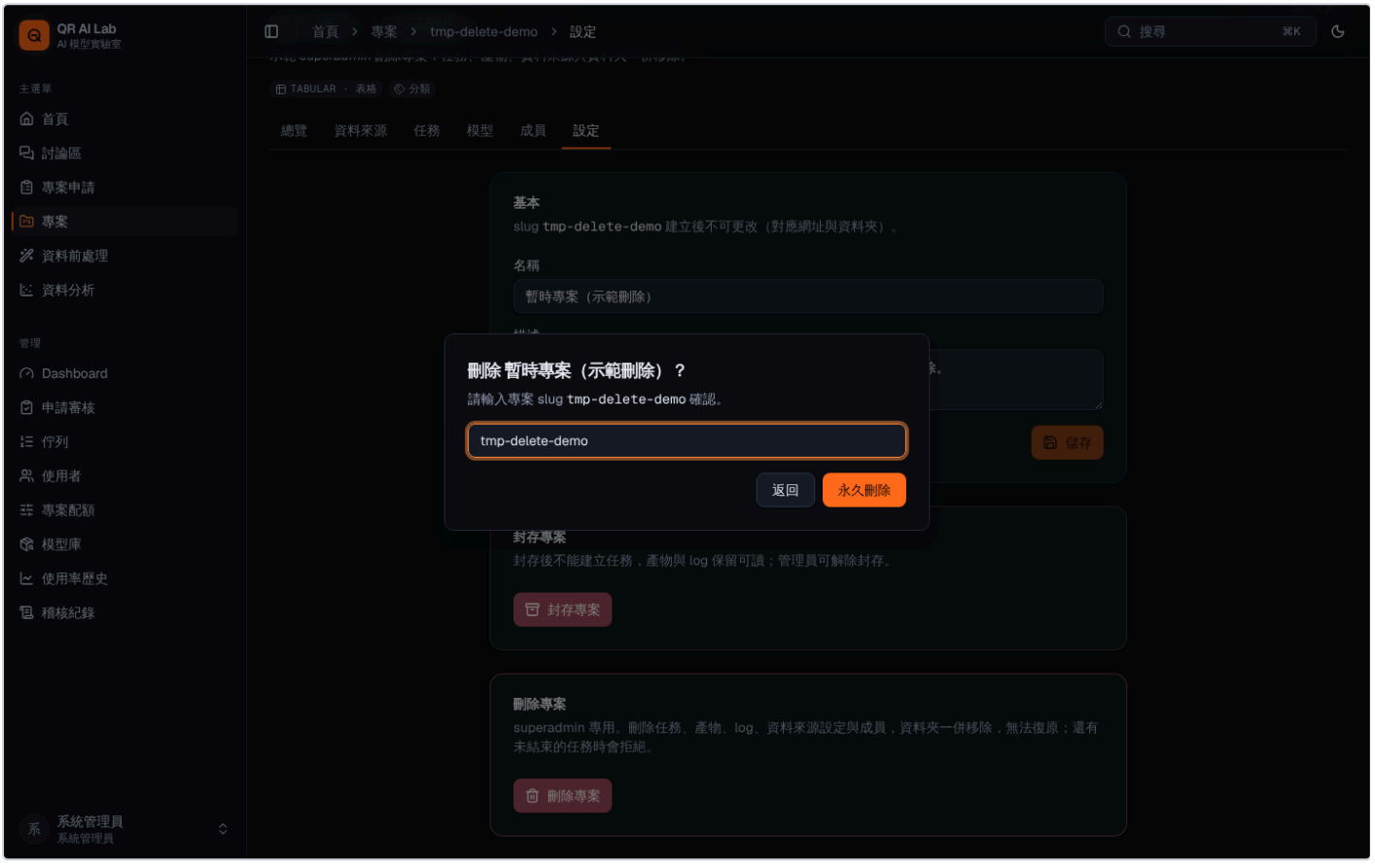
85 superadmin 的專案設定頁多了「刪除專案」（admin 只能封存）

dark



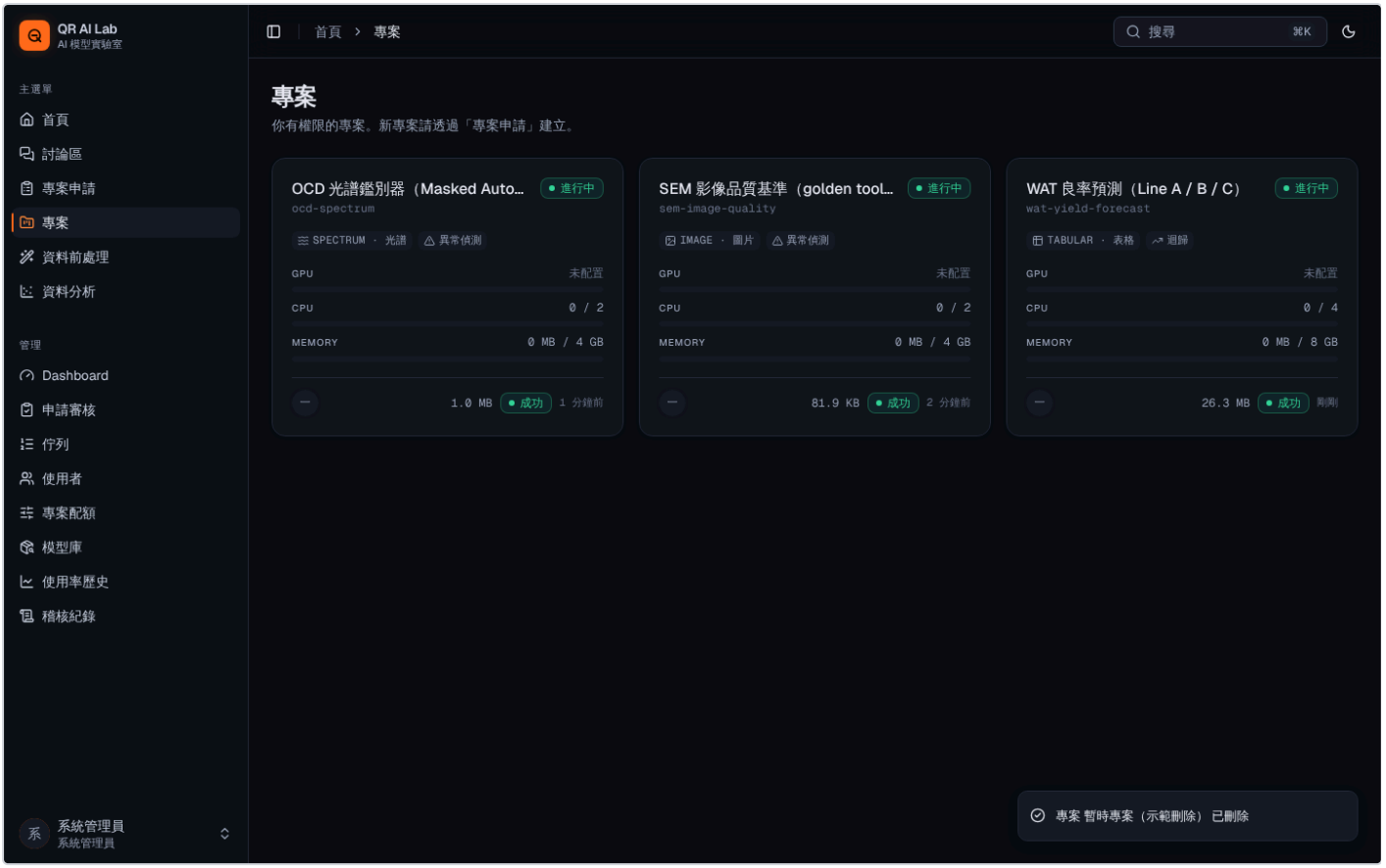
86 刪除要輸入 slug 確認；還有未結束任務時後端會拒絕（PROJECT_HAS_ACTIVE_JOBS）

dark



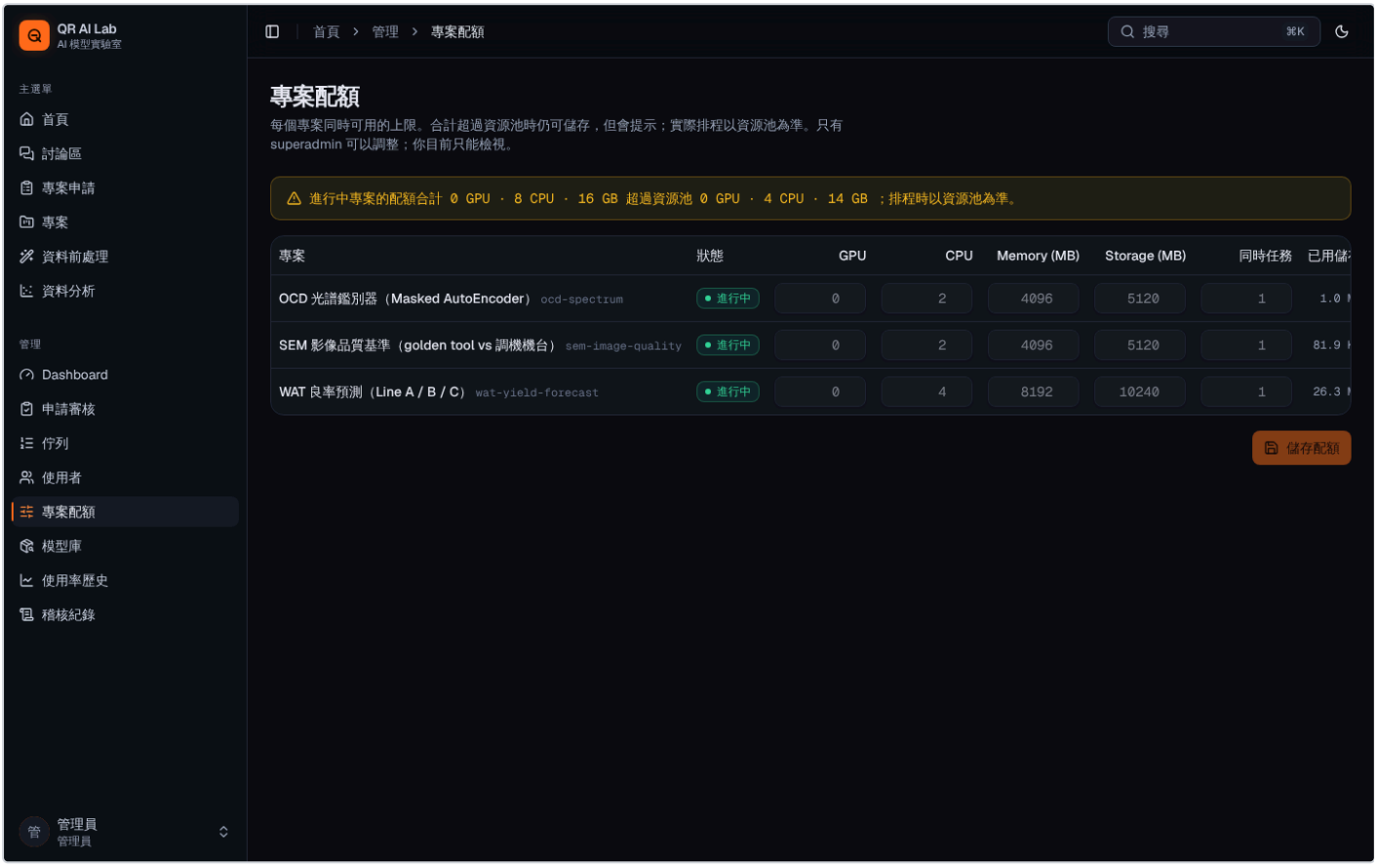
87 刪除後回到專案列表；稽核紀錄記下 slug 與刪掉的任務 / 產物數

dark

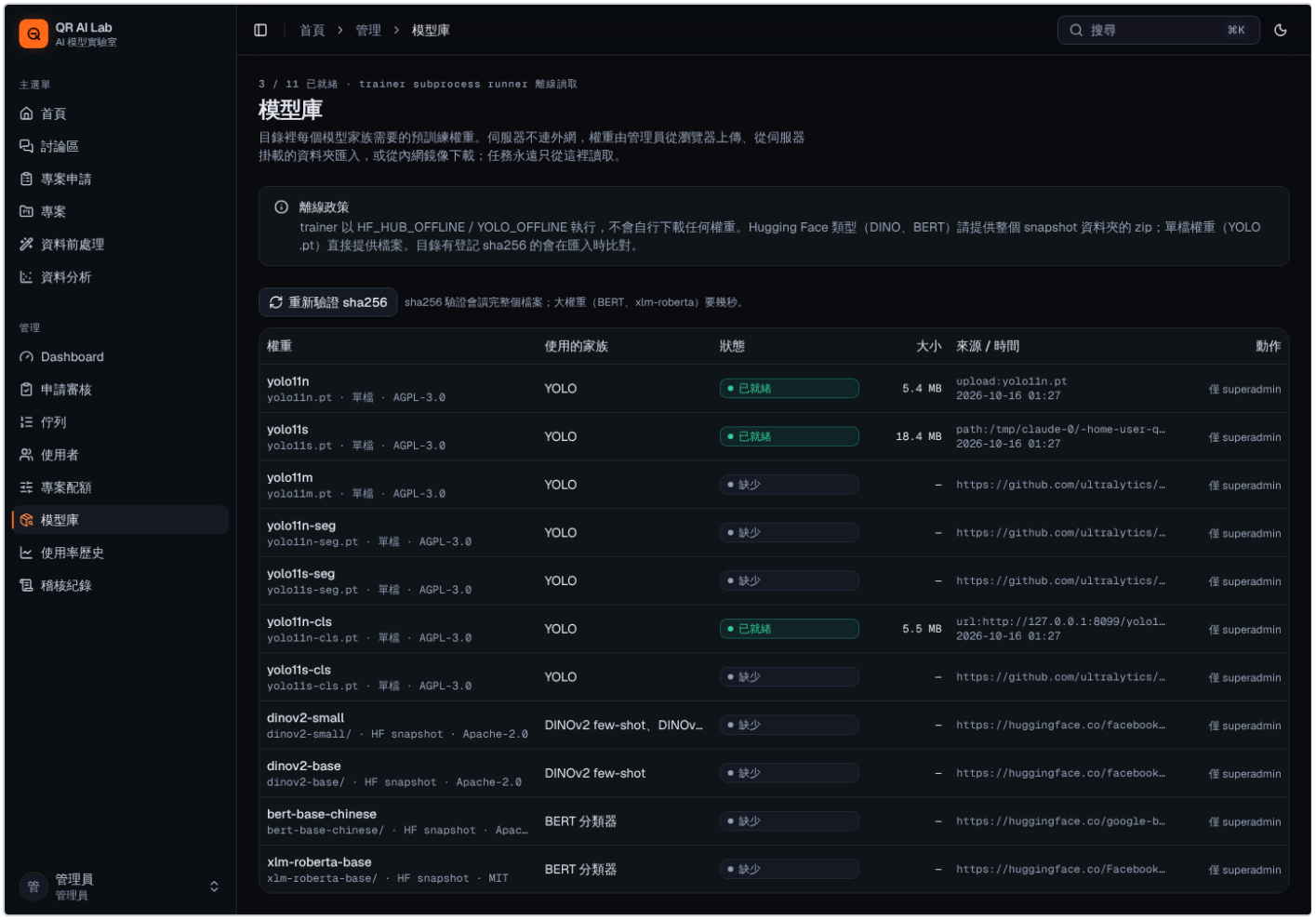


88 admin 看專案配額：唯讀（PERMISSION_DENIED 由後端把關，前端只是不給改）

dark



89 admin 看模型庫：只有狀態，沒有上傳 / 匯入 / 移除 dark



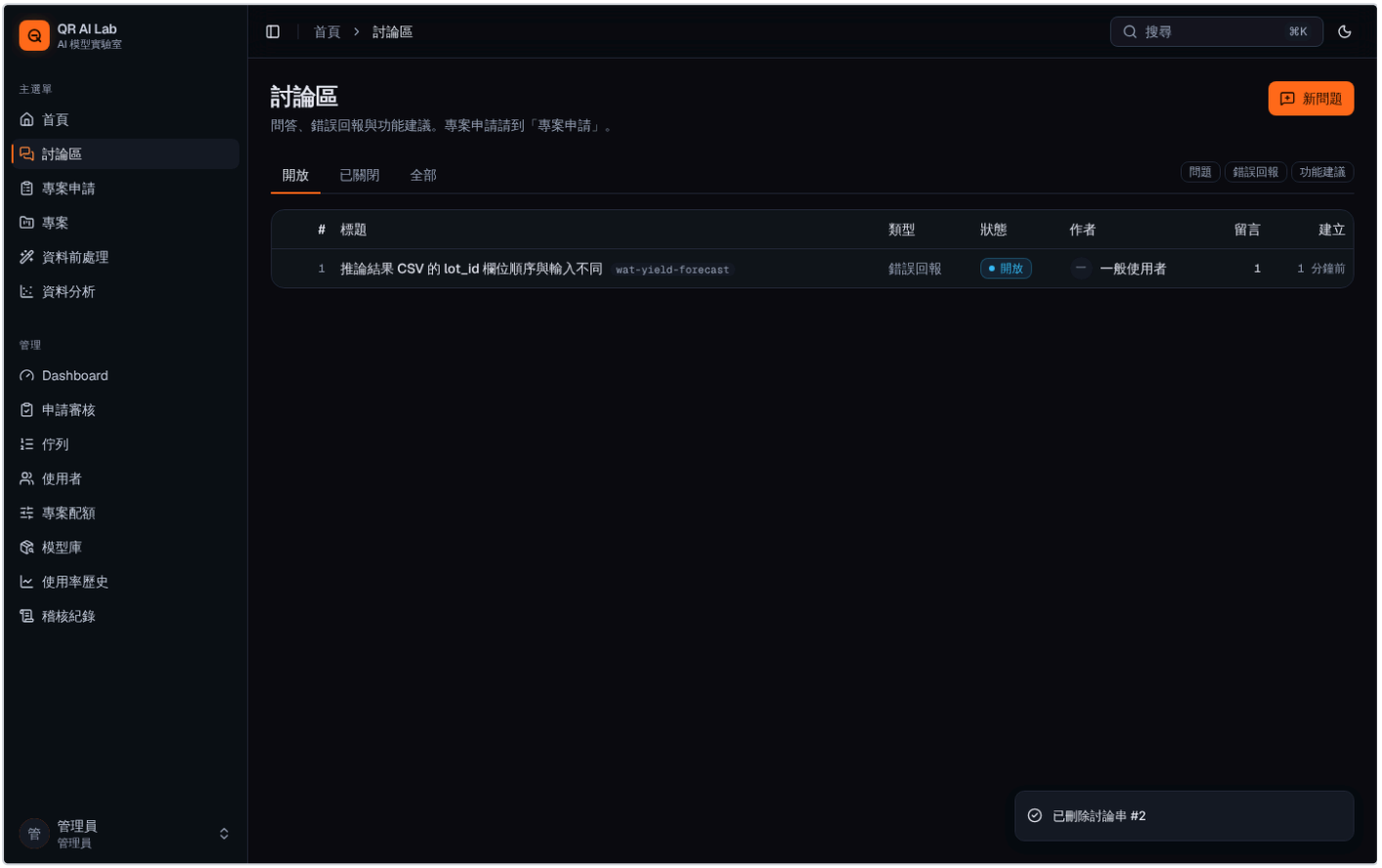
90 admin 刪除討論串（連同留言）需確認；寫入稽核紀錄

dark



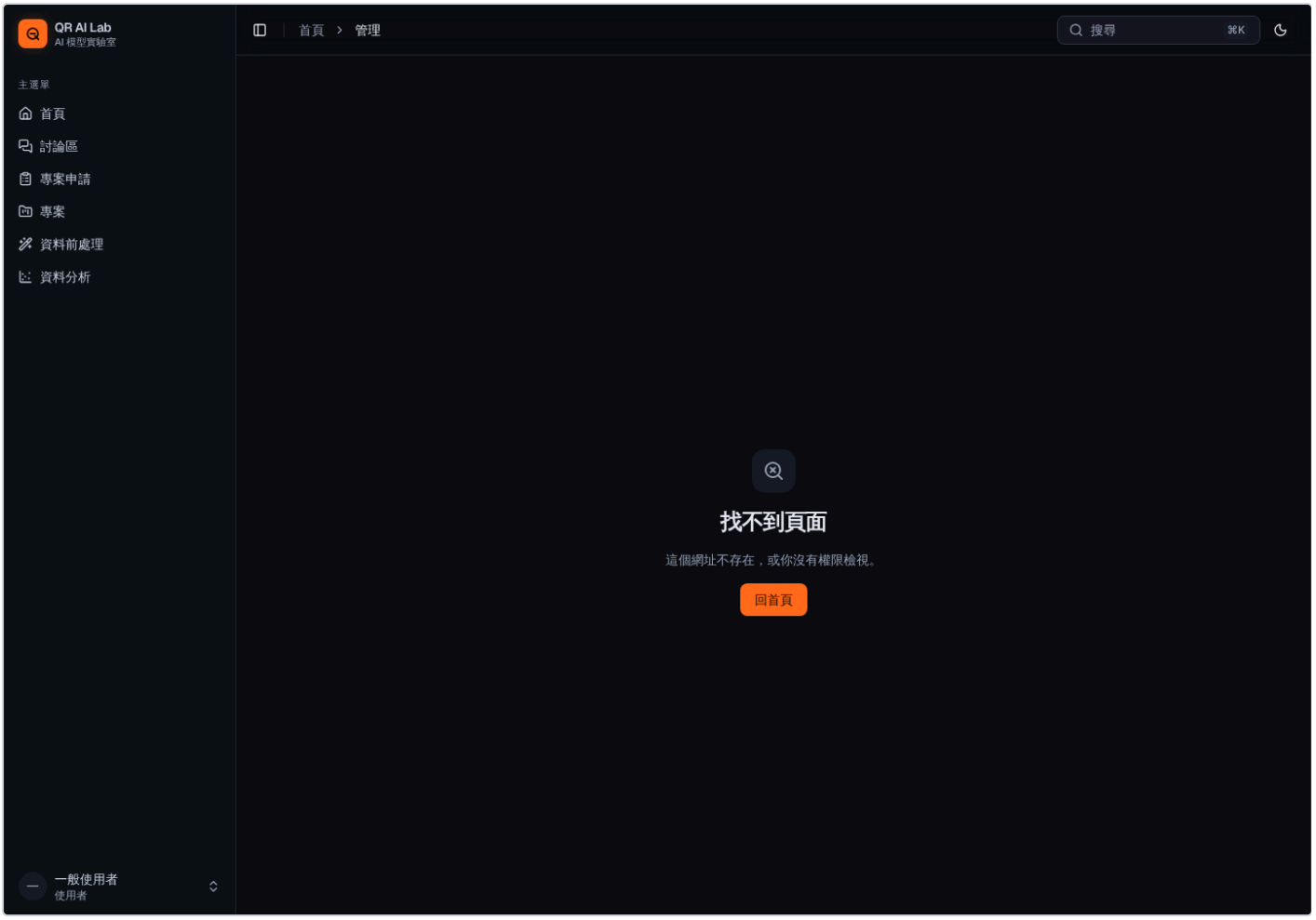
91 刪除後回到討論區列表

dark



92 一般使用者沒有「管理」選單；直接打 /admin 得到 404（後端 API 也回 403）

dark



11. 資源使用率歷史

排程器每 30 秒取樣節點 / 資源池 / 各專案；1 小時到 30 天分桶；分母可切專案配額 / 資源池 / 系統總量；各專案的時間加權占用與實測 CPU / 記憶體。

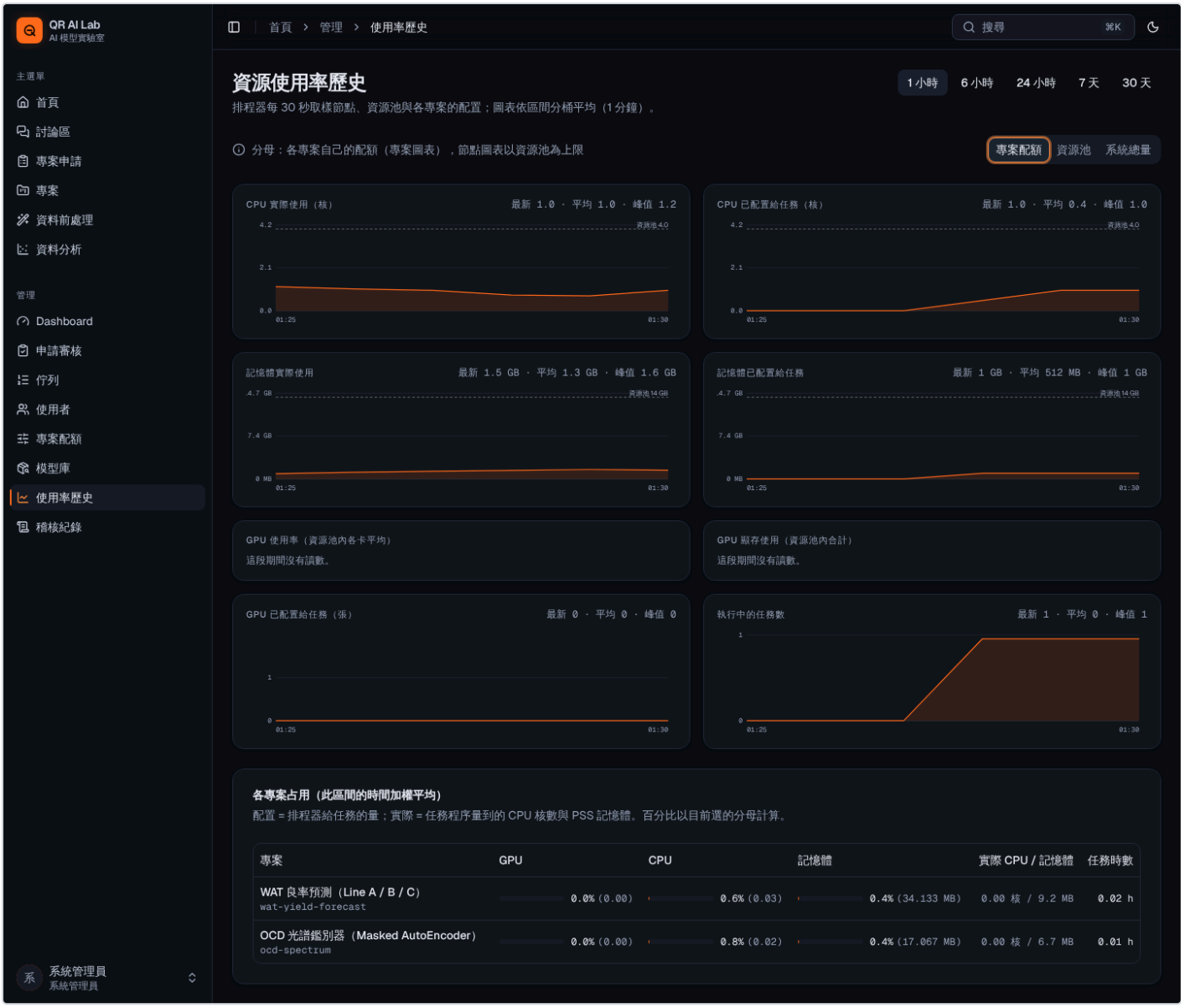
93 資源使用率歷史（1 小時）：節點實際 CPU / 記憶體、配置給任務的量、GPU 使用率與顯存、執行中任務數； dark 分母預設「資源池」



94 分母切成「系統總量」：虛線上限改為伺服器實際核數 / 記憶體；各專案占用也改以系統總量計算百分比 dark



95 分母「專案配額」：各專案的時間加權平均配置相對自己配額的百分比，以及實際量到的 CPU / 記憶體 dark



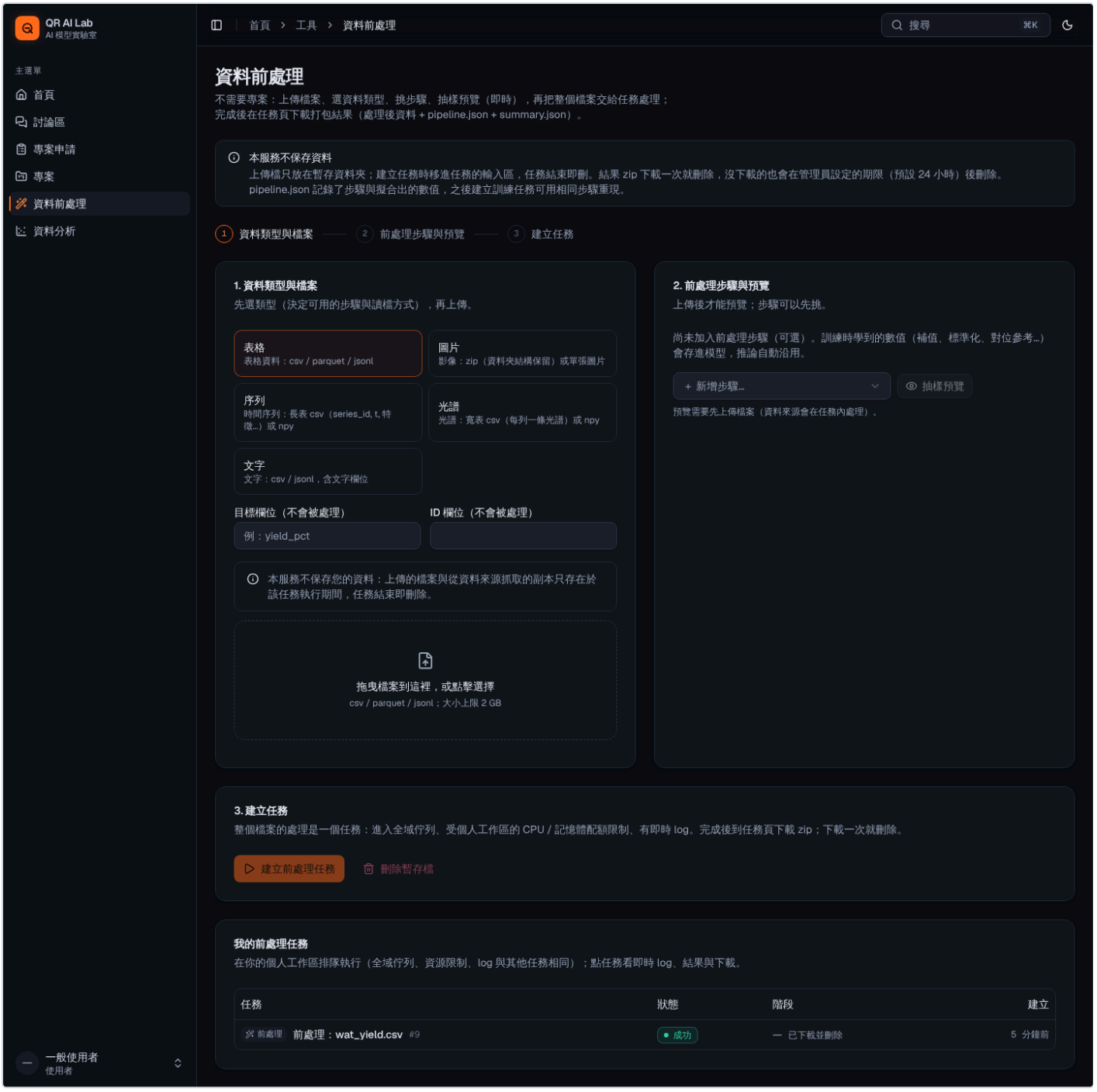
12. 資料前處理服務（任務模式、不保存）

選資料類型 → 上傳 → 指定不處理的欄位 → 加步驟並即時抽樣預覽 → 建立 preprocess 任務（個人工作區、全域佇列、配額、log）→ 任務頁下載 zip（processed.csv + pipeline.json + summary.json）一次即刪。

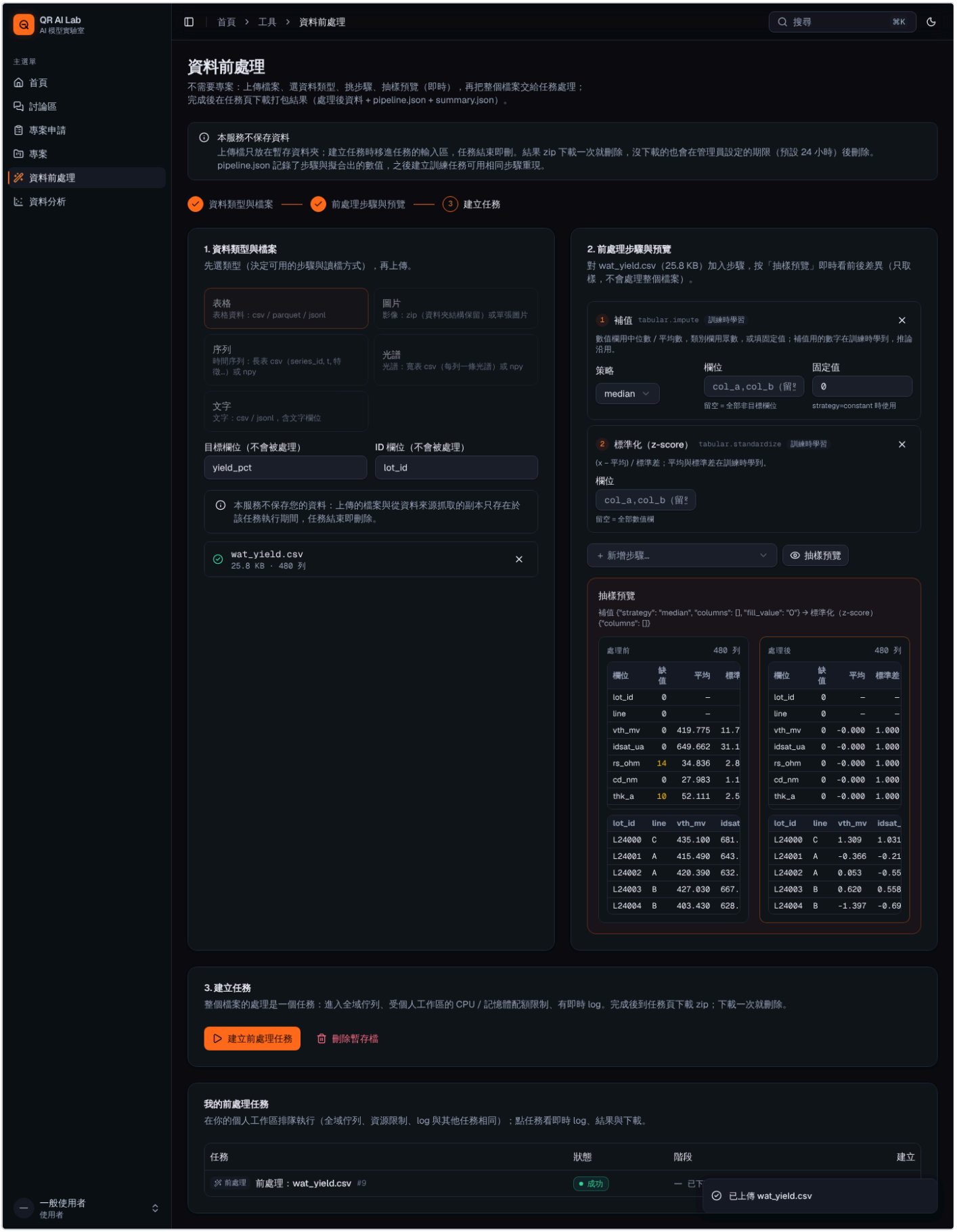
96 工具首頁：資料前處理與資料分析兩個不需專案的工具、個人工作區的配額（管理員「工具預設」）、我的工具任務 dark



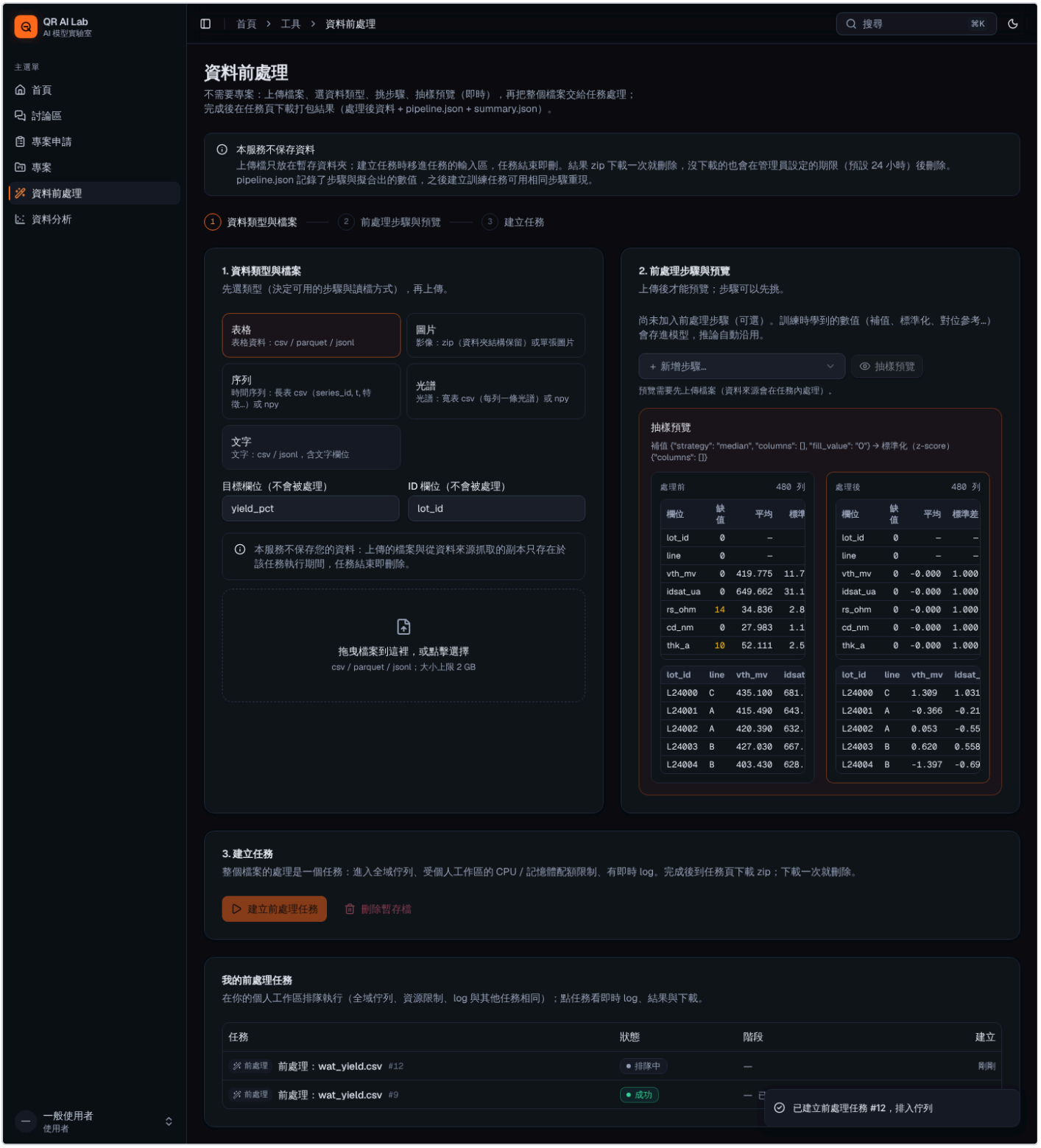
97 資料前處理服務：不需專案，上傳 → 選類型 → 挑步驟 → 即時抽樣預覽 → 建立任務（整檔處理在個人工作區排隊執行）；不保存資料 dark



98 上傳 WAT CSV、指定目標 / ID 欄位（不處理）、加入補值與標準化，抽樣預覽（即時，只取樣） dark



99 建立後成為個人工作區（ws-user）裡的 preprocess 任務：進佇列、受工具預設的配額限制，列表每 5 秒自動更新 dark



100 前處理任務頁：和其他任務一樣有 log（含 limits: 實際核心 / 執行緒）、步驟、檔案與一次性的下載按鈕 dark



101 下載 wat_yield-preprocessed.zip（processed.csv + pipeline.json + summary.json）後，結果檔立即從伺服器刪除；任務與 log 保留 dark



12b. 資料分析服務

表格資料探勘（相關性、VIF 共線性、對目標的解釋力與建議）與影像品質分析（三台機台的 IQA 指標飄移、embedding 散佈與相似度、離群影像），都是個人工作區的 analysis 任務，報告在任務頁。

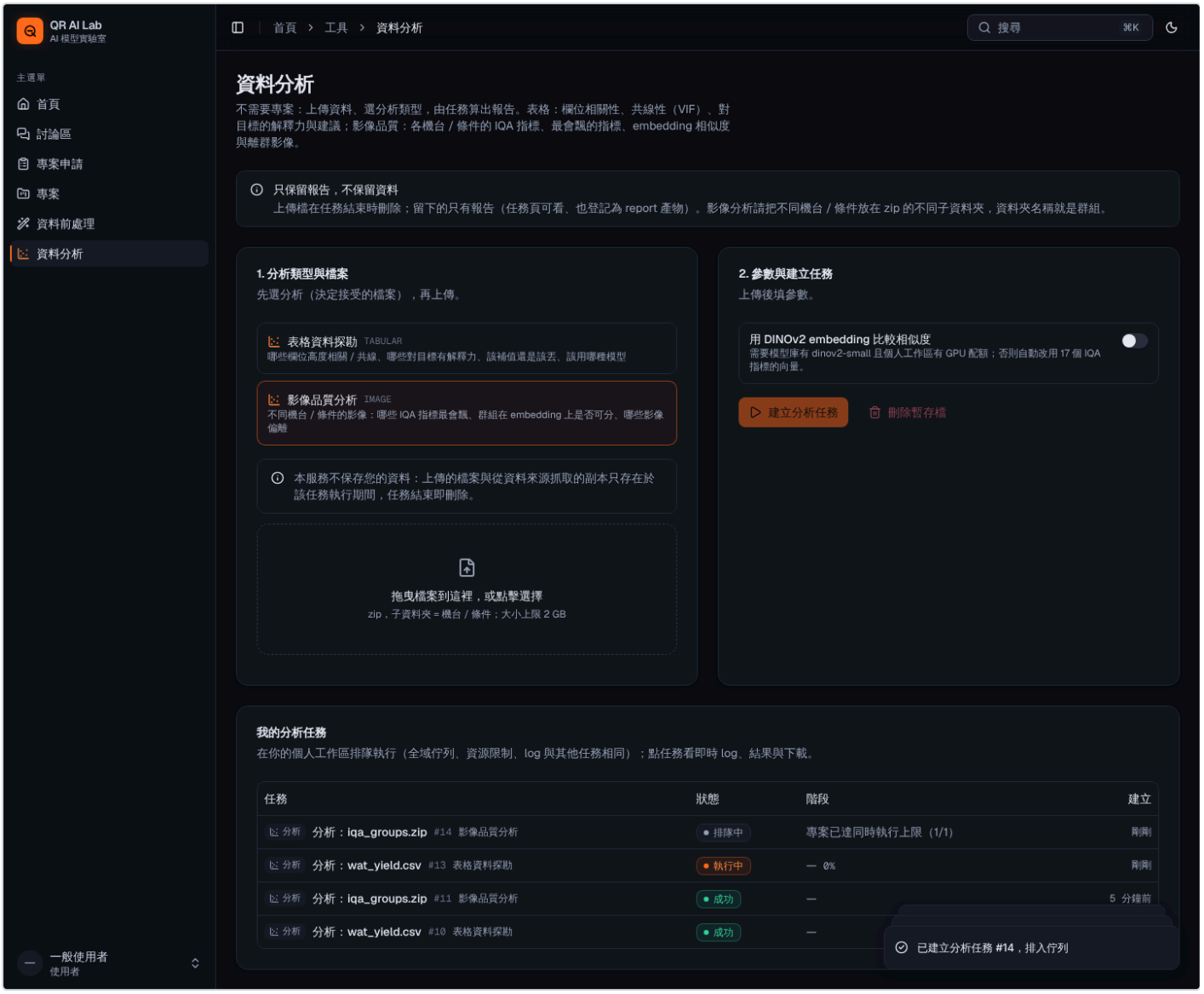
102 資料分析服務：表格資料探勘或影像品質分析，同樣以個人工作區的任務執行；只留報告不留資料 dark



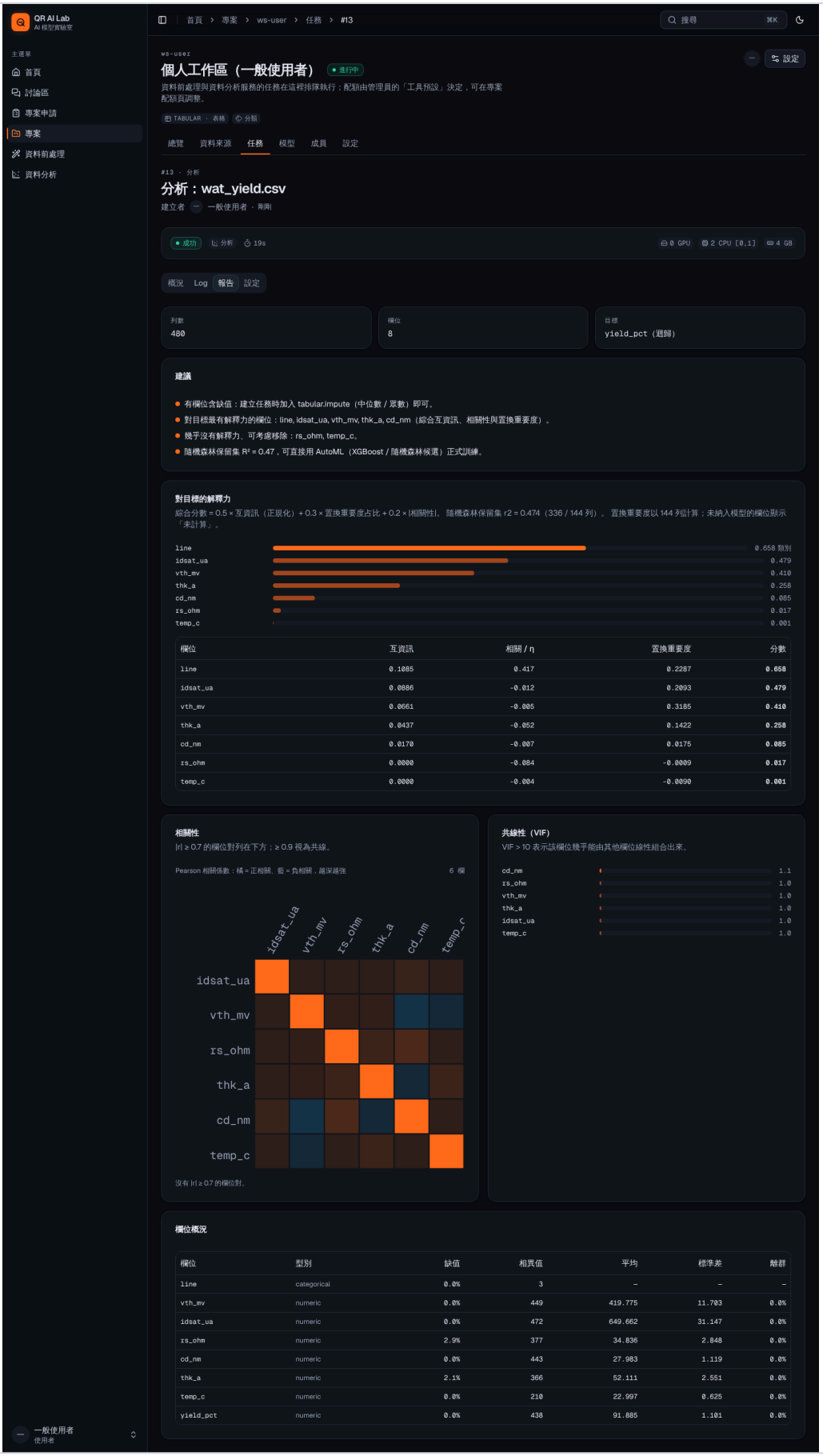
103 影像品質分析：zip 的子資料夾（golden / tuned / bad 三台機台）就是群組；可選 DINOv2 embedding（沒有 GPU 配額時自動改用 IQA 指標向量） dark



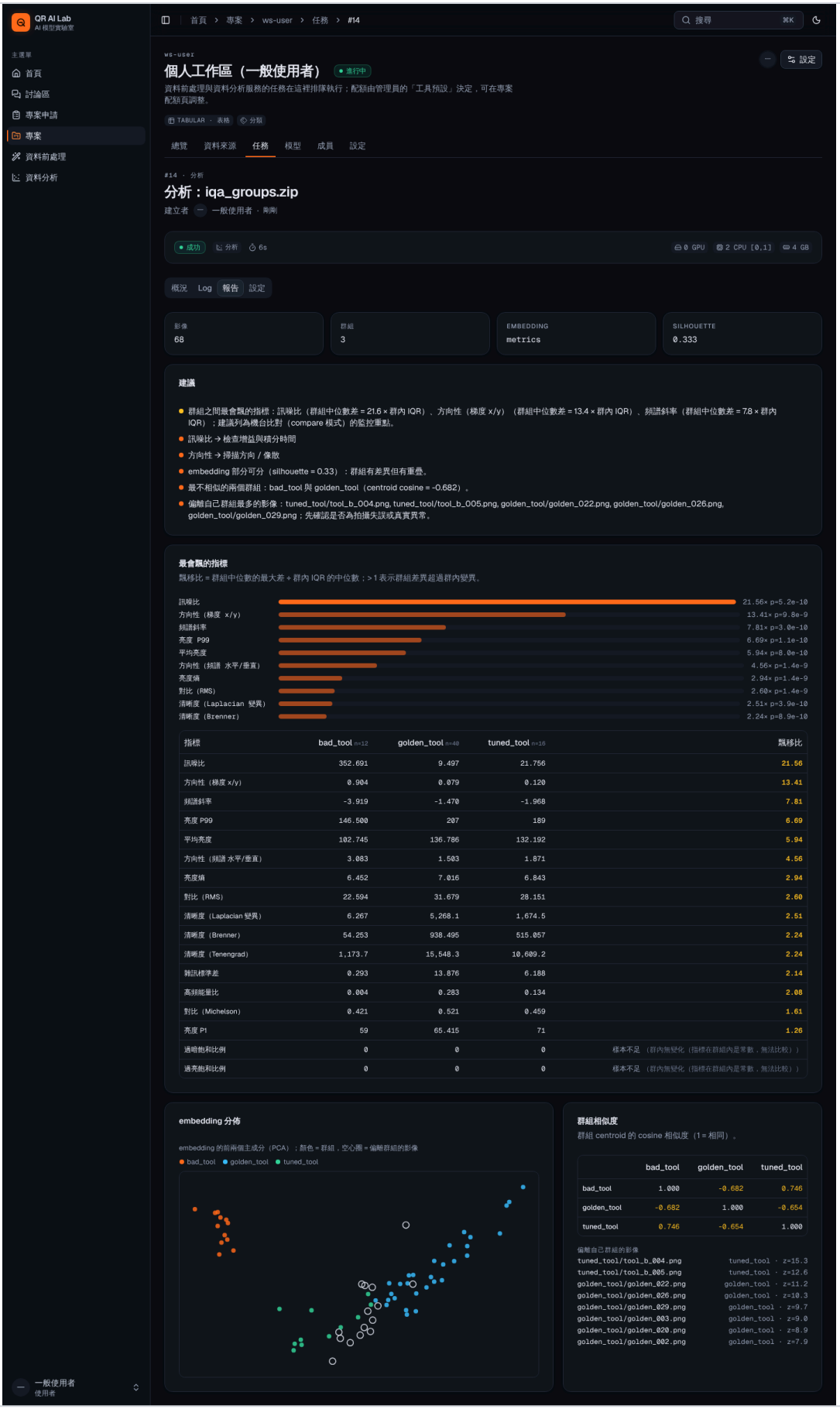
104 兩個分析任務排隊中（個人工作區同時只跑 1 個） dark



105 表格資料探勘報告：建議（共線、缺值、解釋力、模型選擇）、對目標的解釋力（互資訊 + 相關性 + 置換重要度）、相關矩陣與 VIF、欄位概況 dark



106 影像品質分析報告：三台機台最會飄的 IQA 指標（飄移比、p 值、各群組中位數）、embedding PCA 散佈與群組相似度、偏離群組的影像、建議 dark

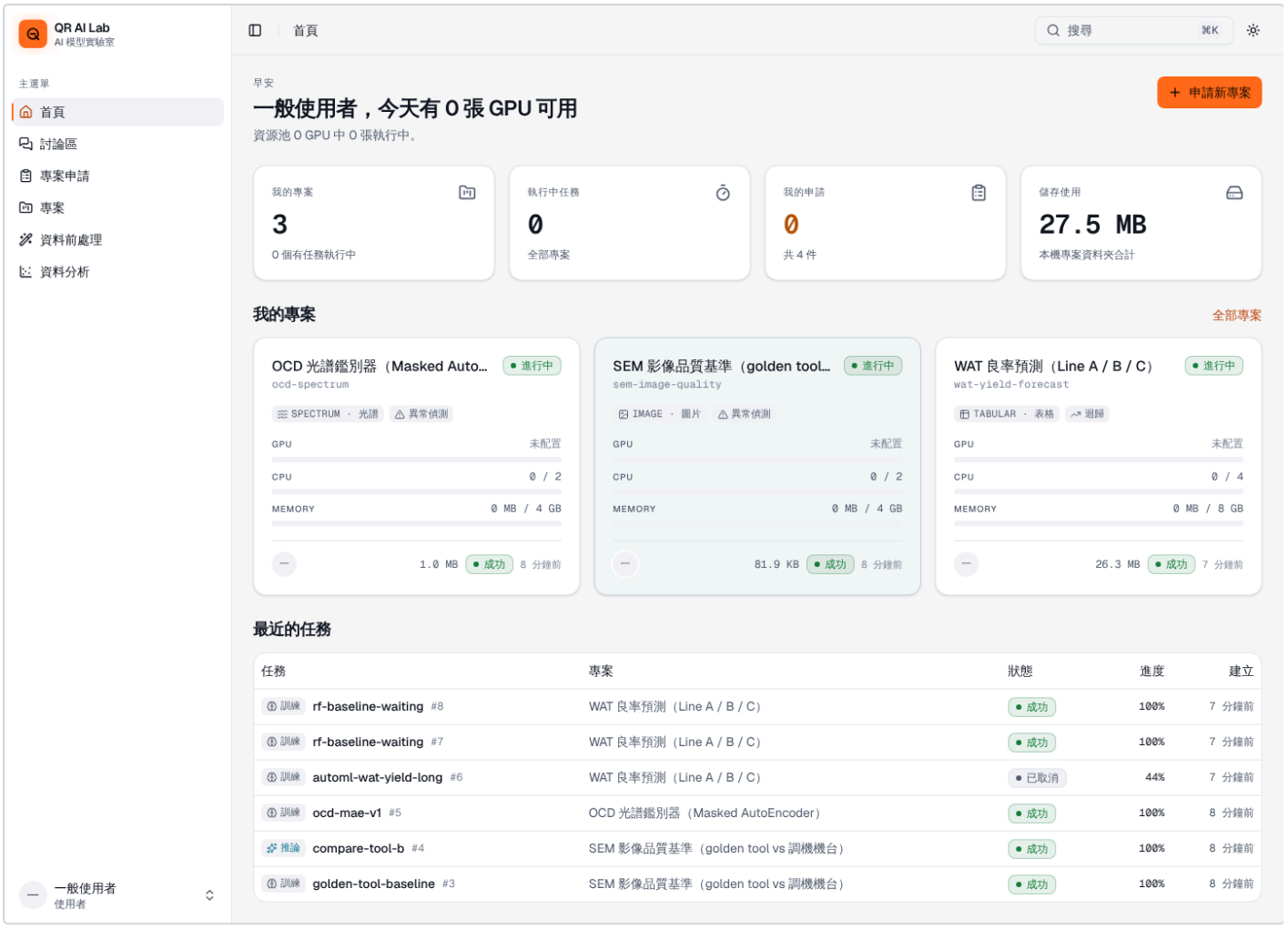


13. Light mode 與手機版

同一套語意 token 的淺色主題（含使用率歷史）；手機寬度側欄收成抽屜。

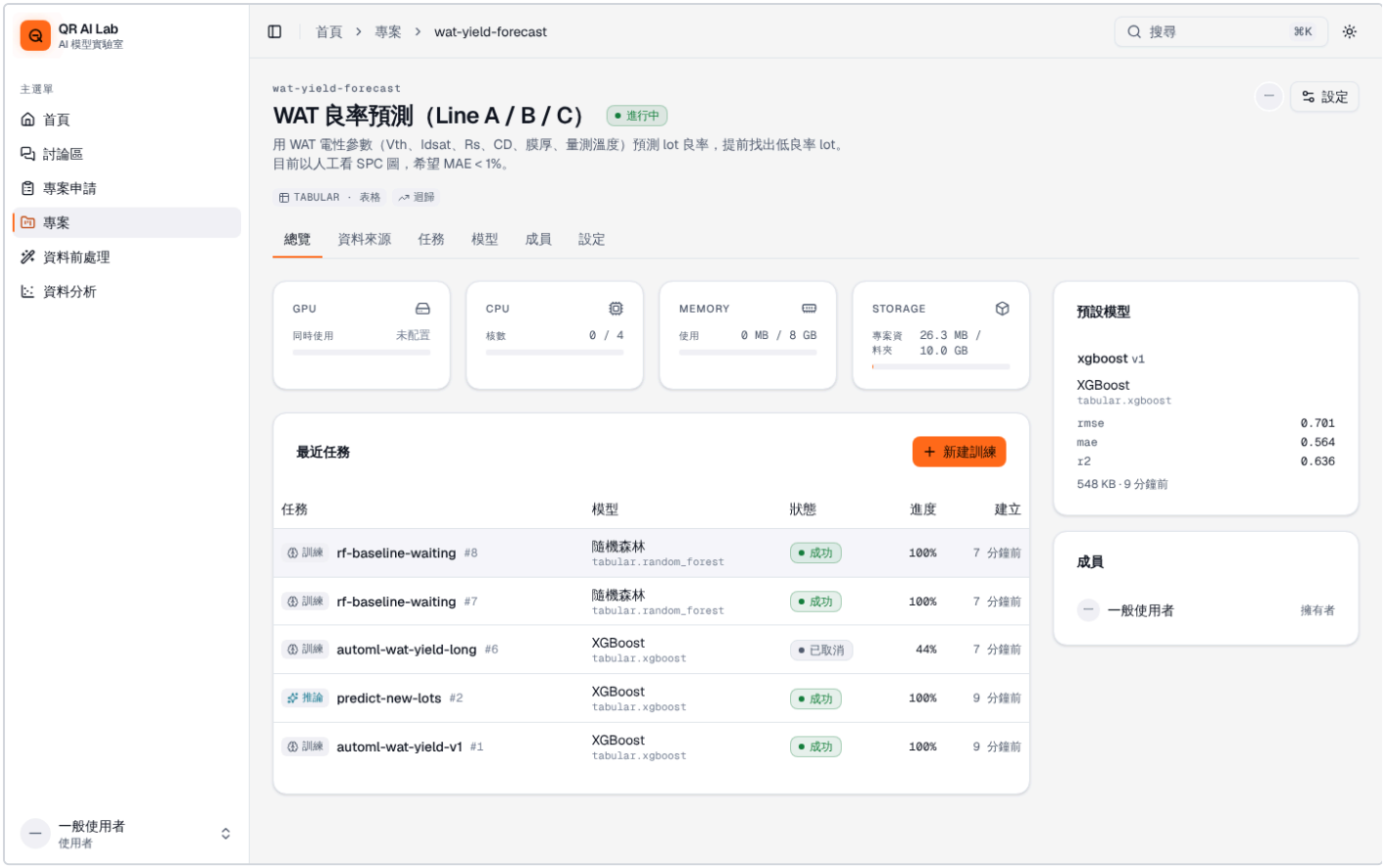
107 Light mode：首頁

light



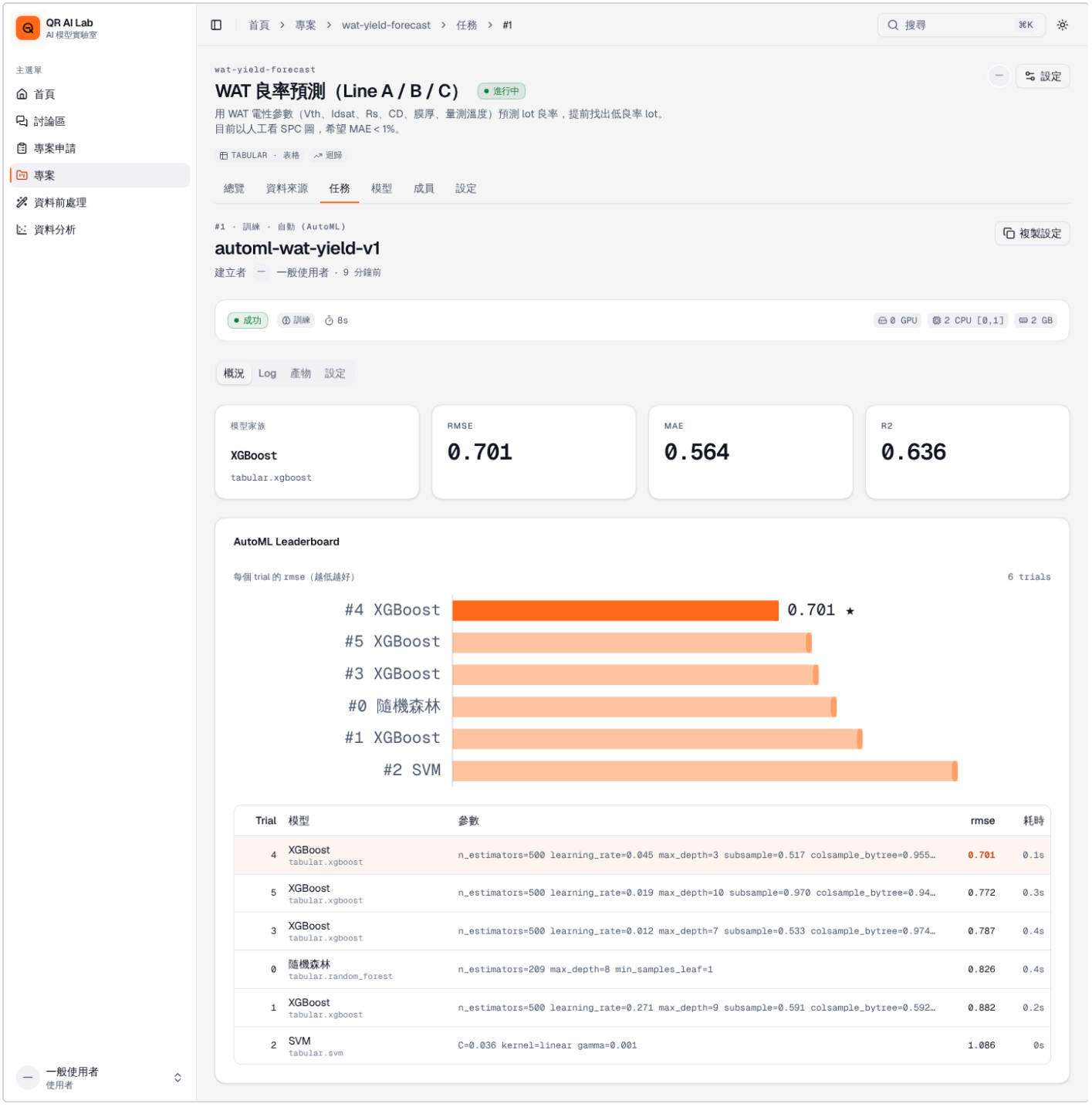
108 Light mode：專案總覽

light



109 Light mode：AutoML 任務詳情（leaderboard）

light



110 Light mode：建立任務精靈

light



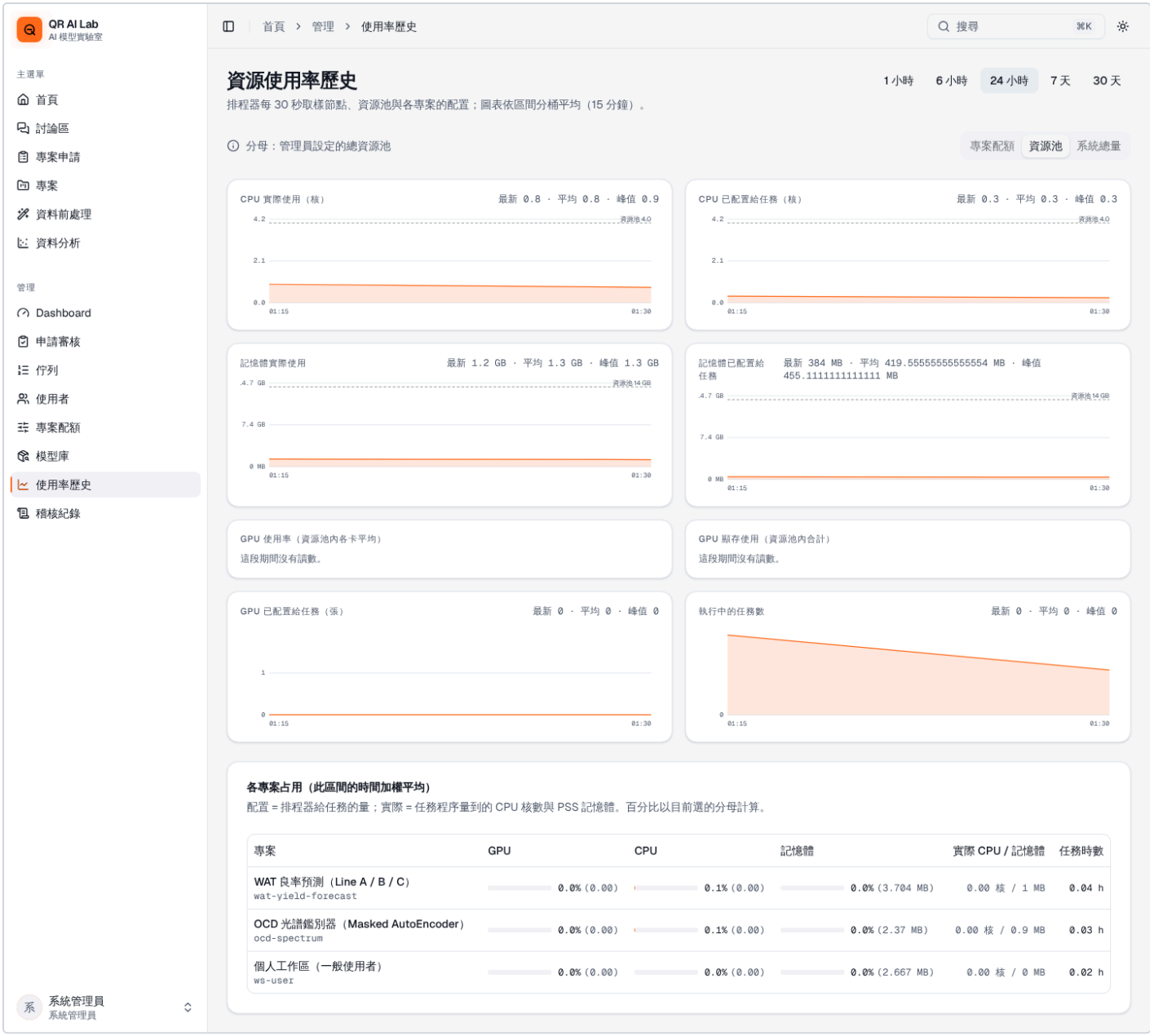
111 Light mode：管理員佇列

light



112 Light mode：資源使用率歷史

light



113 手機版首頁（側欄收成抽屜）

dark · 390px



114 手機版側欄抽屜

dark · 390px

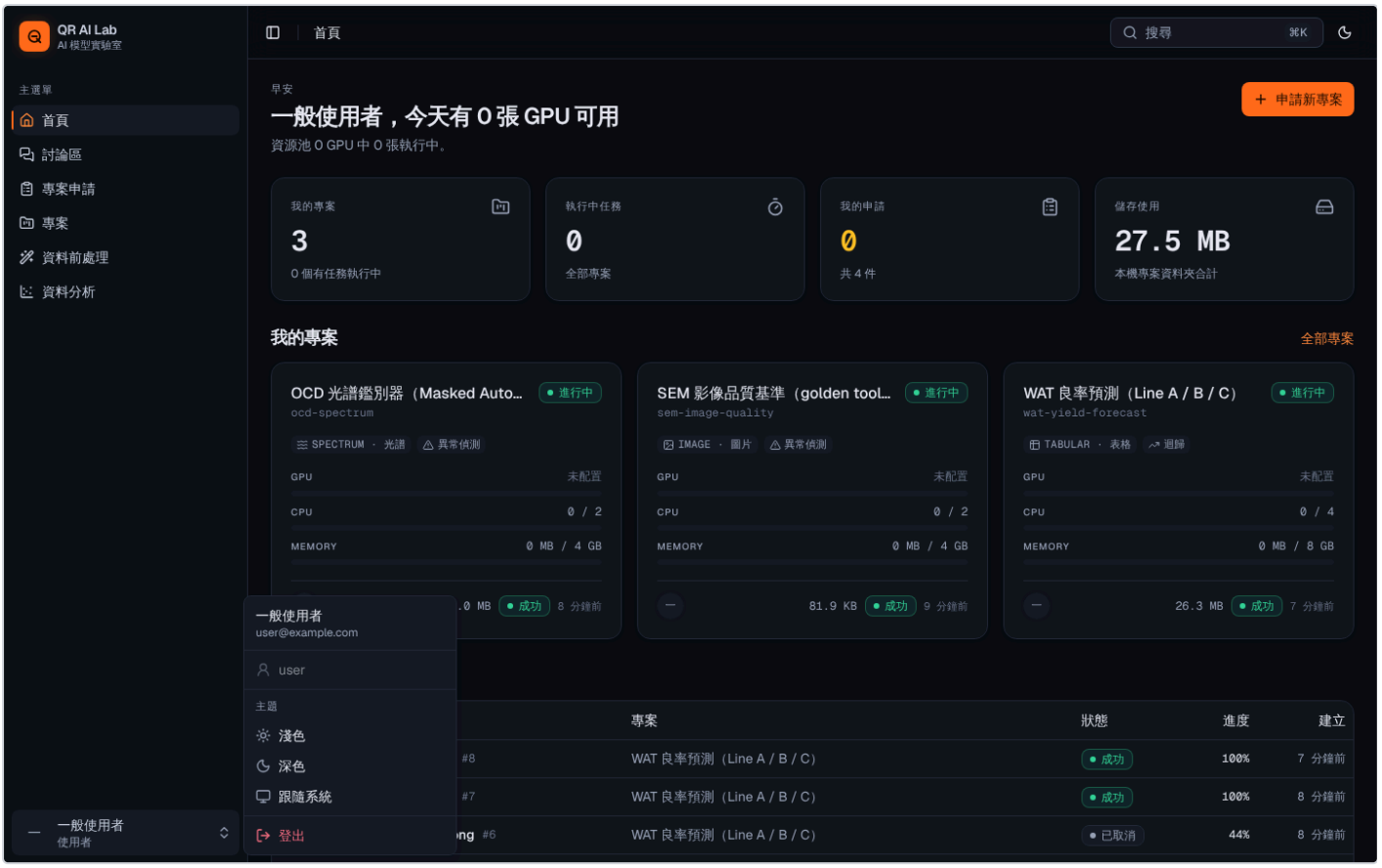



任務 › #1
wat-yield-forecast
WAT 良率預測（Line A / B / C）
進行中
用 WAT 電性參數（Vth、Idsat、Rs、CD、膜厚、量測溫度）預測 lot 良率，提前找出低良率 lot。目前以人工看 SPC 圖，希望 MAE < 1%。
設定
TABULAR · 表格
迴歸
總覽
資料來源
任務
模型
成員
設定
#1 · 訓練 · 自動（AutoML）
automl-wat-yield-v1
建立者
一般使用者 · 10 分鐘前
複製設定
成功
訓練
8s
0 GPU
2 CPU [0,1]
2 GB
概況
Log
產物
設定
模型家族
XGBoost
tabular.xgboost

14. 登出

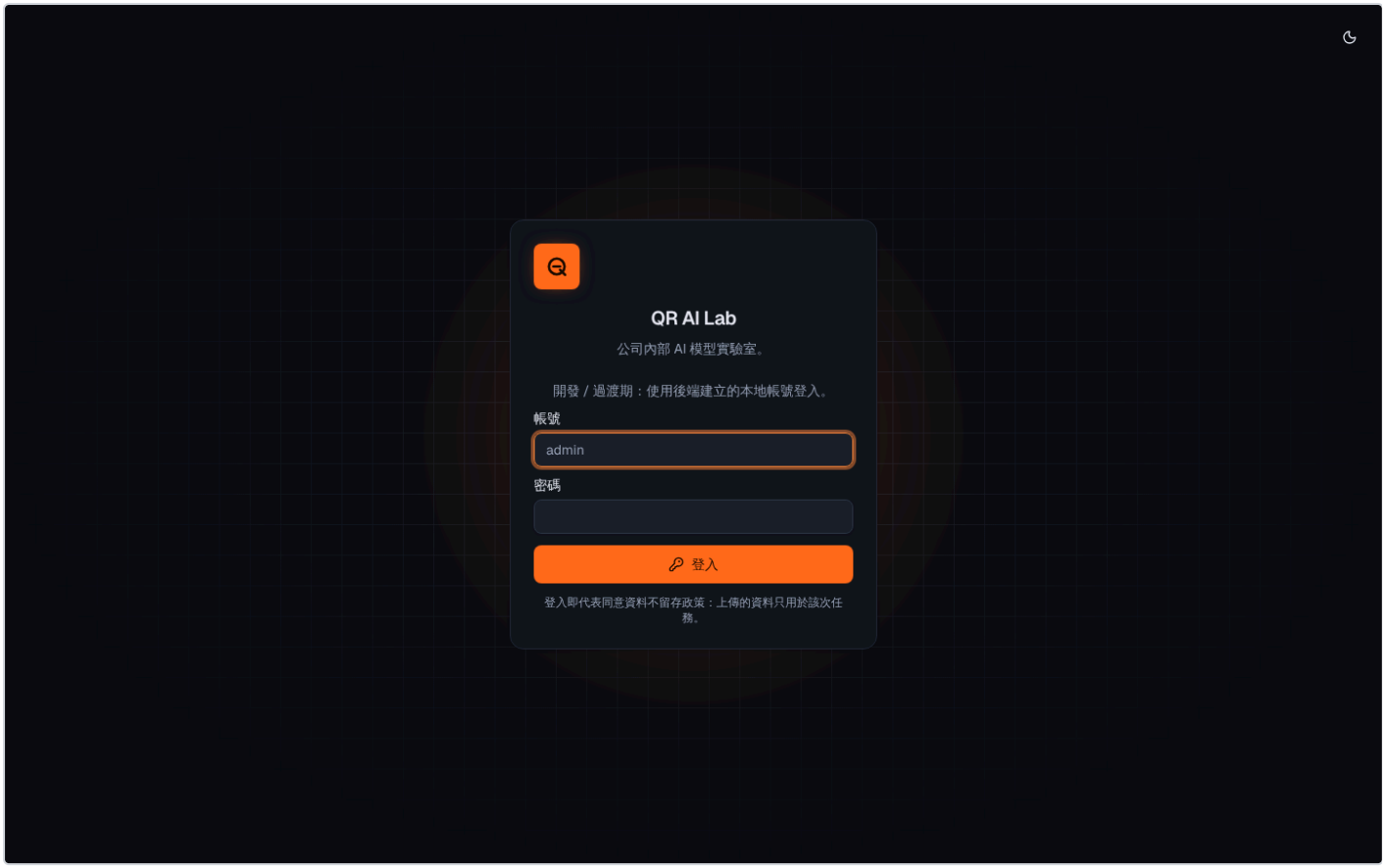
使用者選單登出 → 回到登入頁；受保護頁面由 proxy.ts 導向 /login?next=。

116 使用者選單：主題、登出 dark



117 登出後回到登入頁；受保護頁面由 proxy.ts 導向 /login?next=

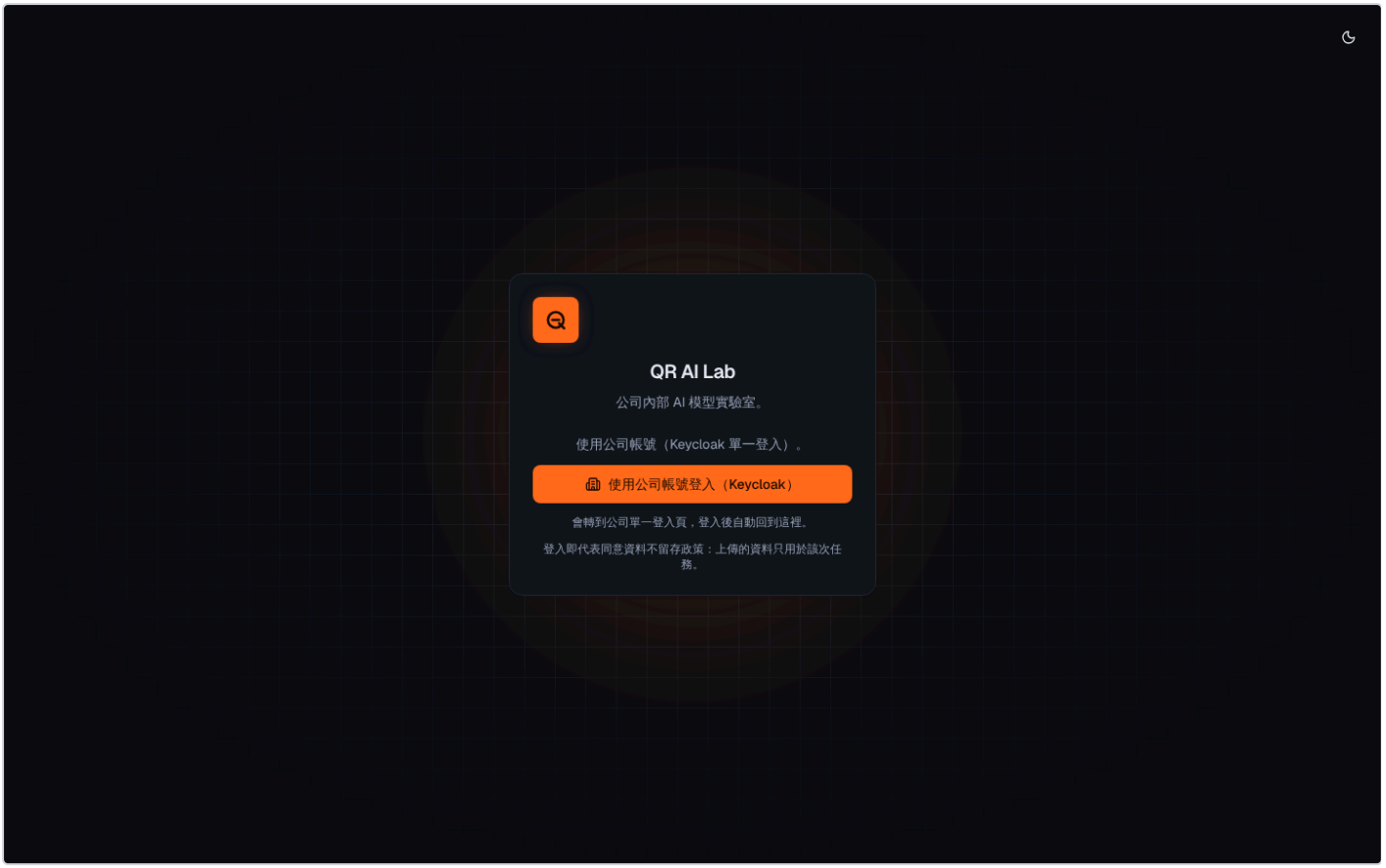
dark



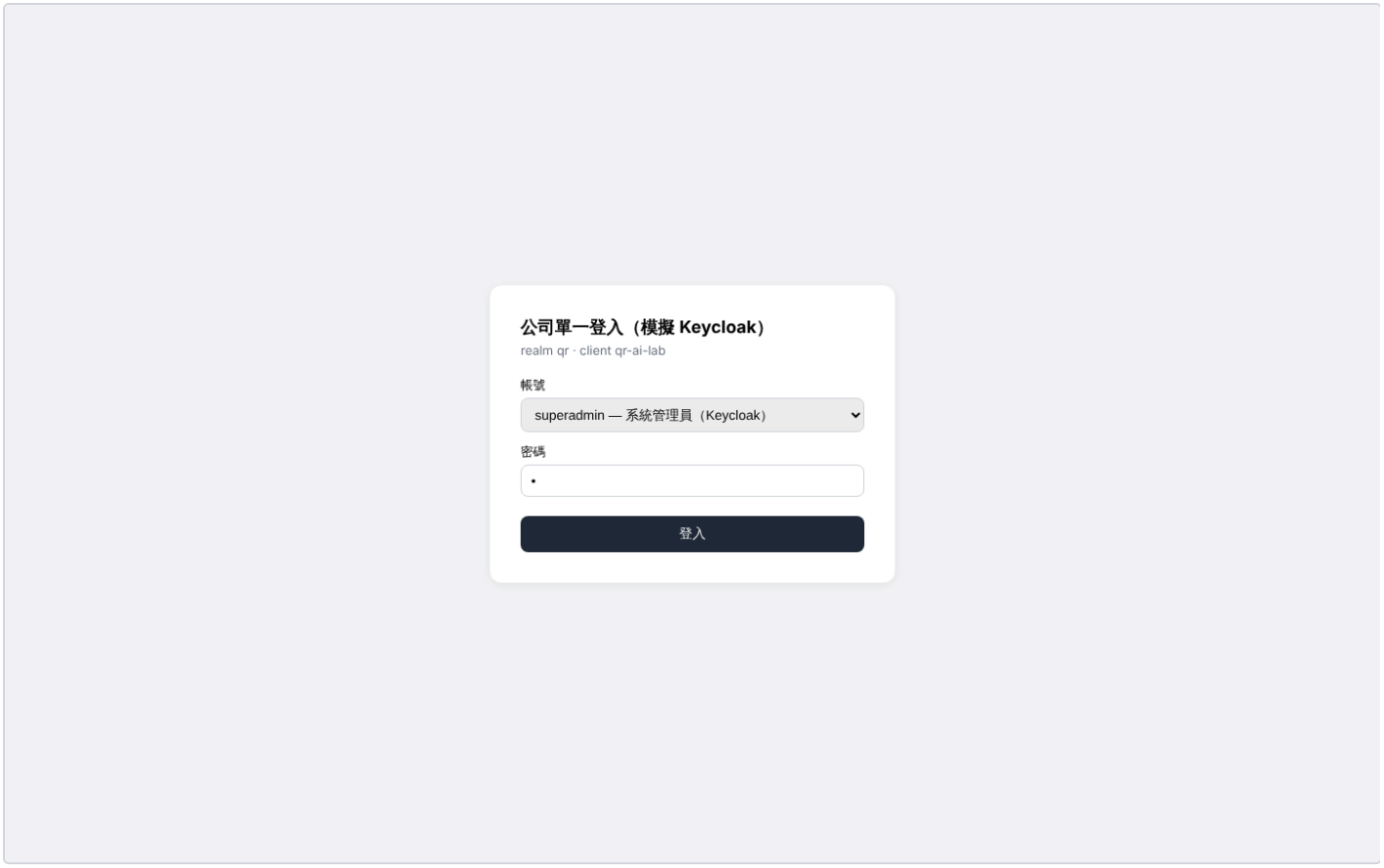
15. Keycloak 登入（AUTH_MODE=keycloak）

前後端切到 keycloak 模式並指向一個模擬的 realm（scripts/fake_keycloak.py）：登入頁只剩公司帳號按鈕 → Authorization Code + PKCE → realm 登入頁 → 回到首頁並依 KEYCLOAK_ROLE_MAP 取得角色 → 使用者頁標記 keycloak → 登出走 end-session；新帳號第一次登入自動建立。

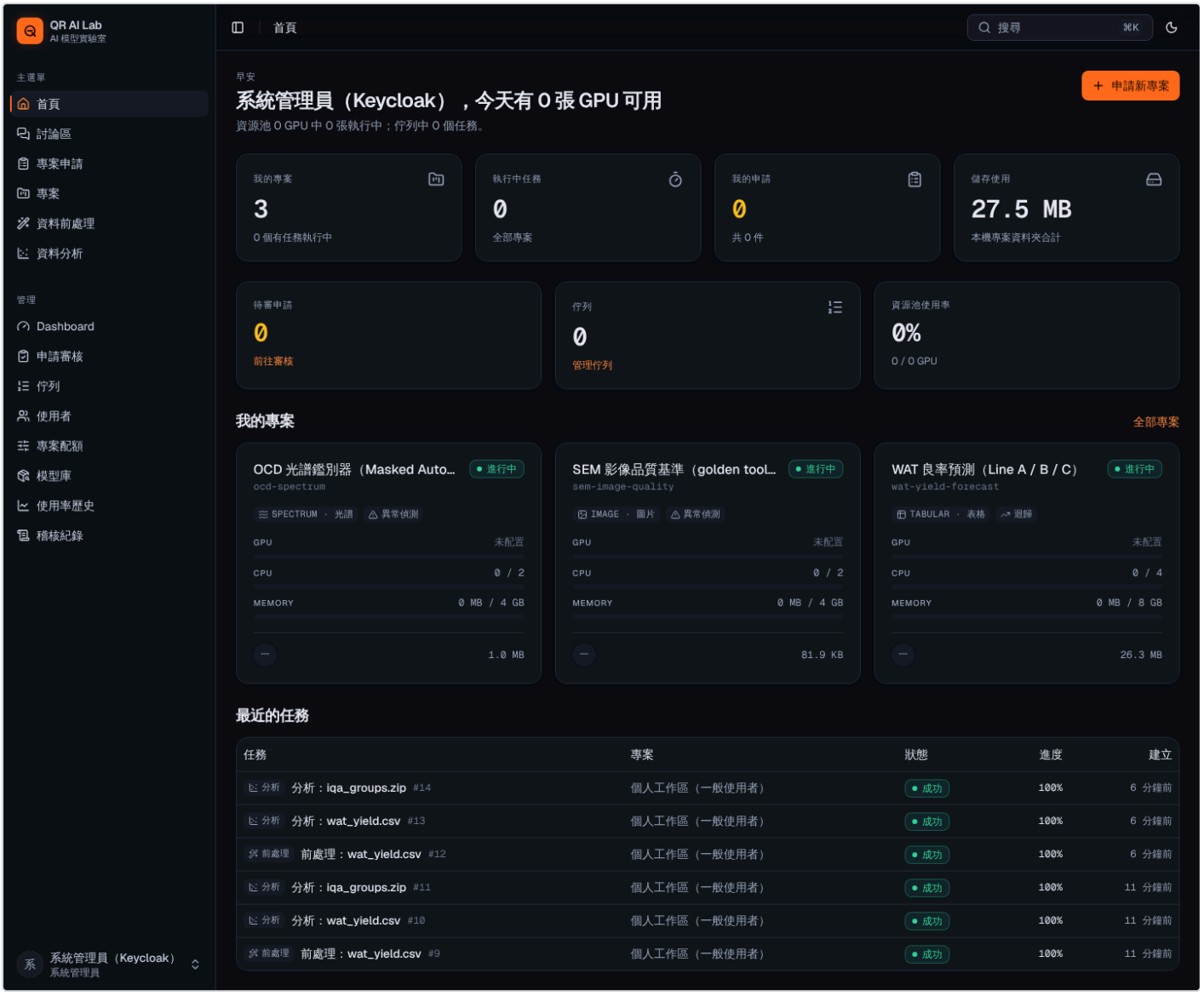
118 AUTH_MODE=keycloak 的登入頁：只有「使用公司帳號登入（Keycloak）」按鈕（AUTH_LOCAL_FALLBACK=1 時多一個摺疊的本地表單） dark



119 Auth.js 帶著 PKCE + state 把瀏覽器導到 realm 的登入頁（這裡是模擬的 Keycloak；正式環境是公司 SSO） dark



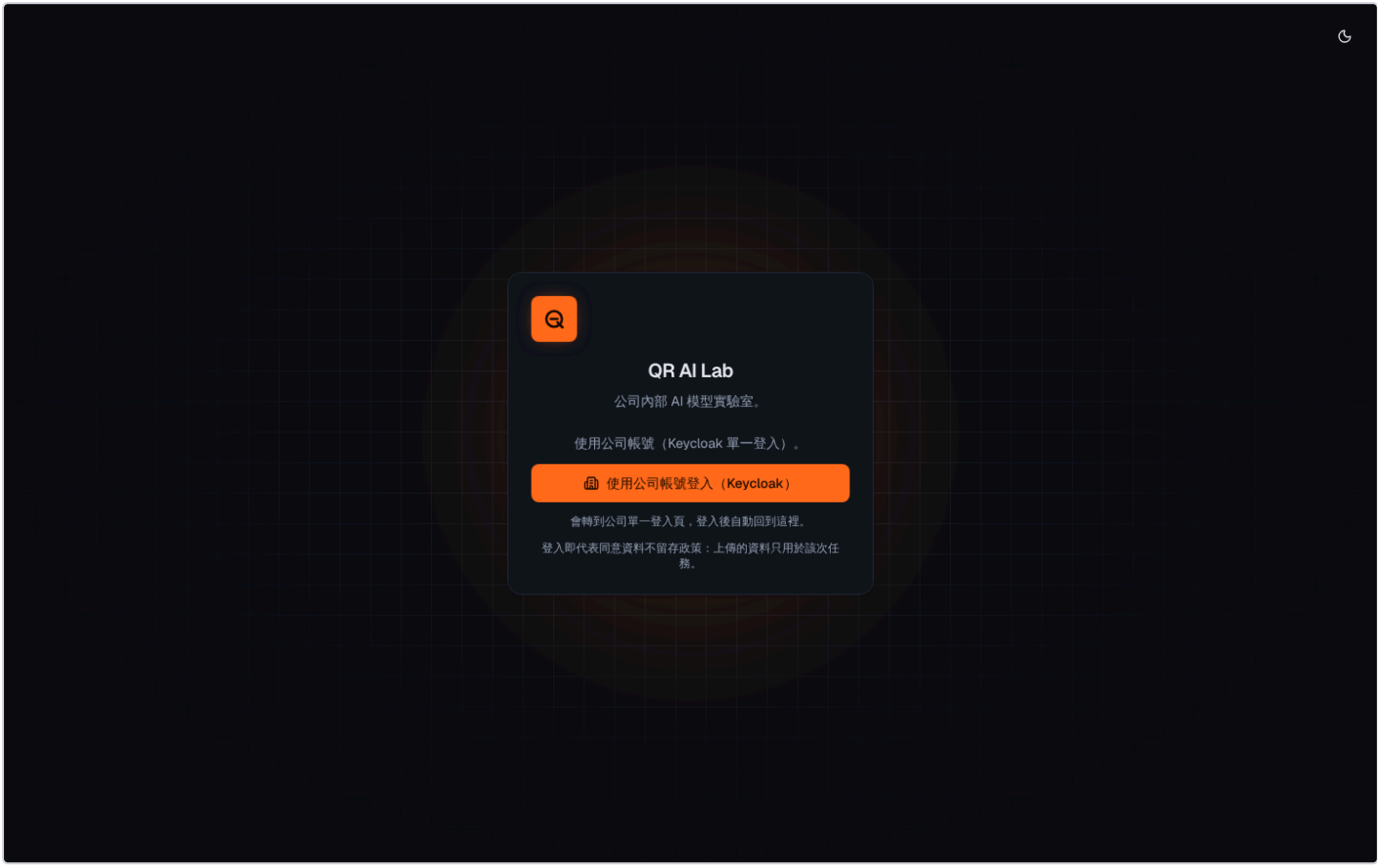
120 回到首頁：Keycloak 的 superadmin realm role 經 KEYCLOAK_ROLE_MAP 對應成系統角色 superadmin；同名的本地帳號被接管（auth_provider=keycloak） dark



121 使用者頁：Keycloak 登入過的帳號標記 keycloak；角色仍可在這裡調整（token 沒帶對應 role 時以這裡為準） dark

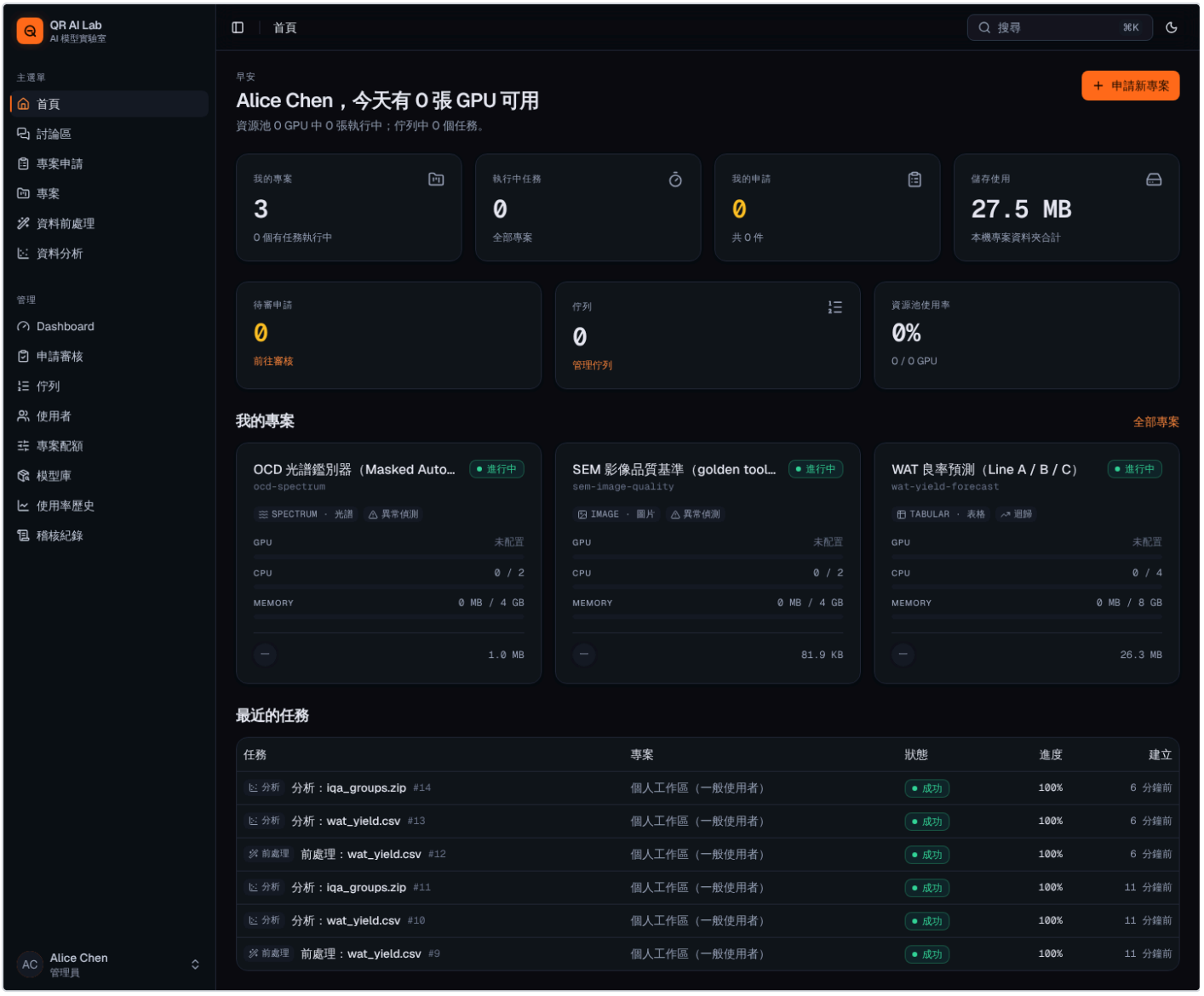


122 登出：清掉 Auth.js session cookie，並導到 Keycloak 的 end-session（id_token_hint）結束 SSO，再回到 /login dark



123 第一次登入的新帳號 alice：users 表自動建立；她的 client role qr-admin 對應到 admin（KEYCLOAK_ROLE_MAP=...,qr-admin:admin），所以看得到「管理」選單

dark



124 alice（admin）的儀表板：資源池唯讀、GPU 卡片沒有重置按鈕 dark

